TKINTER

# CHEAT SHEETS

*A Complete Reference for Python Desktop GUIs*

Windows · Widgets · Text · pack · grid · place · Events & Bindings · Menus & Dialogs · ttk Themes & Styles · Treeview · Canvas · Threading · CustomTkinter · PyInstaller

62 REFERENCE SHEETS

CONTENTS


**FOUNDATIONS**

| | |
|---|---|
| 01–03 | Introduction & Setup |
| 04–06 | Windows |
| 07–15 | Classic Widgets |
| 16–20 | Layout Managers |

**INTERACTION & LOOK**

| | |
|---|---|
| 21–26 | Variables & Events |
| 27–32 | Menus & Dialogs |
| 33–41 | ttk Themed Widgets |
| 42–46 | Canvas |

**BUILDING APPS**

| | |
|---|---|
| 47–49 | Colors, Fonts & Images |
| 50–57 | Application Patterns |
| 58–60 | External Libraries |
| 61–62 | Quick Reference |

TK

# Table of Contents

62 sheets · syntax · examples · notes


## INTRODUCTION & SETUP 01 – 03

- 01 Tkinter Introduction & …
- 02 Installation & Setup
- 03 First App & Program St…

## WINDOWS 04 – 06

- 04 Root Window Setup
- 05 Toplevel Windows & Lif…
- 06 Window State & Positi…

## CLASSIC WIDGETS 07 – 15

- 07 Widget Options & Con…
- 08 Label & Button
- 09 Entry
- 10 Text Widget — Basics
- 11 Text Widget — Tags, M…
- 12 Checkbutton & Radiob…
- 13 Listbox & Scrollbar
- 14 Scale, Spinbox & Optio…
- 15 Frame, LabelFrame & P…

## LAYOUT MANAGERS 16 – 20

- 16 pack()
- 17 grid() — Basics
- 18 grid() — Resizing & Wei…
- 19 place()
- 20 Layout Patterns & Pitfa…

## VARIABLES & EVENTS 21 – 26

- 21 Control Variables
- 22 Command Callbacks
- 23 bind() & Event Sequen…
- 24 The Event Object
- 25 Binding Levels & Virtua…
- 26 Focus & Keyboard Nav…

## MENUS & DIALOGS 27 – 32

- 27 Menu Bars
- 28 Menu Items & Context …
- 29 messagebox
- 30 filedialog
- 31 simpledialog & colorch…
- 32 Custom Modal Dialogs

## TTK THEMED WIDGETS 33 – 41

- 33 ttk Introduction & The…
- 34 ttk.Style — configure &…
- 35 ttk Layouts & Elements
- 36 ttk Core Widgets & Sta…
- 37 Combobox & ttk.Spinb…
- 38 Notebook & ttk.Paned…
- 39 Progressbar, Separator…
- 40 Treeview — Basics
- 41 Treeview — Selection, …

## CANVAS 42 – 46

- 42 Canvas Basics & Shap…
- 43 Canvas Items & Tags
- 44 Canvas Text, Images & …
- 45 Canvas Events & Drag…
- 46 Canvas Scrolling & Ani…

## COLORS, FONTS & IMAGES 47 – 49

- 47 Colors, Reliefs & Borde…
- 48 Fonts
- 49 Images — PhotoImage …

## APPLICATION PATTERNS 50 – 57

- 50 after() & Timers
- 51 Threading & Responsiv…
- 52 Input Validation
- 53 Scrollable Frames
- 54 Application Architectu…
- 55 DPI & Cross-Platform …
- 56 Testing & Debugging
- 57 Packaging & Distributi…

## EXTERNAL LIBRARIES 58 – 60

- 58 Modern Themes — sv-…
- 59 CustomTkinter
- 60 Embedding Matplotlib …

## QUICK REFERENCE 61 – 62

- 61 Quick Reference — Wi…
- 62 Quick Reference — Eve…

# Tkinter Introduction & History

Tcl/Tk · tkinter vs tkinter.ttk · Tk 8.6 vs 9.0 · tix removed 3.13 · alternatives (PyQt, wxPython, Kivy)

## WHAT TKINTER IS

**Tkinter** is Python's standard GUI toolkit: a thin Python layer over **Tcl/Tk**, a cross-platform widget library from the early 1990s. Every Tkinter call is translated into a Tcl command run by an embedded Tcl interpreter. It ships with the official Python installers, so a desktop app needs no third-party packages. Two widget sets live side by side: the **classic** `tkinter` widgets (fully customizable colors) and the **themed** `tkinter.ttk` widgets (native look per platform).

## NOTES

- Tk = the widget toolkit; Tcl = the scripting language that drives it
- `tkinter.ttk` arrived in Python 2.7/3.1 (Tk 8.5)
- Tk 8.6 is the long-standing baseline; Tk 9.0 shipped in 2024
- `tkinter.tix` was deprecated in 3.6 and removed in 3.13
- Docs: docs.python.org/3/library/tkinter.html and tkdocs.com

## CLASSIC VS THEMED WIDGETS

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Hello Tkinter")

# Classic widget: every color and border is an option
tk.Label(root, text="Classic tk.Label",
         bg="navy", fg="white").pack(padx=10, pady=5)

# Themed widget: look comes from the active ttk theme
ttk.Label(root, text="Themed ttk.Label").pack(padx=10, pady=5)
ttk.Button(root, text="Quit", command=root.destroy).pack(pady=5)

# Peek at the Tcl/Tk underneath
print(tk.TkVersion, tk.TclVersion)       # e.g. 8.6 8.6
print(root.tk.call("info", "patchlevel")) # e.g. 8.6.15

root.mainloop()
```

## CODE EXPLAINED

- → `import tkinter as tk` is the conventional alias
- → `ttk` widgets replace most classic widgets with themed versions
- → `bg` / `fg` work on classic widgets; ttk uses styles instead
- → `root.tk.call()` sends a raw Tcl command to the interpreter
- → `mainloop()` hands control to Tk until the window closes

## TIMELINE

| YEAR | MILESTONE |
|---|---|
| 1988 | Tcl created by John Ousterhout |
| 1991 | Tk released on top of Tcl |
| 1994 | Tkinter added to Python |
| 2007 | Tk 8.5 adds the ttk themed widgets |
| 2012 | Tk 8.6 (Python's long-running baseline) |
| 2024 | Tk 9.0 released |

## KEY MODULES

| MODULE | PURPOSE |
|---|---|
| `tkinter` | Core widgets, Tk(), variables |
| `tkinter.ttk` | Themed widgets and Style |
| `tkinter.messagebox` | Standard alert dialogs |
| `tkinter.filedialog` | Open/save file pickers |
| `tkinter.font` | Font objects and metrics |
| `tkinter.constants` | END, LEFT, BOTH, … |

## ALTERNATIVES

| TOOLKIT | TRADE-OFF |
|---|---|
| `PyQt / PySide` | Rich, large; GPL or LGPL |
| `wxPython` | Native widgets; extra install |
| `Kivy` | Touch/mobile; custom look |
| `Dear PyGui` | GPU-drawn, tool-style UIs |
| `Toga (BeeWare)` | Native, mobile targets |
| `Tkinter` | Built in, small, simple |

## ⚠ COMMON MISTAKES

- Naming your script `tkinter.py`, which shadows the real module
- Using Python 2 imports: `Tkinter` and `ttk` are gone
- Expecting `bg=` to recolor ttk widgets

## ✓ TIPS & BEST PRACTICES

- Prefer ttk widgets for a native look; drop to classic for full color control
- Learn from tkdocs.com; it covers modern ttk usage
- Check `tk.TkVersion` when a feature depends on Tk 8.6 vs 9.0

## QUICK SUMMARY

- Python wrapper around Tcl/Tk
- Ships with Python; no install on Windows/macOS
- Classic `tk` + themed `ttk` widget sets
- Event-driven: build widgets, then `mainloop()`

# Installation & Setup



## GETTING TKINTER RUNNING

On **Windows** and **macOS**, the python.org installers include Tcl/Tk, so `import tkinter` works immediately. Most **Linux** distributions split it into a separate package, and Homebrew's Python needs `python-tk`. The fastest health check is `python -m tkinter`, which opens a small demo window showing the Tk version. Inside a virtual environment, Tkinter comes from the base interpreter. It is not on PyPI, so `pip install tkinter` never works.

## NOTES

- A `ModuleNotFoundError: _tkinter` means Tk libraries are missing
- pyenv builds need Tk dev headers installed before `pyenv install`
- conda environments include `tk` by default
- WSL2 on Windows 11 can show Tk windows through WSLg
- Headless servers need a display (or Xvfb) to create windows

## CHECK, INSTALL, IMPORT

```
# 1. Health check: opens a demo window with the Tk version
python -m tkinter

# 2. Install when missing
sudo apt install python3-tk          # Debian / Ubuntu
sudo dnf install python3-tkinter     # Fedora
sudo pacman -S tk                    # Arch
brew install python-tk@3.13          # Homebrew Python

# 3. Versions from Python
python -c "import tkinter as tk; print(tk.TkVersion)"
python -c "import tkinter; tkinter._test()"

# 4. Standard imports in your code
#    import tkinter as tk
#    from tkinter import ttk, messagebox, filedialog
```

## CODE EXPLAINED

- → `python -m tkinter` runs the module's built-in test window
- → Distro packages add the `_tkinter` C extension and Tk libraries
- → Homebrew's formula is versioned to match your Python
- → `tk.TkVersion` is a float like `8.6` or `9.0`
- → Submodules such as `ttk` must be imported explicitly

## INSTALL BY PLATFORM

| PLATFORM | HOW |
|---|---|
| Windows | python.org installer (Tcl/Tk box ticked) |
| macOS | python.org installer includes Tk |
| Homebrew | brew install python-tk@3.x |
| Debian/Ubuntu | apt install python3-tk |
| Fedora | dnf install python3-tkinter |
| conda | Included; conda install tk |

## IMPORT STYLES

| IMPORT | USE |
|---|---|
| `tkinter as tk` | Recommended; clear prefix |
| `ttk` | from tkinter import ttk |
| `*` | Short demos only |
| `constants` | END, LEFT, BOTH, … |
| `tkinter.font` | import … as tkfont |

## VERSION CHECKS

| CHECK | RETURNS |
|---|---|
| `tk.TkVersion` | 8.6 (float) |
| `tk.TclVersion` | 8.6 (float) |
| `info patchlevel` | Full Tk version, '8.6.15' |
| `windowingsystem` | win32 / aqua / x11 |

## ⚠ COMMON MISTAKES

- Running `pip install tkinter` (it is not on PyPI)
- Using `from tkinter import *` and then shadowing names like `Image`
- Forgetting `from tkinter import ttk`; `tk.ttk` is not auto-imported

## ✓ TIPS & BEST PRACTICES

- Run `python -m tkinter` first whenever imports fail
- Use `import tkinter as tk` so tk vs ttk widgets stay obvious
- Rebuild pyenv Pythons after installing `tk-dev`

## QUICK SUMMARY

- Bundled on Windows and macOS installers
- Separate package on most Linux distros
- Test with `python -m tkinter`
- Import as `tk`, plus `ttk` explicitly

# First App & Program Structure

tk.Tk() · mainloop() · root · class App(tk.Tk) · Frame subclass · if __name__ == "__main__"

## YOUR FIRST APPLICATION

Every Tkinter app follows the same shape: create **one** `Tk()` root window, build widgets with a **parent** (the master they live inside), place each one with a **geometry manager**, then call `mainloop()`. `mainloop()` blocks and processes events until the window closes. For anything past a demo, wrap the UI in a **class**, either subclassing `tk.Tk` for the app or `ttk.Frame` for a reusable panel, so widgets and callbacks share state through `self`.

## NOTES

- Only one `Tk()` per program; extra windows use `Toplevel`
- A widget isn't visible until a geometry manager places it
- Code after `mainloop()` runs only after the window closes
- `root.destroy()` closes the window and ends `mainloop()`
- Keep references to widgets you need to read or update later

## CLASS-BASED APP SKELETON

```
import tkinter as tk
from tkinter import ttk

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Greeter")
        self.geometry("300x120")

        self.name = tk.StringVar()
        ttk.Label(self, text="Name:").grid(row=0, column=0, padx=8, pady=8)
        ttk.Entry(self, textvariable=self.name).grid(row=0, column=1)
        ttk.Button(self, text="Greet", command=self.greet).grid(
            row=1, column=0, columnspan=2)
        self.out = ttk.Label(self)
        self.out.grid(row=2, column=0, columnspan=2, pady=8)

    def greet(self):
        self.out.config(text=f"Hello, {self.name.get() or 'world'}!")

if __name__ == "__main__":
    App().mainloop()
```

## CODE EXPLAINED

- → Subclassing `tk.Tk` makes the app object the root window
- → `StringVar` links the Entry text to Python
- → Widgets take the parent as their first argument ( `self` )
- → `command=self.greet` passes the method, not its result
- → The `__main__` guard lets the module be imported safely

## APP LIFECYCLE

| STEP | CALL |
|---|---|
| `Create root` | root = tk.Tk() |
| `Configure` | title(), geometry() |
| `Build widgets` | ttk.Button(root, …) |
| `Place widgets` | pack() / grid() / place() |
| `Run` | root.mainloop() |

## STRUCTURE OPTIONS

| PATTERN | WHEN |
|---|---|
| `Script, no class` | Tiny demos |
| `class App(tk.Tk)` | Single-window app |
| `class Panel(ttk.Frame)` | Reusable section |
| `App + Frame pages` | Multi-screen app |
| `Separate model module` | Logic you can test |

## KEY TERMS

| TERM | MEANING |
|---|---|
| `root` | The main Tk window |
| `master / parent` | Widget that contains another |
| `widget` | Any UI element |
| `geometry manager` | pack, grid or place |
| `event loop` | mainloop() dispatching events |

## ⚠ COMMON MISTAKES

- Writing `command=self.greet()` , which calls it once at startup
- Chaining `x = ttk.Entry(...).grid()` ; `x` becomes `None`
- Creating a second `tk.Tk()` instead of a `Toplevel`

## ✓ TIPS & BEST PRACTICES

- Create and place widgets on separate lines when you keep a reference
- Put each screen in its own `ttk.Frame` subclass
- Keep business logic out of callbacks so it can be tested

## QUICK SUMMARY

- One `Tk()` root per app
- Create widget → place it → `mainloop()`
- Subclass `tk.Tk` or `ttk.Frame` for structure
- Pass callbacks without parentheses

# Root Window Setup

title() · geometry("WxH+X+Y") · minsize() · maxsize() · resizable() · iconphoto() · attributes("-topmost")

## CONFIGURING THE MAIN WINDOW

The root `Tk()` object is a window with its own set of **window-manager (wm) methods**. They control the title bar, size, position, size limits, icon and stacking. `geometry()` takes a string `"WIDTHxHEIGHT+X+Y"` where any part can be left out. Most of these calls are **requests** to the operating system's window manager, so results can vary slightly between Windows, macOS and Linux desktops.

## NOTES

- All wm methods also exist on `Toplevel` windows
- Sizes are in pixels; positions are from the screen's top-left
- Without `geometry()`, the window sizes itself to fit its widgets
- `wm_title()` is the long name for `title()`
- Call with no arguments to read the current value

## ROOT WINDOW SETUP

```
import tkinter as tk

root = tk.Tk()
root.title("Inventory Manager")
root.geometry("640x400+100+80")    # W x H + X + Y
root.minsize(400, 250)             # user can't shrink below this
root.maxsize(1200, 900)
root.resizable(True, False)        # width yes, height no

icon = tk.PhotoImage(file="app.png")
root.iconphoto(True, icon)         # True = default for all windows

root.attributes("-topmost", True) # keep above other windows
root.configure(bg="#1e1e2e")       # classic background color

print(root.title())                # 'Inventory Manager'
root.update_idletasks()
print(root.winfo_width(), root.winfo_height())

root.mainloop()
```

## CODE EXPLAINED

- → `geometry()` sets size and position in one string
- → `minsize` / `maxsize` bound manual resizing
- → `resizable(width, height)` takes two booleans
- → `iconphoto(True, …)` applies the icon to future Toplevels too
- → `update_idletasks()` makes `winfo_*` report real sizes

## GEOMETRY() STRINGS

| STRING | EFFECT |
|---|---|
| `"640x400"` | Size only |
| `"+100+80"` | Position only |
| `"640x400+0+0"` | Size at top-left |
| `"-0-0"` | Bottom-right corner |
| `""` | Back to natural size |

## WINDOW METHODS

| METHOD | PURPOSE |
|---|---|
| `title(s)` | Title bar text |
| `minsize(w, h)` | Smallest size |
| `maxsize(w, h)` | Largest size |
| `resizable(w, h)` | Allow resizing |
| `iconphoto(d, img)` | Window icon (PNG/GIF) |
| `iconbitmap(f)` | Windows .ico icon |

## ATTRIBUTES()

| OPTION | EFFECT |
|---|---|
| `-topmost` | Stay above others |
| `-alpha` | Opacity 0.0–1.0 |
| `-fullscreen` | Full screen |
| `-disabled` | Block input (Windows) |
| `-toolwindow` | Small title bar (Windows) |
| `-type` | Window type hint (X11) |

## ⚠ COMMON MISTAKES

- Writing `geometry(640, 400)`; it takes one string
- Reading `winfo_width()` before the window is drawn (returns 1)
- Using `.ico` with `iconphoto`; it needs a PhotoImage

## ✓ TIPS & BEST PRACTICES

- Set `minsize` so layouts never collapse
- Use `iconbitmap('app.ico')` on Windows for the taskbar icon
- Keep a reference to the icon image object

## QUICK SUMMARY

- `title`, `geometry`, `minsize`, `resizable`
- `"WxH+X+Y"` geometry strings
- `attributes()` for topmost/alpha/fullscreen
- wm calls are requests to the OS

# Toplevel Windows & Lifecycle



## EXTRA WINDOWS WITH TOPLEVEL

A `tk.Toplevel` is an additional window that belongs to the same application and the same event loop as the root. It accepts the same wm methods as the root. `transient(parent)` marks it as a child window (no taskbar entry, stays above its parent). `protocol("WM_DELETE_WINDOW", handler)` intercepts the close button, so you can confirm or clean up first. Destroying the **root** destroys every Toplevel; destroying a Toplevel only removes that window.

## NOTES

- Never create a second `Tk()`; use `Toplevel`
- `destroy()` removes a window and all its child widgets
- `quit()` stops `mainloop()` but leaves windows alive
- `winfo_exists()` checks whether a window is still open
- A Toplevel can be reused by hiding it with `withdraw()`

## OPENING AND CLOSING A TOPLEVEL

```
import tkinter as tk
from tkinter import ttk, messagebox

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.settings = None
        ttk.Button(self, text="Settings…",
                   command=self.open_settings).pack(padx=20, pady=20)
        self.protocol("WM_DELETE_WINDOW", self.on_close)

    def open_settings(self):
        if self.settings and self.settings.winfo_exists():
            self.settings.lift()          # already open: bring to front
            return
        self.settings = win = tk.Toplevel(self)
        win.title("Settings")
        win.transient(self)               # child of the main window
        ttk.Label(win, text="Options go here").pack(padx=30, pady=20)
        ttk.Button(win, text="Close", command=win.destroy).pack(pady=8)
    def on_close(self):
        if messagebox.askokcancel("Quit", "Exit the app?", parent=self):
            self.destroy()
App().mainloop()
```

## CODE EXPLAINED

- → `winfo_exists()` prevents opening duplicate windows
- → `lift()` raises an existing window instead
- → `transient(self)` ties the window to the main one
- → `win.destroy` closes only the Toplevel
- → `protocol` replaces the default close behavior

## LIFECYCLE CALLS

| CALL | EFFECT |
|---|---|
| `Toplevel(parent)` | New window |
| `destroy()` | Remove window + children |
| `withdraw()` | Hide, keep alive |
| `deiconify()` | Show again |
| `quit()` | Exit mainloop only |

## PROTOCOL() NAMES

| NAME | WHEN |
|---|---|
| `WM_DELETE_WINDOW` | Close button clicked |
| `WM_TAKE_FOCUS` | WM gives focus (X11) |
| `WM_SAVE_YOURSELF` | Session save (old X11) |

## DESTROY VS QUIT

| ASPECT | DESTROY() / QUIT() |
|---|---|
| `Windows` | Removed / stay open |
| `mainloop` | Ends / ends |
| `Widgets usable` | No / yes |
| `Typical use` | Close app / embedded loops |

## ⚠ COMMON MISTAKES

- Opening a new Toplevel on every click without checking
- Using `quit()` to close; windows remain on screen
- Forgetting `self.destroy()` inside a custom close handler

## ✓ TIPS & BEST PRACTICES

- Store the Toplevel on `self` so you can reuse or check it
- Pass `parent=` to dialogs so they center on the right window
- Use `transient()` for tool and settings windows

## QUICK SUMMARY

- `Toplevel` = extra window, same app
- `transient()` for child windows
- `protocol("WM_DELETE_WINDOW", fn)` intercepts close
- `destroy()` removes, `quit()` just stops the loop

# Window State & Positioning

winfo_screenwidth() · centering · state("zoomed") · iconify() · withdraw() · deiconify() · overrideredirect() · attributes("-alpha")

## PLACING AND SHOWING WINDOWS

To place a window precisely, ask Tk for the **screen size** with `winfo_screenwidth()`/`winfo_screenheight()` and compute the position. A window can be **normal**, **iconic** (minimized), **withdrawn** (hidden, no taskbar entry) or **zoomed** (maximized; Windows and macOS). `overrideredirect(True)` removes the title bar and border, which suits splash screens.

## NOTES

- Call `update_idletasks()` before measuring a new window
- On Linux, maximize with `attributes("-zoomed", True)`
- Frameless windows can't be moved or closed by the user
- `winfo_x()` / `winfo_y()` give the window's current position
- Multi-monitor setups report the primary or combined screen

## CENTER, MINIMIZE, HIDE, RESTORE

```
import tkinter as tk

def center(win, w, h):
    win.update_idletasks()
    sw, sh = win.winfo_screenwidth(), win.winfo_screenheight()
    x, y = (sw - w) // 2, (sh - h) // 3
    win.geometry(f"{w}x{h}+{x}+{y}")

root = tk.Tk()
center(root, 500, 300)

print(root.state())            # 'normal'
root.iconify()                 # minimize to taskbar
root.deiconify()               # restore
root.state("zoomed")           # maximize (Windows/macOS)
# Splash screen: no title bar, semi-transparent
splash = tk.Toplevel(root)
splash.overrideredirect(True)
splash.attributes("-alpha", 0.9)
tk.Label(splash, text="Loading…", padx=40, pady=30).pack()
center(splash, 200, 100)
root.withdraw()                # hide main window meanwhile
root.after(1500, lambda: (splash.destroy(), root.deiconify()))
root.mainloop()
```

## CODE EXPLAINED

→ Screen size minus window size, halved, gives a centered origin

→ Dividing height by 3 sits slightly above center

→ `state()` reads or sets normal/iconic/withdrawn/zoomed

→ `overrideredirect(True)` removes the window decorations

→ `after(1500, …)` swaps splash for main window

## WINDOW STATES

| STATE() | MEANING |
|---|---|
| `normal` | Visible, regular |
| `iconic` | Minimized |
| `withdrawn` | Hidden entirely |
| `zoomed` | Maximized (Win/mac) |
| `icon` | Shown as icon (rare) |

## SCREEN QUERIES

| METHOD | RETURNS |
|---|---|
| `winfo_screenwidth()` | Screen width px |
| `winfo_screenheight()` | Screen height px |
| `winfo_x(), winfo_y()` | Window position |
| `winfo_rootx()` | Widget x on screen |
| `winfo_pointerxy()` | Mouse position |

## SHOW / HIDE

| CALL | EFFECT |
|---|---|
| `iconify()` | Minimize |
| `deiconify()` | Restore / show |
| `withdraw()` | Hide completely |
| `lift()` | Raise to front |
| `lower()` | Send behind |

## ⚠ COMMON MISTAKES

- Centering before `update_idletasks()`, using 1×1 sizes
- Calling `state("zoomed")` on Linux, which raises `TclError`
- Hiding the only window with `withdraw()` and losing the app

## ✓ TIPS & BEST PRACTICES

- Make `center()` a helper you reuse for dialogs
- Withdraw the root while a splash or login window is shown
- Save and restore `geometry()` between sessions

## QUICK SUMMARY

- `winfo_screenwidth/height` for positioning
- `iconify`, `deiconify`, `withdraw`, `state()`
- `overrideredirect(True)` for frameless windows
- `-alpha` attribute for transparency

Sheet 07

# Widget Options & Configuration

configure() · cget() · widget["text"] · keys() · option_add() · winfo_class() · standard options

## HOW WIDGET OPTIONS WORK

Every widget is configured through **options** (keyword arguments) such as `text`, `width` or `state`. Set them when you create the widget, change them later with `configure()` (alias `config()`), and read them with `cget()` or dictionary syntax `widget["text"]`. `keys()` lists every option a widget supports. `option_add()` sets app-wide **defaults** through Tk's option database, which saves repeating the same font or color on every widget.

## NOTES

- `config()` with no arguments returns a dict of all options
- Values come back as strings or Tcl objects; convert if needed
- Units: plain ints are pixels; `"2c"`, `"1i"`, `"10p"` also work
- `width` on text widgets is in characters, not pixels
- ttk widgets support fewer appearance options (use styles)

## SET, CHANGE AND READ OPTIONS

```
import tkinter as tk

root = tk.Tk()
root.option_add("*Font", "Helvetica 11")        # default for all widgets
root.option_add("*Button.relief", "groove")      # only classic Buttons

btn = tk.Button(root, text="Save", width=12, state="disabled")
btn.pack(padx=10, pady=10)

btn.configure(text="Save now", state="normal")  # change several at once
btn["fg"] = "darkgreen"                         # dict-style set

print(btn.cget("text"))        # 'Save now'
print(btn["width"])            # 12
print(btn.winfo_class())       # 'Button'
print("command" in btn.keys()) # True

info = btn.configure("relief")
print(info)    # ('relief', 'relief', 'Relief', <raised>, 'groove')

root.mainloop()
```

## CODE EXPLAINED

- → `option_add` patterns: `*Font` for all, `*Button.relief` per class
- → `configure()` accepts any number of keyword options
- → Item assignment `btn["fg"] = …` is shorthand for `configure`
- → `winfo_class()` gives the class name used in option patterns
- → `configure("relief")` returns name, db name, class, default, value

## COMMON OPTIONS

| OPTION | MEANING |
|---|---|
| `text` | Displayed string |
| `width / height` | Size (chars or px) |
| `state` | normal / disabled |
| `bg / fg` | Background / text color |
| `font` | Font spec |
| `cursor` | Mouse pointer shape |

## READING & WRITING

| FORM | DOES |
|---|---|
| `w.configure(k=v)` | Set options |
| `w.config(k=v)` | Same (alias) |
| `w["k"] = v` | Set one |
| `w.cget("k")` | Read one |
| `w["k"]` | Read one |
| `w.keys()` | List option names |

## SCREEN UNITS

| VALUE | UNIT |
|---|---|
| `10` | Pixels |
| `"2c"` | Centimeters |
| `"5m"` | Millimeters |
| `"1i"` | Inches |
| `"12p"` | Points (1/72 in) |

## ⚠ COMMON MISTAKES

- Using `bg=` on ttk widgets (raises `TclError: unknown option`)
- Comparing `w["width"] == "12"` when it returns an int
- Misspelling an option; Tk raises instead of ignoring it

## ✓ TIPS & BEST PRACTICES

- Set fonts and colors once with `option_add` before creating widgets
- Use `keys()` in the REPL to explore any widget
- Group related changes in a single `configure()` call

## QUICK SUMMARY

- Options are keyword args on create
- `configure()` to change, `cget()` / `[]` to read
- `option_add()` for app-wide defaults
- `keys()` lists everything a widget supports

# Label & Button



## SHOWING TEXT AND TAKING CLICKS

A `Label` displays text or an image; it can show a fixed string with `text=` or a live value through `textvariable=`. A `Button` runs a **callback** when clicked, given by `command=` with no arguments. Both share many options: `image` with `compound` to mix picture and text, `anchor` and `justify` for alignment, `wraplength` for automatic line wrapping, and `underline` to mark a keyboard mnemonic. A disabled Button greys out and ignores clicks.

## NOTES

- `textvariable` beats `text` when both are given
- `wraplength` is in screen units (pixels by default)
- `invoke()` runs a Button's command as if clicked
- `Message` is an older wrapping label; prefer `wraplength`
- `ttk.Button` has no `bg`; use a style instead

## LABELS AND BUTTONS TOGETHER

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
count = tk.IntVar(value=0)

ttk.Label(root, text="Clicks:", font=("Segoe UI", 12)).grid(row=0, column=0)
ttk.Label(root, textvariable=count, width=5,
          anchor="e").grid(row=0, column=1)

def add():
    count.set(count.get() + 1)
    reset.config(state="normal")

ttk.Button(root, text="Add", underline=0, command=add).grid(row=1, column=0)
reset = ttk.Button(root, text="Reset", state="disabled",
                   command=lambda: count.set(0))
reset.grid(row=1, column=1)
root.bind("<Alt-a>", lambda e: add())       # Alt+A, matches underline

tk.Label(root, text="A long help message that wraps neatly "
         "inside two hundred pixels.", wraplength=200,
         justify="left").grid(row=2, column=0, columnspan=2, pady=6)

root.mainloop()
```

## CODE EXPLAINED

- → `textvariable=count` updates the label automatically
- → `anchor="e"` right-aligns text inside the label's width
- → `underline=0` draws the mnemonic but doesn't bind the key
- → A `lambda` adapts the bind event argument to `add()`
- → `wraplength` plus `justify` controls multi-line text

## LABEL OPTIONS

| OPTION | PURPOSE |
|---|---|
| `text` | Static string |
| `textvariable` | Live value |
| `image` | PhotoImage to show |
| `compound` | Image position vs text |
| `wraplength` | Wrap width |

## BUTTON OPTIONS

| OPTION | PURPOSE |
|---|---|
| `command` | Callback on click |
| `state` | normal / disabled |
| `underline` | Mnemonic index |
| `default` | active = default button |
| `width` | Width in characters |

## ANCHOR VALUES

| VALUE | POSITION |
|---|---|
| `n` / `s` | Top / bottom center |
| `e` / `w` | Right / left middle |
| `ne` `nw` `se` `sw` | Corners |
| `center` | Middle (default) |

## ⚠ COMMON MISTAKES

- Expecting `underline` to bind the key by itself
- Changing `text` when `textvariable` is also set
- Using `justify` to align a single line (use `anchor`)

## ✓ TIPS & BEST PRACTICES

- Prefer `ttk.Label` / `ttk.Button` for native styling
- Disable buttons until their action makes sense
- Use `invoke()` to trigger buttons from key bindings

## QUICK SUMMARY

- Label: `text` or `textvariable`, optional `image`
- Button: `command=` callback, `state`
- `anchor` aligns, `justify` aligns lines
- `wraplength` wraps long text

# Entry



## SINGLE-LINE TEXT INPUT

An `Entry` holds one line of editable text. Read it with `get()`, and modify it with `insert(index, text)` and `delete(first, last)`. Indices are character positions, or the names `0`, `"end"` (also `tk.END`), `"insert"` (the cursor) and `"sel.first"`/`"sel.last"`. Linking a `StringVar` through `textvariable=` is often cleaner than calling `get()`. `show="*"` masks passwords, and `state="readonly"` allows selecting and copying but not editing.

## NOTES

- `get()` always returns a `str`; convert numbers yourself
- `delete(0, "end")` clears the whole field
- `width` is in average characters, not pixels
- `readonly` keeps the text selectable; `disabled` doesn't
- Validation on each keystroke is covered on Sheet 52

## READING, EDITING AND MASKING

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
user = tk.StringVar(value="guest")

ttk.Label(root, text="User").grid(row=0, column=0, sticky="w")
name = ttk.Entry(root, textvariable=user, width=24)
name.grid(row=0, column=1, pady=4)

ttk.Label(root, text="Password").grid(row=1, column=0, sticky="w")
pw = ttk.Entry(root, show="•", width=24)
pw.grid(row=1, column=1, pady=4)

def submit(event=None):
    print(user.get(), len(pw.get()))
    pw.delete(0, "end")               # clear password field
    name.select_range(0, "end")       # highlight user name
    name.icursor("end")               # cursor to end
    name.focus_set()

pw.bind("<Return>", submit)
ttk.Button(root, text="Log in", command=submit).grid(row=2, column=1)
name.insert("end", "_01")             # 'guest_01'
root.mainloop()
```

## CODE EXPLAINED

- → `textvariable` keeps the Entry and `user` in sync
- → `show="•"` displays a mask character instead of the text
- → `event=None` lets one function serve both bind and command
- → `select_range` + `icursor` mimic native field behavior
- → `insert("end", …)` appends to the current text

## ENTRY METHODS

| METHOD | DOES |
|---|---|
| `get()` | Current text |
| `insert(i, s)` | Insert at index |
| `delete(a, b)` | Remove range |
| `icursor(i)` | Move cursor |
| `select_range(a, b)` | Select text |

## INDEX NAMES

| INDEX | POSITION |
|---|---|
| `0` | Start |
| `"end"` | After last char |
| `"insert"` | Cursor position |
| `"sel.first"` | Selection start |
| `"sel.last"` | Selection end |

## USEFUL OPTIONS

| OPTION | EFFECT |
|---|---|
| `show` | Mask character |
| `state` | normal/readonly/disabled |
| `justify` | Text alignment |
| `textvariable` | Linked StringVar |
| `exportselection` | Share selection (0 = no) |

## ⚠ COMMON MISTAKES

- Calling `delete(0)` and expecting the field to clear
- Doing math on `get()` without `int()` / `float()`
- Setting `state="disabled"` when users still need to copy

## ✓ TIPS & BEST PRACTICES

- Use a `StringVar` when several widgets show the same value
- Bind `<Return>` to submit forms
- Set `exportselection=0` when many Entries share a window

## QUICK SUMMARY

- `get`, `insert`, `delete(0, "end")`
- Indices: ints, `"end"`, `"insert"`, `sel.*`
- `show=` masks, `readonly` locks editing
- Link with `textvariable=StringVar`

# Text Widget — Basics



## MULTI-LINE TEXT EDITING

The `Text` widget edits many lines of text and is powerful enough to build a small editor. Positions are **text indices** written as `"line.column"` strings: lines start at **1**, columns at **0**, so the very start is `"1.0"`. `"end"` is just after the last character, and Text always keeps an extra trailing newline there. Indices accept modifiers such as `"insert linestart"` or `"end-1c"`. `undo=True` turns on the built-in undo stack.

## NOTES

- `get("1.0", "end-1c")` drops the automatic final newline
- `state="disabled"` makes Text read-only (also blocks `insert`)
- Re-enable with `state="normal"` before changing a read-only Text
- `wrap` can be `"none"`, `"char"` or `"word"`
- Attach a Scrollbar for long content (Sheet 13)

## INSERT, READ, REPLACE, UNDO

```
import tkinter as tk

root = tk.Tk()
text = tk.Text(root, width=50, height=10, wrap="word",
               undo=True, font=("Consolas", 11))
text.pack(fill="both", expand=True)

text.insert("1.0", "Line one\nLine two\n")
text.insert("end", "Appended line")

content = text.get("1.0", "end-1c")      # everything, no extra \n
first = text.get("1.0", "1.end")         # 'Line one'
lines = int(text.index("end-1c").split(".")[0])

text.delete("2.0", "3.0")                # remove line 2
text.replace("1.5", "1.8", "ONE")        # 'Line ONE'
text.edit_separator()                    # mark an undo step
def log(msg):                            # read-only output pattern
    text.config(state="normal")
    text.insert("end", msg + "\n")
    text.see("end")
    text.config(state="disabled")
root.bind("<Control-z>", lambda e: text.edit_undo())
root.mainloop()
```

## CODE EXPLAINED

- → `"1.0"` is line 1, column 0: the first character
- → `"end-1c"` steps back over the trailing newline
- → `index()` converts any index to `"line.col"` form
- → `replace()` (Tk 8.6+) deletes and inserts in one step
- → `see("end")` scrolls so the newest line is visible

## INDEX FORMS

| INDEX | MEANS |
|---|---|
| `"1.0"` | Line 1, column 0 |
| `"3.end"` | End of line 3 |
| `"end"` | After final newline |
| `"insert"` | Cursor position |
| `"@x,y"` | Char under pixel |

## INDEX MODIFIERS

| MODIFIER | MOVES |
|---|---|
| `+5c / -1c` | By characters |
| `+2l / -1l` | By lines |
| `linestart` | Start of line |
| `lineend` | End of line |
| `wordstart` | Start of word |

## UNDO / MODIFIED

| METHOD | DOES |
|---|---|
| `edit_undo()` | Undo last step |
| `edit_redo()` | Redo |
| `edit_separator()` | End an undo step |
| `edit_reset()` | Clear undo stack |
| `edit_modified()` | Get/set dirty flag |

## ⚠ COMMON MISTAKES

- Using `"0.0"` or `"1.1"` as the start (it's `"1.0"`)
- Saving `get("1.0", "end")` and gaining a newline each time
- Inserting into a disabled Text and wondering why nothing happens

## ✓ TIPS & BEST PRACTICES

- Track unsaved changes with `<<Modified>>` and `edit_modified(False)`
- Use the disable/insert/re-disable pattern for log panes
- Prefer `wrap="word"` for prose, `"none"` for code

## QUICK SUMMARY

- Indices are `"line.column"`, from `"1.0"`
- `get`, `insert`, `delete`, `replace`
- `"end-1c"` excludes the trailing newline
- `undo=True` enables `edit_undo` / `edit_redo`

# Text Widget — Tags, Marks & Search



## FORMATTING AND FINDING TEXT

**Tags** are named styles applied to ranges of a Text widget: configure a tag's look once with `tag_configure()`, then attach it to ranges with `tag_add()` or as an extra argument to `insert()`. Tags can also receive events through `tag_bind()`, which is how clickable links are made. **Marks** are named positions that float with edits; `"insert"` and `"current"` are built-in marks. `search()` finds text (optionally with regex) and returns the index of the match.

## NOTES

- The `"sel"` tag is the user's selection
- Later-created tags win when styles overlap; reorder with `tag_raise()`
- Marks have gravity: `"right"` (default) or `"left"`
- `search()` returns `""` when nothing matches
- `window_create()` embeds a widget inside the text

## HIGHLIGHT, LINK AND SEARCH

```
import tkinter as tk, webbrowser

text = tk.Text(width=60, height=12); text.pack()
text.tag_configure("h1", font=("Helvetica", 16, "bold"))
text.tag_configure("hit", background="yellow")
text.tag_configure("link", foreground="blue", underline=True)

text.insert("end", "Report\n", "h1")
text.insert("end", "See the docs for details. More docs here.\n")
text.insert("end", "tkdocs.com", ("link",))
text.tag_bind("link", "<Button-1>",
              lambda e: webbrowser.open("https://tkdocs.com"))
def highlight(word):
    text.tag_remove("hit", "1.0", "end")
    start = "1.0"
    while True:
        count = tk.IntVar()
        pos = text.search(word, start, "end", nocase=True, count=count)
        if not pos: break
        end = f"{pos}+{count.get()}c"
        text.tag_add("hit", pos, end)
        start = end
highlight("docs"); text.mark_set("insert", "2.0")
text.mainloop()
```

## CODE EXPLAINED

- → Third `insert` argument applies one or more tags
- → `tag_bind` makes tagged text respond to clicks
- → `search(..., count=var)` reports the match length
- → Loop from the end of each match to find the next
- → `mark_set("insert", …)` moves the cursor

## TAG METHODS

| METHOD | DOES |
|---|---|
| `tag_configure()` | Set tag style |
| `tag_add(t, a, b)` | Apply to range |
| `tag_remove(t, a, b)` | Remove from range |
| `tag_ranges(t)` | All tagged ranges |
| `tag_bind(t, ev, fn)` | Events on tag |

## TAG OPTIONS

| OPTION | EFFECT |
|---|---|
| `font` | Font for range |
| `foreground` | Text color |
| `background` | Highlight color |
| `underline` | Underline |
| `lmargin1` | Left indent |

## SEARCH() OPTIONS

| OPTION | EFFECT |
|---|---|
| `nocase=True` | Ignore case |
| `regexp=True` | Tcl regex pattern |
| `backwards=True` | Search upward |
| `count=IntVar` | Match length |
| `stopindex` | Search limit |

## ⚠ COMMON MISTAKES

- Forgetting `"end"` as stopindex, so search wraps forever
- Passing a tag string where a tuple is needed in loops
- Expecting Python regex syntax; `regexp=True` uses Tcl's

## ✓ TIPS & BEST PRACTICES

- Clear old highlights with `tag_remove` before re-searching
- Use tags for syntax highlighting and log levels
- Change the `"current"` cursor on `<Enter>` of link tags

## QUICK SUMMARY

- Tags = named styles on ranges
- `tag_bind` for clickable text
- Marks = floating positions ( `insert` , `current` )
- `search()` returns an index or `""`

# Checkbutton & Radiobutton

Checkbutton · onvalue · offvalue · BooleanVar · Radiobutton · value · shared variable · indicatoron

## BOOLEAN AND CHOICE INPUTS

A `Checkbutton` toggles a variable between an **on** and an **off** value (by default `1` and `0`; change them with `onvalue`/`offvalue`). A group of `Radiobuttons` shares **one variable**; each button has its own `value`, and selecting it writes that value to the variable. Because the state lives in the variable, you read the user's choice with `var.get()` rather than querying the widgets, and you can preselect by calling `var.set()`.

## NOTES

- Use `BooleanVar` for simple on/off checkboxes
- Radiobuttons are grouped only by sharing a variable
- Both accept `command=` to react immediately
- `ttk` versions look native; classic ones allow colors
- `indicatoron=False` makes classic radios look like toggle buttons

## OPTIONS PANEL

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
dark = tk.BooleanVar(value=False)
size = tk.StringVar(value="M")
level = tk.StringVar(value="beta")

def show():
    print(dark.get(), size.get(), level.get())
ttk.Checkbutton(root, text="Dark mode", variable=dark,
                command=show).grid(row=0, column=0, columnspan=3, sticky="w")

for col, (label, val) in enumerate([("Small", "S"), ("Medium", "M"),
                                    ("Large", "L")]):
    ttk.Radiobutton(root, text=label, value=val, variable=size,
                    command=show).grid(row=1, column=col, padx=4)
ttk.Checkbutton(root, text="Use beta channel", variable=level,
                onvalue="beta", offvalue="stable").grid(
                row=2, column=0, columnspan=3, sticky="w")
align = tk.StringVar(value="Left")          # classic toolbar-style group
for i, v in enumerate(("Left", "Center", "Right")):
    tk.Radiobutton(root, text=v, value=v, variable=align,
                   indicatoron=False, width=7).grid(row=3, column=i)
root.mainloop()
```

## CODE EXPLAINED

→ `BooleanVar` returns `True` / `False` from `get()`

→ All three Radiobuttons share `size`, so only one is selected

→ The initial `value="M"` preselects Medium

→ `onvalue` / `offvalue` store strings instead of 1/0

→ `indicatoron=False` draws push-button style radios

## CHECKBUTTON

| OPTION | PURPOSE |
|---|---|
| `variable` | Linked var |
| `onvalue` | Value when checked |
| `offvalue` | Value when cleared |
| `command` | Called on toggle |
| `select()/deselect()` | Classic only |

## RADIOBUTTON

| OPTION | PURPOSE |
|---|---|
| `variable` | Shared group var |
| `value` | This button's value |
| `command` | Called on select |
| `indicatoron` | Classic: dot or button |
| `state` | normal / disabled |

## TTK STATE

| CALL | MEANING |
|---|---|
| `instate(['selected'])` | Is checked? |
| `state(['!alternate'])` | Clear 'mixed' |
| `invoke()` | Toggle in code |
| `alternate` | Tri-state look |

## ⚠ COMMON MISTAKES

- Giving each Radiobutton its own variable (they never exclude)
- Leaving the variable unset, so no radio appears selected
- Letting a local variable get garbage-collected; keep a reference

## ✓ TIPS & BEST PRACTICES

- Store vars on `self` in class-based apps
- Build radio groups from a list of (label, value) pairs
- Read choices from variables, never from widgets

## QUICK SUMMARY

- Checkbutton: on/off values in one variable
- Radiobuttons: shared variable, unique `value`
- Read state with `var.get()`
- Preselect with `var.set()`

# Listbox & Scrollbar



## LISTS AND SCROLLING

A `Listbox` shows a list of strings the user can select from. Items are addressed by **index** (0-based) or `"end"`, and `curselection()` returns a tuple of selected indices. `selectmode` controls single or multiple selection. Scrollbars connect in **two directions**: the widget's `yscrollcommand` is set to the scrollbar's `set`, and the scrollbar's `command` is set to the widget's `yview`. The same pairing works for Text, Canvas, Entry (horizontal) and Treeview.

## NOTES

- `listvariable` links a `StringVar` holding a Tcl list
- `<<ListboxSelect>>` fires when the selection changes
- `curselection()` may be empty; always check
- Selections are lost when focus moves unless `exportselection=False`
- For columns or icons, use `ttk.Treeview` instead

## LISTBOX WITH SCROLLBAR

```
import tkinter as tk
from tkinter import ttk

frame = ttk.Frame(tk.Tk(), padding=8)
frame.pack(fill="both", expand=True)

fruits = ["Apple", "Banana", "Cherry", "Date", "Elderberry",
          "Fig", "Grape", "Honeydew", "Kiwi", "Lemon"]
lb = tk.Listbox(frame, height=6, selectmode="extended",
                exportselection=False)
sb = ttk.Scrollbar(frame, orient="vertical", command=lb.yview)
lb.configure(yscrollcommand=sb.set)

lb.grid(row=0, column=0, sticky="nsew")
sb.grid(row=0, column=1, sticky="ns")
frame.rowconfigure(0, weight=1); frame.columnconfigure(0, weight=1)

lb.insert("end", *fruits)
lb.itemconfigure(0, foreground="red")
def on_select(event):
    print([lb.get(i) for i in lb.curselection()])
lb.bind("<<ListboxSelect>>", on_select)
lb.selection_set(2); lb.see(2)          # select and scroll to Cherry
lb.delete("end")                        # remove last item
frame.mainloop()
```

## CODE EXPLAINED

- → Scrollbar `command` → `lb.yview` scrolls the list
- → `yscrollcommand` → `sb.set` moves the scrollbar thumb
- → `sticky="ns"` stretches the scrollbar to full height
- → `curselection()` gives indices; `get(i)` gives text
- → `selection_set` + `see` select in code and scroll into view

## SELECTMODE

| MODE | BEHAVIOR |
|---|---|
| `browse` | One item, drag moves it |
| `single` | One item |
| `multiple` | Click toggles items |
| `extended` | Shift/Ctrl ranges |

## LISTBOX METHODS

| METHOD | DOES |
|---|---|
| `insert(i, *items)` | Add items |
| `delete(a, b)` | Remove items |
| `get(a, b)` | Text of items |
| `curselection()` | Selected indices |
| `selection_clear(a, b)` | Deselect |

## SCROLL WIRING

| SIDE | SETTING |
|---|---|
| `Widget` | yscrollcommand=sb.set |
| `Scrollbar` | command=w.yview |
| `Horizontal` | xscrollcommand / xview |
| `Scrollbar` | orient="horizontal" |

## ⚠ COMMON MISTAKES

- Wiring only one direction, so the thumb never moves
- Indexing `curselection()[0]` when nothing is selected
- Losing the selection when clicking into another Entry

## ✓ TIPS & BEST PRACTICES

- Use `grid` with weights so the list grows with the window
- Prefer `ttk.Scrollbar` for a native look
- Use `<<ListboxSelect>>` instead of `<Button-1>`

## QUICK SUMMARY

- Listbox: index-based string list
- `curselection()` → tuple of indices
- `selectmode` for single vs multi
- Scrollbars wire both directions

# Scale, Spinbox & OptionMenu



## NUMERIC SLIDERS AND CHOOSERS

A `Scale` is a slider for numbers between `from_` and `to` (note the trailing underscore, because `from` is a Python keyword). A `Spinbox` is an Entry with up/down arrows that steps through a numeric range or a fixed list of `values`. An `OptionMenu` is a classic drop-down built from a variable and a list of choices; `ttk.Combobox` (Sheet 37) is the modern alternative. All three can share a Tk variable so the current value is always one `get()` away.

## NOTES

- `Scale` passes the new value as a **string** to its `command`
- `resolution` sets the Scale step; `tickinterval` adds labels
- Spinbox `wrap=True` loops past the ends
- OptionMenu takes the default value and choices positionally
- `ttk.Scale` has no `resolution`; round the value yourself

## VOLUME, QUANTITY AND SIZE

```
import tkinter as tk

root = tk.Tk()
vol = tk.DoubleVar(value=50)
qty = tk.IntVar(value=1)
size = tk.StringVar(value="Medium")

tk.Scale(root, from_=0, to=100, resolution=5, orient="horizontal",
         label="Volume", tickinterval=25, length=240,
         variable=vol, command=lambda v: print("vol", float(v))
         ).pack(padx=10, pady=4)
tk.Spinbox(root, from_=1, to=10, increment=1, width=5,
           textvariable=qty, wrap=True).pack(pady=4)

tk.Spinbox(root, values=("XS", "S", "M", "L", "XL"),
           state="readonly", width=6).pack(pady=4)

opts = ["Small", "Medium", "Large"]
menu = tk.OptionMenu(root, size, size.get(), *opts,
                     command=lambda v: print("size", v))
menu.config(width=10); menu.pack(pady=4)
def reset():
    vol.set(50); qty.set(1); size.set("Medium")
tk.Button(root, text="Reset", command=reset).pack(pady=6)
root.mainloop()
```

## CODE EXPLAINED

- → `from_` / `to` set the range; `resolution=5` snaps to steps
- → `command` receives the value as text, so convert it
- → Spinbox with `values=` cycles through a fixed list
- → OptionMenu args: master, var, default, *choices
- → Setting each variable resets every linked widget

## SCALE OPTIONS

| OPTION | PURPOSE |
|---|---|
| `from_ / to` | Range ends |
| `resolution` | Step size |
| `orient` | horizontal / vertical |
| `tickinterval` | Labelled ticks |
| `length` | Pixel length |

## SPINBOX OPTIONS

| OPTION | PURPOSE |
|---|---|
| `from_ / to` | Numeric range |
| `increment` | Step per click |
| `values` | Fixed list |
| `wrap` | Loop at ends |
| `format` | e.g. "%05.2f" |

## WHICH TO USE

| NEED | WIDGET |
|---|---|
| `Rough number` | Scale |
| `Exact small int` | Spinbox |
| `Pick from list` | Combobox |
| `Few fixed choices` | Radiobuttons |
| `Classic dropdown` | OptionMenu |

## ⚠ COMMON MISTAKES

- Writing `from=0`, a syntax error (use `from_`)
- Doing math on the Scale callback value without `float()`
- Passing the choices as one list to OptionMenu without `*`

## ✓ TIPS & BEST PRACTICES

- Show the live number in a Label via the same variable
- Use `readonly` Spinboxes to prevent invalid typing
- Prefer `ttk.Combobox` for long choice lists

## QUICK SUMMARY

- Scale: slider, `from_` / `to` / `resolution`
- Spinbox: arrows over range or `values`
- OptionMenu: classic drop-down
- Share one Tk variable per value

# Frame, LabelFrame & PanedWindow



## CONTAINERS FOR GROUPING

Container widgets group other widgets so they can be laid out and styled as one unit. A `Frame` is an invisible (or bordered) box. A `LabelFrame` adds a titled border, which suits form sections. A `PanedWindow` splits space between children with **draggable sashes**, like a file browser's sidebar. Containers are the key to good layouts: each Frame can use its own geometry manager independently of its parent, which is how complex screens are built from simple pieces.

## NOTES

- `padding=` on ttk Frames adds inner space
- Classic Frames use `padx` / `pady`, `relief`, `borderwidth`
- PanedWindow children are added with `add()`, not pack/grid
- A Frame shrinks to fit its children unless propagation is off
- `ttk.Panedwindow` is the themed version (Sheet 38)

## FRAMES, LABELFRAME AND PANES

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk(); root.geometry("520x260")
panes = tk.PanedWindow(root, orient="horizontal", sashwidth=6,
                       sashrelief="raised")
panes.pack(fill="both", expand=True)

sidebar = ttk.Frame(panes, padding=6)
for name in ("Inbox", "Sent", "Archive"):
    ttk.Button(sidebar, text=name).pack(fill="x", pady=2)

main = ttk.Frame(panes, padding=8)
info = ttk.LabelFrame(main, text="Contact", padding=8)
info.pack(fill="x")
ttk.Label(info, text="Name").grid(row=0, column=0, sticky="w")
ttk.Entry(info).grid(row=0, column=1, sticky="ew", padx=4)
info.columnconfigure(1, weight=1)
box = tk.Frame(main, relief="sunken", borderwidth=2, height=60)
box.pack(fill="x", pady=8)
panes.add(sidebar, minsize=100)
panes.add(main, stretch="always")
root.mainloop()
```

## CODE EXPLAINED

→ The PanedWindow fills the window; sash is draggable

→ Sidebar Frame uses `pack`, the LabelFrame inside uses `grid`

→ `columnconfigure(1, weight=1)` lets the Entry stretch

→ `relief="sunken"` draws a classic inset border

→ `add(…, minsize=100)` keeps the sidebar from vanishing

## CONTAINERS

| WIDGET | USE |
|---|---|
| `Frame` | Invisible grouping box |
| `LabelFrame` | Titled border group |
| `PanedWindow` | Resizable split |
| `Toplevel` | Separate window |
| `ttk.Notebook` | Tabbed pages |

## FRAME OPTIONS

| OPTION | EFFECT |
|---|---|
| `padding` | Inner space (ttk) |
| `relief` | flat/raised/sunken/groove/ridge |
| `borderwidth` | Border thickness |
| `width / height` | Size if not propagating |
| `labelanchor` | LabelFrame title spot |

## PANEDWINDOW

| CALL | DOES |
|---|---|
| `add(w, …)` | Add a pane |
| `forget(w)` | Remove a pane |
| `sash_coord(i)` | Sash position |
| `sash_place(i, x, y)` | Move a sash |
| `paneconfigure()` | Pane options |

## ⚠ COMMON MISTAKES

- Packing children into a PanedWindow instead of `add()`
- Setting Frame `width` / `height` and seeing no effect (propagation)
- Mixing `pack` and `grid` inside the same Frame

## ✓ TIPS & BEST PRACTICES

- One Frame per logical region of the screen
- Use LabelFrames to group related form fields
- Set `minsize` on panes the user shouldn't collapse

## QUICK SUMMARY

- Frame groups, LabelFrame adds a title
- PanedWindow gives draggable splits
- Each container has its own geometry manager
- Nest frames to build complex layouts

# pack()



## STACKING WIDGETS WITH PACK()

`pack()` places widgets one after another against a **side** of the remaining space in their parent: `top` (default), `bottom`, `left` or `right`. Each packed widget claims a full-width (or full-height) **parcel**. `fill` stretches the widget within its parcel, and `expand=True` lets the parcel grab leftover space when the window grows. Packing **order matters**: widgets packed first get space first and are the last to shrink, so pack toolbars and status bars before the main content.

## NOTES

- `fill` = grow inside the parcel; `expand` = grow the parcel
- `padx` / `pady` add space outside; `ipadx` / `ipady` inside
- `pack_forget()` hides a widget and discards its pack options
- `before=` / `after=` insert relative to an already packed widget
- Best for simple rows, columns and edge-docked bars

## TOOLBAR, STATUS BAR, CONTENT

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.geometry("420x260")

toolbar = ttk.Frame(root, padding=4)
toolbar.pack(side="top", fill="x")
for name in ("New", "Open", "Save"):
    ttk.Button(toolbar, text=name).pack(side="left", padx=2)
ttk.Button(toolbar, text="Help").pack(side="right")

status = ttk.Label(root, text="Ready", relief="sunken", anchor="w")
status.pack(side="bottom", fill="x")        # packed before content

sidebar = tk.Listbox(root, width=14)
sidebar.pack(side="left", fill="y")

editor = tk.Text(root)
editor.pack(side="left", fill="both", expand=True, padx=(4, 0))

status.pack_forget()                        # hide status bar
status.pack(side="bottom", fill="x", before=sidebar)  # show again
root.mainloop()
```

## CODE EXPLAINED

- → Toolbar docks to the top and stretches horizontally
- → Buttons pack left-to-right; Help sits at the far right
- → Status bar is packed before content so it never gets squeezed
- → Only the editor has `expand=True`, so it absorbs extra space
- → `before=sidebar` restores the bar's original packing slot

## PACK() OPTIONS

| OPTION | EFFECT |
|---|---|
| `side` | top / bottom / left / right |
| `fill` | none / x / y / both |
| `expand` | Claim extra space |
| `anchor` | Position in parcel |
| `padx / pady` | Outer spacing |
| `ipadx / ipady` | Inner spacing |

## FILL VS EXPAND

| SETTING | RESULT |
|---|---|
| `neither` | Natural size |
| `fill="x"` | Stretches across parcel |
| `expand=True` | Parcel grows, widget doesn't |
| `both` | Widget grows with window |

## PACK METHODS

| METHOD | DOES |
|---|---|
| `pack_forget()` | Unmap, forget options |
| `pack_info()` | Current options |
| `pack_slaves()` | Packed children |
| `pack_propagate()` | Parent auto-size |
| `before= / after=` | Insert position |

## ⚠ COMMON MISTAKES

- Using `expand=True` without `fill`, then wondering why it doesn't stretch
- Packing the status bar last, so it gets cut off when shrinking
- Mixing `pack` and `grid` for children of the same parent

## ✓ TIPS & BEST PRACTICES

- Pack edge bars first, the flexible area last
- Use a Frame per row when you need several horizontal groups
- Tuple padding `padx=(4, 0)` pads one side only

## QUICK SUMMARY

- Widgets stack against a side
- `fill` stretches, `expand` claims space
- Order of packing decides priority
- Great for bars and simple stacks

# grid() — Basics



## ROWS AND COLUMNS WITH GRID()

`grid()` places widgets in a table of **rows** and **columns** inside their parent. Rows and columns are created on demand and size themselves to the largest widget they hold. A widget can span several cells with `columnspan`/`rowspan`. Inside its cell, a widget is centered unless `sticky` pins it to edges using compass letters: `"w"` aligns left, `"ew"` stretches horizontally, `"nsew"` fills the cell. `grid` is the go-to manager for forms and anything aligned in columns.

## NOTES

- Empty rows/columns take no space
- Row and column numbers needn't be consecutive
- `grid_remove()` hides but remembers options; `grid_forget()` doesn't
- `sticky` only stretches within the cell, not the cell itself
- Cell growth needs `columnconfigure` / `rowconfigure` (Sheet 18)

## A CLASSIC FORM

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
form = ttk.Frame(root, padding=10)
form.grid(row=0, column=0, sticky="nsew")

fields = ["First name", "Last name", "Email"]
entries = {}
for r, label in enumerate(fields):
    ttk.Label(form, text=label + ":").grid(row=r, column=0,
                                           sticky="e", padx=(0, 6), pady=3)
    e = ttk.Entry(form, width=30)
    e.grid(row=r, column=1, sticky="ew", pady=3)
    entries[label] = e

ttk.Label(form, text="Notes:").grid(row=3, column=0, sticky="ne", pady=3)
tk.Text(form, width=30, height=4).grid(row=3, column=1, pady=3)

buttons = ttk.Frame(form)
buttons.grid(row=4, column=0, columnspan=2, sticky="e", pady=(8, 0))
ttk.Button(buttons, text="Cancel").grid(row=0, column=0, padx=4)
ttk.Button(buttons, text="Save").grid(row=0, column=1)
entries["Email"].grid_remove()       # hide…
entries["Email"].grid()              # …and restore with old options
root.mainloop()
```

## CODE EXPLAINED

- → Labels in column 0 align right with `sticky="e"`
- → Entries fill their cell width with `sticky="ew"`
- → `sticky="ne"` keeps the Notes label at the top of its tall row
- → A nested Frame spanning both columns holds the buttons
- → `grid()` with no args re-shows a `grid_remove` d widget

## GRID() OPTIONS

| OPTION | EFFECT |
|---|---|
| `row / column` | Cell position |
| `rowspan` | Span rows |
| `columnspan` | Span columns |
| `sticky` | Edges to stick to |
| `padx / pady` | Outer spacing |

## STICKY VALUES

| VALUE | RESULT |
|---|---|
| `""` | Centered |
| `"w" / "e"` | Left / right |
| `"n" / "s"` | Top / bottom |
| `"ew"` | Stretch horizontally |
| `"ns"` | Stretch vertically |

## HIDE & INSPECT

| METHOD | DOES |
|---|---|
| `grid_remove()` | Hide, keep options |
| `grid_forget()` | Hide, drop options |
| `grid_info()` | Current options |
| `grid_size()` | (columns, rows) |
| `grid_slaves(r, c)` | Widgets in a cell |

## ⚠ COMMON MISTAKES

- Writing `sticky="left"` ; it uses compass letters
- Expecting `sticky="ew"` to widen a column with no weight
- Calling `.grid()` inline and storing the `None` it returns

## ✓ TIPS & BEST PRACTICES

- Build forms in a loop over field names
- Put button rows in their own Frame
- Use `grid_remove` for widgets you toggle often

## QUICK SUMMARY

- Rows × columns, created on demand
- `columnspan` / `rowspan` merge cells
- `sticky` pins or stretches in the cell
- Ideal for forms and aligned layouts

# grid() — Resizing & Weights



## MAKING GRIDS RESIZE PROPERLY

By default, grid rows and columns stay at their natural size when the window grows. To let them grow, give them a **weight** with `columnconfigure(index, weight=n)` and `rowconfigure(index, weight=n)` on the **parent**. Extra space is shared in proportion to the weights. `minsize` sets a minimum pixel size, `pad` adds space, and `uniform` forces columns in the same group to equal width. Remember to configure the root too, or the frame inside it never grows.

## NOTES

- Weight 0 (default) = fixed size
- Configure the parent of the widgets, not the widgets
- A list of indices configures several at once
- Widgets still need `sticky` to fill the growing cell
- `uniform="grp"` + equal weights = equal columns

## A RESIZABLE TWO-PANE LAYOUT

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.geometry("500x300")
root.columnconfigure(0, weight=1)        # let the frame grow
root.rowconfigure(0, weight=1)

frame = ttk.Frame(root, padding=6)
frame.grid(row=0, column=0, sticky="nsew")

frame.columnconfigure(0, weight=1, minsize=120)    # sidebar
frame.columnconfigure(1, weight=3)                 # main: 3× share
frame.rowconfigure(1, weight=1)                    # content row grows
ttk.Label(frame, text="Folders").grid(row=0, column=0, sticky="w")
ttk.Label(frame, text="Messages").grid(row=0, column=1, sticky="w")
tk.Listbox(frame).grid(row=1, column=0, sticky="nsew", padx=(0, 4))
tk.Text(frame).grid(row=1, column=1, sticky="nsew")
bar = ttk.Frame(frame)
bar.grid(row=2, column=0, columnspan=2, sticky="ew", pady=(6, 0))
bar.columnconfigure((0, 1, 2), weight=1, uniform="btn")
for i, t in enumerate(("Reply", "Forward", "Delete")):
    ttk.Button(bar, text=t).grid(row=0, column=i, sticky="ew", padx=2)
root.mainloop()
```

## CODE EXPLAINED

- → Root weights let the outer frame follow the window size
- → Columns share extra width 1:3 (sidebar:main)
- → Only row 1 grows; header and button rows stay fixed
- → `sticky="nsew"` makes widgets fill their growing cells
- → `uniform="btn"` keeps the three buttons the same width

## CONFIGURE OPTIONS

| OPTION | EFFECT |
|---|---|
| `weight` | Share of extra space |
| `minsize` | Minimum size (px) |
| `pad` | Extra space in row/col |
| `uniform` | Equal-size group name |

## WEIGHT EXAMPLES

| WEIGHTS | EXTRA SPACE |
|---|---|
| `0, 0` | None; fixed |
| `1, 0` | All to first |
| `1, 1` | Split evenly |
| `1, 3` | 25% / 75% |
| `2, 1, 1` | 50% / 25% / 25% |

## CHECKLIST

| STEP | CALL |
|---|---|
| `Parent grows` | root weights |
| `Frame fills` | sticky="nsew" |
| `Column grows` | columnconfigure(i, weight=1) |
| `Row grows` | rowconfigure(i, weight=1) |
| `Widget fills` | sticky on widget |

## ⚠ COMMON MISTAKES

- Configuring weights on the child widget instead of its parent
- Forgetting root weights, so nothing below it can grow
- Giving weights but no `sticky`, leaving widgets centered

## ✓ TIPS & BEST PRACTICES

- Resize the window while developing to test weights
- Use `minsize` to keep sidebars readable
- Use `uniform` for toolbars of equal buttons

## QUICK SUMMARY

- Weights distribute extra space
- Set on the parent with `columnconfigure` / `rowconfigure`
- `minsize` and `uniform` for limits and equality
- Pair weights with `sticky`

# place()



## EXACT POSITIONING WITH PLACE()

`place()` positions a widget by **coordinates** inside its parent, either absolute pixels (`x`, `y`, `width`, `height`) or **relative** fractions of the parent size (`relx`, `rely`, `relwidth`, `relheight` from 0.0 to 1.0). The two can be combined: relative values are computed first, then the absolute ones are added as offsets. `anchor` chooses which point of the widget lands on the coordinates. `place` ignores other widgets entirely, so overlapping is possible and sometimes the point.

## NOTES

- `place` doesn't resize the parent to fit children
- Relative values make overlays follow window resizing
- Placed widgets stack in creation order; use `lift()` to raise
- Good for overlays, badges, splash screens and games
- Poor for forms: no automatic alignment or resizing

## OVERLAY, BADGE AND CENTERED BOX

```
import tkinter as tk

root = tk.Tk()
root.geometry("400x260")

bg = tk.Label(root, bg="#2b3a55")
bg.place(x=0, y=0, relwidth=1, relheight=1)      # full background

box = tk.Frame(root, bg="white", padx=12, pady=12)
box.place(relx=0.5, rely=0.5, anchor="center")    # always centered
tk.Label(box, text="Sign in", bg="white",
         font=("Helvetica", 14)).pack()
tk.Entry(box).pack(pady=6)

badge = tk.Label(root, text="3", bg="red", fg="white", width=2)
badge.place(relx=1.0, x=-10, y=10, anchor="ne")   # 10 px from top-right

bar = tk.Frame(root, bg="#ffcc00", height=4)
bar.place(relx=0, rely=1.0, relwidth=0.6, anchor="sw")  # progress strip
def grow():
    w = float(bar.place_info()["relwidth"])
    bar.place_configure(relwidth=min(1.0, w + 0.1))
root.bind("<space>", lambda e: grow())
root.mainloop()
```

## CODE EXPLAINED

- → `relwidth=1, relheight=1` stretches the background
- → `relx=0.5, rely=0.5, anchor="center"` centers the box
- → `relx=1.0, x=-10` = right edge minus 10 pixels
- → `anchor="sw"` pins the strip's bottom-left corner
- → `place_info()` returns strings; convert before math

## PLACE() OPTIONS

| OPTION | MEANING |
|---|---|
| `x / y` | Pixel position |
| `relx / rely` | Fraction of parent |
| `width / height` | Pixel size |
| `relwidth / relheight` | Fraction size |
| `anchor` | Widget point used |

## COMMON RECIPES

| GOAL | OPTIONS |
|---|---|
| `Fill parent` | relwidth=1, relheight=1 |
| `Center` | relx=.5, rely=.5, anchor=center |
| `Top-right` | relx=1, anchor=ne |
| `Bottom bar` | rely=1, relwidth=1, anchor=sw |
| `Inset` | x=10, y=10 |

## PLACE METHODS

| METHOD | DOES |
|---|---|
| `place_configure()` | Change options |
| `place_forget()` | Remove |
| `place_info()` | Current options |
| `place_slaves()` | Placed children |
| `lift() / lower()` | Stacking order |

## ⚠ COMMON MISTAKES

- Building whole forms with `place` and hard-coded pixels
- Expecting the parent to grow around placed children
- Mixing up `anchor` with the position it applies to

## ✓ TIPS & BEST PRACTICES

- Combine `relx` with `x` for offsets from edges
- Use `place` for overlays on top of a grid layout
- Keep pixel values in constants for HiDPI scaling

## QUICK SUMMARY

- Absolute `x` / `y` or relative `relx` / `rely`
- `anchor` picks the widget's reference point
- Can overlap; no auto layout
- Best for overlays and special effects

# Layout Patterns & Pitfalls

nested Frames · never mix pack and grid · pack_propagate(False) · grid_propagate() · form layout · toolbar + status bar

## BUILDING REAL LAYOUTS

Real screens are built by **nesting Frames**, each with the geometry manager that suits its contents: `pack` for bars and simple stacks, `grid` for forms and tables, `place` for overlays. The one hard rule is: **never use `pack` and `grid` for children of the same parent**; Tk 8.6 raises a `TclError`, and older versions could freeze as both managers fought over the parent's size. Containers normally shrink to fit their children; turn that off with `pack_propagate(False)` or `grid_propagate(False)` to keep a fixed size.

## NOTES

- Different managers in different Frames are fine
- Propagation off = honor the Frame's `width` / `height`
- Sketch the screen as nested boxes before coding
- One Frame per region keeps code easy to change
- `winfo_children()` lists a container's widgets

## APP SHELL: TOOLBAR, FORM, STATUS

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.geometry("520x300")

top = ttk.Frame(root, padding=4)                    # pack in root
body = ttk.Frame(root, padding=10)
status = ttk.Label(root, text="Ready", anchor="w", padding=(6, 2))
top.pack(fill="x"); status.pack(side="bottom", fill="x")
body.pack(fill="both", expand=True)
for t in ("New", "Open", "Save"):                   # pack in toolbar
    ttk.Button(top, text=t).pack(side="left")
body.columnconfigure(1, weight=1)                   # grid in body
for r, lbl in enumerate(("Title", "Author", "Year")):
    ttk.Label(body, text=lbl).grid(row=r, column=0, sticky="w", pady=2)
    ttk.Entry(body).grid(row=r, column=1, sticky="ew", pady=2)
preview = tk.Frame(body, width=120, height=80, bg="#ddd")
preview.grid(row=0, column=2, rowspan=3, padx=(10, 0))
preview.grid_propagate(False)                       # keep 120x80
tk.Label(preview, text="Cover", bg="#ddd").place(relx=.5, rely=.5,
                                                 anchor="center")
root.mainloop()
```

## CODE EXPLAINED

- → Root children all use `pack` : toolbar, status, body
- → The toolbar's buttons use `pack` inside their own Frame
- → The body uses `grid` for an aligned form
- → `grid_propagate(False)` keeps the preview at 120×80
- → `place` centers a label inside the fixed preview

## CHOOSING A MANAGER

| CONTENT | MANAGER |
|---|---|
| `Toolbar / status bar` | pack |
| `Vertical stack` | pack |
| `Form fields` | grid |
| `Table-like layout` | grid |
| `Overlay / centered box` | place |

## PROPAGATION

| CALL | EFFECT |
|---|---|
| `pack_propagate(False)` | Fixed size (pack) |
| `grid_propagate(False)` | Fixed size (grid) |
| `width / height` | Used when off |
| `Default True` | Shrink to fit |

## LAYOUT DEBUGGING

| TRICK | SHOWS |
|---|---|
| `bg="red" on Frames` | Region bounds |
| `winfo_geometry()` | Actual size/pos |
| `grid_size()` | Rows and columns used |
| `pack_info()` | Applied options |
| `update_idletasks()` | Force layout now |

## ⚠ COMMON MISTAKES

- `pack` and `grid` in the same parent ( `TclError` in Tk 8.6)
- Setting Frame `width` / `height` without disabling propagation
- One giant grid for the whole window, hard to change later

## ✓ TIPS & BEST PRACTICES

- Name frames after regions: `toolbar` , `sidebar` , `body`
- Color frames temporarily to see their bounds
- Wrap each region in its own class as it grows

## QUICK SUMMARY

- Nest Frames; one manager per parent
- pack for bars, grid for forms, place for overlays
- Propagation off for fixed-size containers
- Debug with colors and `winfo_*`

# Control Variables

StringVar · IntVar · DoubleVar · BooleanVar · get() · set() · trace_add() · trace_remove()

## TK VARIABLES LINK DATA AND WIDGETS

**Control variables** (`StringVar`, `IntVar`, `DoubleVar`, `BooleanVar`) are values held inside the Tcl interpreter that widgets can watch. Connect one through `textvariable=` or `variable=`, and every widget using it updates when you call `set()`; every user edit is visible through `get()`. `trace_add()` registers a callback that fires when the variable is **written**, **read** or **unset**, which is how you build live previews and dependent fields.

## NOTES

- Create variables after `Tk()` exists (or pass `master=`)
- `IntVar.get()` raises `TclError` if the text isn't a valid int
- Trace callbacks receive `(name, index, mode)` strings
- `trace_add` replaces the deprecated `trace` / `trace_variable`
- Keep a reference: an unreferenced variable is deleted

## LIVE TOTAL WITH TRACES

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
price = tk.DoubleVar(value=9.99)
qty = tk.IntVar(value=1)
total = tk.StringVar()

def update_total(*_):
    try:
        total.set(f"Total: ${price.get() * qty.get():,.2f}")
    except tk.TclError:              # empty or invalid field
        total.set("Total: –")

for v in (price, qty):
    v.trace_add("write", update_total)
update_total()
ttk.Entry(root, textvariable=price, width=8).grid(row=0, column=0)
ttk.Spinbox(root, from_=1, to=99, textvariable=qty,
            width=4).grid(row=0, column=1, padx=4)
ttk.Label(root, textvariable=total).grid(row=1, column=0, columnspan=2)
tid = qty.trace_add("write", lambda *a: print("qty", qty.get()))
qty.trace_remove("write", tid)       # stop watching
print(qty.trace_info())              # [(('write',), 'name…')]
root.mainloop()
```

## CODE EXPLAINED

- → `*_` absorbs the three trace arguments
- → `try` / `except TclError` handles half-typed numbers
- → The same callback traces both inputs
- → `textvariable=` links Entry, Spinbox and Label
- → `trace_add` returns an id needed by `trace_remove`

## VARIABLE TYPES

| CLASS | GET() RETURNS |
|---|---|
| `StringVar` | str |
| `IntVar` | int |
| `DoubleVar` | float |
| `BooleanVar` | bool |
| `Variable` | Raw Tcl value |

## VARIABLE METHODS

| METHOD | DOES |
|---|---|
| `get()` | Read value |
| `set(v)` | Write value |
| `trace_add(mode, f)` | Watch; returns id |
| `trace_remove(m, id)` | Stop watching |
| `trace_info()` | List traces |

## TRACE MODES

| MODE | FIRES WHEN |
|---|---|
| `"write"` | Value changes |
| `"read"` | Value read |
| `"unset"` | Variable deleted |
| `"array"` | Tcl array access |

## ⚠ COMMON MISTAKES

- Creating a `StringVar()` before `Tk()` with no master
- Letting `IntVar.get()` crash on an empty Entry
- Writing to the same variable inside its write trace (recursion)

## ✓ TIPS & BEST PRACTICES

- Use variables for any value shown in more than one place
- Wrap numeric `get()` calls in `try` / `except tk.TclError`
- Give variables a `name=` to debug traces

## QUICK SUMMARY

- String/Int/Double/BooleanVar
- `get()` / `set()` sync all linked widgets
- `trace_add("write", fn)` reacts to changes
- Keep references so they aren't deleted

# Command Callbacks

command= · lambda · functools.partial · late binding in loops · default-arg capture · bound methods

## WIRING BUTTONS TO FUNCTIONS

`command=` expects a **callable that takes no arguments**. To pass data, wrap the call in a `lambda` or `functools.partial`. The classic trap is **late binding** in loops: a lambda looks up loop variables when it runs, not when it's created, so every button ends up using the last value. Fix it by binding the current value as a **default argument** (`lambda n=n: …`) or with `partial`. Bound methods (`self.save`) are the cleanest choice in class-based apps.

## NOTES

- `command=f()` calls `f` once immediately; drop the parentheses
- `partial` freezes arguments at creation time
- Callbacks run on the main thread and block the UI while running
- A callback's return value is ignored for `command=`
- Exceptions print to stderr; the app keeps running

## PASSING ARGUMENTS SAFELY

```
import tkinter as tk
from functools import partial

root = tk.Tk()
display = tk.StringVar(value="")
tk.Entry(root, textvariable=display, justify="right",
         font=("Consolas", 14)).grid(row=0, column=0, columnspan=3)

def press(key):
    display.set(display.get() + key)
# WRONG: all buttons would type "9" (late binding)
# for i, k in enumerate("123456789"):
#     tk.Button(root, text=k, command=lambda: press(k))
for i, k in enumerate("123456789"):            # default-arg capture
    tk.Button(root, text=k, width=4,
              command=lambda k=k: press(k)).grid(row=1 + i // 3,
                                                 column=i % 3)
tk.Button(root, text="0", width=4,
          command=partial(press, "0")).grid(row=4, column=1)
tk.Button(root, text="C", width=4,
          command=lambda: display.set("")).grid(row=4, column=0)
tk.Button(root, text="=", width=4,
          command=lambda: display.set(str(eval(display.get() or "0")))
          ).grid(row=4, column=2)
root.mainloop()
```

## CODE EXPLAINED

- → `press(key)` needs an argument, so it can't go in `command` bare
- → The commented loop shows the late-binding bug
- → `k=k` copies the current value into the lambda
- → `partial(press, "0")` is the import-based alternative
- → Demo only: never `eval` untrusted input in real apps

## CALLBACK FORMS

| FORM | USE |
|---|---|
| `command=f` | No arguments |
| `command=self.save` | Method in a class |
| `lambda: f(1)` | Fixed arguments |
| `lambda x=x: f(x)` | Loop values |
| `partial(f, x)` | Freeze arguments |

## LATE BINDING

| CODE | EACH BUTTON GETS |
|---|---|
| `lambda: f(i)` | Last i |
| `lambda i=i: f(i)` | Its own i |
| `partial(f, i)` | Its own i |
| `def make(i)` | Its own i (closure) |

## COMMAND VS BIND

| ASPECT | COMMAND / BIND |
|---|---|
| `Arguments` | none / event |
| `Trigger` | widget action / any event |
| `Keyboard` | space on focus / any key |
| `Return "break"` | n/a / stops chain |

## ⚠ COMMON MISTAKES

- `command=save()` runs at startup, not on click
- Loop lambdas capturing the loop variable by reference
- Doing slow work in a callback and freezing the window

## ✓ TIPS & BEST PRACTICES

- Prefer methods for callbacks with real logic
- Use `partial` when the intent is clearer than a lambda
- Move long tasks to threads or `after()` (Sheets 50–51)

## QUICK SUMMARY

- `command=` takes a zero-argument callable
- Use `lambda` or `partial` to pass data
- Capture loop values with defaults
- Keep callbacks short

# bind() & Event Sequences

bind() · <Button-1> · <Double-Button-1> · <Key> · <Return> · <Control-s> · <Configure> · unbind()

## BINDING EVENTS TO HANDLERS

`widget.bind(sequence, handler)` calls `handler(event)` whenever the **event sequence** occurs on that widget. Sequences are strings in angle brackets: `<Modifier-Type-Detail>`, such as `<Button-1>` (left click), `<Control-s>`, `<KeyPress-Return>` or `<Double-Button-1>`. Common shorthand works too: `<Return>`, `<Escape>`, `<Key>` (any key). `<Configure>` fires on resize or move, and `<Enter>`/`<Leave>` track the mouse pointer entering or leaving a widget.

## NOTES

- Handlers always receive one `event` argument
- Letters are case-sensitive: `<Control-S>` needs Shift
- `add="+"` adds a handler instead of replacing
- Key events go to the widget with keyboard focus
- macOS uses `<Command-…>` for Cmd shortcuts

## MOUSE, KEYBOARD AND RESIZE

```
import tkinter as tk

root = tk.Tk()
canvas = tk.Canvas(root, width=300, height=200, bg="white")
canvas.pack(fill="both", expand=True)
status = tk.Label(root, anchor="w"); status.pack(fill="x")

def on_click(e):
    canvas.create_oval(e.x - 4, e.y - 4, e.x + 4, e.y + 4, fill="red")

def on_drag(e):
    status.config(text=f"drag {e.x},{e.y}")

canvas.bind("<Button-1>", on_click)
canvas.bind("<B1-Motion>", on_drag)
canvas.bind("<Double-Button-1>", lambda e: canvas.delete("all"))
canvas.bind("<Enter>", lambda e: canvas.config(cursor="crosshair"))
canvas.bind("<Configure>",
            lambda e: status.config(text=f"size {e.width}×{e.height}"))
root.bind("<Control-s>", lambda e: print("save"))
root.bind("<Escape>", lambda e: root.destroy())
root.bind("<Key>", lambda e: print("key", e.keysym), add="+")
canvas.unbind("<B1-Motion>")          # remove a binding
root.mainloop()
```

## CODE EXPLAINED

- → `<Button-1>` handler draws a dot at the click position
- → `<B1-Motion>` = mouse moved while button 1 held
- → `<Configure>` reports the new size on every resize
- → Shortcuts bound on root work wherever focus is in that window
- → `add="+"` keeps any existing `<Key>` handler

## MOUSE EVENTS

| SEQUENCE | WHEN |
|---|---|
| `<Button-1>` | Left press |
| `<ButtonRelease-1>` | Left release |
| `<Double-Button-1>` | Double-click |
| `<B1-Motion>` | Drag with left |
| `<Button-3>` | Right click (Win/X11) |

## KEYBOARD EVENTS

| SEQUENCE | WHEN |
|---|---|
| `<Key>` | Any key press |
| `<Return>` | Enter key |
| `<Escape>` | Esc |
| `<Control-s>` | Ctrl+S |
| `<F1>` | F1 key |

## WINDOW EVENTS

| SEQUENCE | WHEN |
|---|---|
| `<Configure>` | Resized / moved |
| `<Enter>` / `<Leave>` | Pointer in / out |
| `<FocusIn>` | Gains focus |
| `<Map>` / `<Unmap>` | Shown / hidden |
| `<Destroy>` | Being destroyed |

## ⚠ COMMON MISTAKES

- Handler without an `event` parameter (TypeError on trigger)
- Binding `<Control-S>` and wondering why Ctrl+S does nothing
- Binding keys to a widget that never gets focus

## ✓ TIPS & BEST PRACTICES

- Bind app shortcuts on the root or with `bind_all`
- Use `def handler(event=None)` to share with `command=`
- Check `e.widget` when one handler serves many widgets

## QUICK SUMMARY

- `bind("<Sequence>", handler)`
- Handlers take one `event` argument
- `<Modifier-Type-Detail>` syntax
- `unbind` and `add="+"` manage handlers

# The Event Object



## WHAT THE EVENT TELLS YOU

Every bound handler receives an **Event object** describing what happened. Which attributes are meaningful depends on the event type: mouse events fill `x`/`y` (relative to the widget) and `x_root`/`y_root` (relative to the screen); key events fill `keysym` (a name like `"Return"`), `char` (the typed character, possibly empty) and `keycode`; `<Configure>` fills `width`/`height`; `<MouseWheel>` fills `delta`. `event.widget` always names the widget that received the event.

## NOTES

- Unused attributes hold `??` or 0
- `char` is `""` for keys like Shift or arrows
- `keysym` is reliable across keyboard layouts
- `state` is a bitmask of held modifiers
- `delta` is ±120 per notch on Windows, small values on macOS

## INSPECTING EVENTS

```
import tkinter as tk

root = tk.Tk()
out = tk.Label(root, width=48, anchor="w", font=("Consolas", 10))
out.pack(fill="x")
area = tk.Frame(root, width=320, height=160, bg="#eef")
area.pack(fill="both", expand=True)

def show(e):
    out.config(text=f"{e.type}: x={e.x} y={e.y} "
                    f"root=({e.x_root},{e.y_root}) num={e.num}")

def key(e):
    shift = bool(e.state & 0x0001)
    ctrl = bool(e.state & 0x0004)
    out.config(text=f"keysym={e.keysym!r} char={e.char!r} "
                    f"shift={shift} ctrl={ctrl}")
def wheel(e):
    step = -1 if e.delta > 0 else 1          # Windows/macOS
    out.config(text=f"wheel delta={e.delta} step={step}")
area.bind("<Button>", show)                  # any mouse button
root.bind("<Key>", key)
area.bind("<MouseWheel>", wheel)
area.bind("<Button-4>", lambda e: out.config(text="X11 wheel up"))
area.bind("<Configure>", lambda e: print(e.widget, e.width, e.height))
root.mainloop()
```

## CODE EXPLAINED

- → `e.type` names the event kind, e.g. `ButtonPress`
- → `e.num` tells which mouse button was pressed
- → Bit masks on `e.state` reveal Shift and Control
- → `e.delta` sign gives wheel direction
- → X11 sends wheel as `<Button-4>` / `<Button-5>` instead

## POSITION

| ATTRIBUTE | MEANING |
|---|---|
| `x, y` | In widget pixels |
| `x_root, y_root` | On screen |
| `width, height` | Configure size |
| `widget` | Receiving widget |
| `time` | Timestamp (ms) |

## KEYBOARD

| ATTRIBUTE | EXAMPLE |
|---|---|
| `keysym` | "Return", "a", "F5" |
| `char` | "a", "\r", "" |
| `keycode` | Hardware code |
| `keysym_num` | Numeric keysym |
| `state` | Modifier mask |

## STATE BITS

| MASK | MODIFIER |
|---|---|
| `0x0001` | Shift |
| `0x0004` | Control |
| `0x0008` | Alt / Mod1 (varies) |
| `0x0100` | Button 1 held |
| `0x0400` | Button 3 held |

## ⚠ COMMON MISTAKES

- Using `e.char` to detect arrows or function keys (it's empty)
- Mixing up widget `x` / `y` and screen `x_root` / `y_root`
- Assuming `delta` is ±120 on every platform

## ✓ TIPS & BEST PRACTICES

- Print the whole event while learning: `print(e)`
- Match on `keysym` for named keys
- Normalize wheel input with the sign of `delta`

## QUICK SUMMARY

- Handlers get an Event object
- `x` / `y`, `x_root` / `y_root` for position
- `keysym`, `char`, `state` for keys
- `widget` names the source

# Binding Levels & Virtual Events

bind_all() · bind_class() · bindtags() · return "break" · event_add() · event_generate() · <<Custom>>

## BIND TAGS AND CUSTOM EVENTS

Each widget has a list of **bindtags** that decides which bindings run for an event, in order: the widget itself, its **class** (`"Entry"`), its **toplevel** window, then `"all"`. `bind()` targets the widget, `bind_class()` a whole class, and `bind_all()` the `"all"` tag. A handler that returns `"break"` stops the remaining tags, which is how you override default behavior. **Virtual events** like <<Save>> are named events you define with `event_add()` or fire with `event_generate()`.

## NOTES

- Class bindings implement default widget behavior
- Changing `bind_class` affects every widget of that class
- Built-in virtual events: `<<Copy>>`, `<<Paste>>`, `<<Modified>>`
- `event_generate` runs handlers immediately by default
- `when="tail"` queues a generated event instead

## OVERRIDE, BROADCAST, SIGNAL

```
import tkinter as tk

root = tk.Tk()
entry = tk.Entry(root); entry.pack(padx=10, pady=10)
print(entry.bindtags())   # ('.!entry', 'Entry', '.', 'all')

def digits_only(e):
    if e.char and not e.char.isdigit() and e.keysym != "BackSpace":
        return "break"           # skip the class binding: no insert

entry.bind("<Key>", digits_only)

# Same shortcut for every widget in the app
root.bind_all("<Control-q>", lambda e: root.destroy())
# Virtual event mapped to two physical key combos
root.event_add("<<Save>>", "<Control-s>", "<F2>")
root.bind("<<Save>>", lambda e: print("saving…"))
# A custom signal any code can fire
root.bind("<<DataLoaded>>", lambda e: print("refresh UI"))
root.after(500, lambda: root.event_generate("<<DataLoaded>>"))
# Reorder tags: run toplevel bindings before the widget's own
tags = entry.bindtags()
entry.bindtags((tags[2], tags[0], tags[1], tags[3]))
root.mainloop()
```

## CODE EXPLAINED

- → Tag order: widget, class, toplevel, all
- → Returning `"break"` prevents the Entry class from inserting
- → `bind_all` catches Ctrl+Q anywhere in the app
- → `event_add` maps several key combos to `<<Save>>`
- → `event_generate` fires a custom event on demand

## BINDING LEVELS

| METHOD | SCOPE |
|---|---|
| `w.bind()` | One widget |
| `w.bind_class(c, …)` | All widgets of class |
| `w.bind_all()` | Whole application |
| `toplevel.bind()` | Widgets in that window |
| `w.bindtags(tags)` | Get/set order |

## BUILT-IN VIRTUAL

| EVENT | FIRES ON |
|---|---|
| `<<Copy>>` `<<Paste>>` | Clipboard keys |
| `<<Modified>>` | Text changed |
| `<<ListboxSelect>>` | Listbox selection |
| `<<ComboboxSelected>>` | Combobox pick |
| `<<TreeviewSelect>>` | Treeview selection |

## VIRTUAL METHODS

| METHOD | DOES |
|---|---|
| `event_add(v, *seq)` | Map sequences |
| `event_delete(v)` | Remove mapping |
| `event_info(v)` | List mappings |
| `event_generate(ev)` | Fire event |
| `when="tail"` | Queue instead |

## ⚠ COMMON MISTAKES

- Returning `"break"` from a `command=` callback (no effect)
- Using `bind_class` and changing every Entry by accident
- Firing `event_generate` from a worker thread

## ✓ TIPS & BEST PRACTICES

- Use `"break"` for input filters and custom keys
- Name app events `<<Noun>>` and fire them on state changes
- Prefer `bind_all` for global shortcuts only

## QUICK SUMMARY

- Bindtags: widget → class → toplevel → all
- `"break"` stops further handlers
- `bind_class` / `bind_all` widen scope
- Virtual events: `<<Name>>`

# Focus & Keyboard Navigation



## KEYBOARD FOCUS AND NAVIGATION

Only one widget at a time has **keyboard focus**, and key events go to it. Users move focus with **Tab** and **Shift+Tab** in creation (stacking) order, skipping widgets whose `takefocus` is false. Set focus in code with `focus_set()`, and check it with `focus_get()`. `<FocusIn>` and `<FocusOut>` fire as focus moves, which suits field validation and highlighting. **Accelerators** shown in menus are just labels; you must also bind the keys yourself.

## NOTES

- `focus_set()` only takes effect when the window is active
- `focus_force()` steals focus from other apps; avoid it
- Labels and Frames don't take focus by default
- Tab order follows stacking order; `lift()` / `lower()` change it
- Use `tk_focusNext()` to find the next widget in order

## FORM FOCUS HANDLING

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
fields = []
for r, name in enumerate(("Name", "Email", "Phone")):
    ttk.Label(root, text=name).grid(row=r, column=0, sticky="w", padx=4)
    e = ttk.Entry(root, width=28)
    e.grid(row=r, column=1, pady=2, padx=4)
    fields.append(e)

ttk.Button(root, text="?", takefocus=False).grid(row=0, column=2)  # no Tab
def next_field(event):
    event.widget.tk_focusNext().focus_set()
    return "break"                          # don't insert newline
def mark(event, on):
    event.widget.configure(style="Focus.TEntry" if on else "TEntry")
ttk.Style().configure("Focus.TEntry", fieldbackground="#fffbd6")
for e in fields:
    e.bind("<Return>", next_field)          # Enter acts like Tab
    e.bind("<FocusIn>", lambda ev: mark(ev, True))
    e.bind("<FocusOut>", lambda ev: mark(ev, False))
fields[0].focus_set()                       # start in Name
root.bind("<Control-l>", lambda e: print(root.focus_get()))
root.mainloop()
```

## CODE EXPLAINED

- → Entries are created in order, so Tab follows rows
- → `takefocus=False` removes the help button from the Tab cycle
- → `tk_focusNext()` gives the next widget; `"break"` stops default
- → `<FocusIn>` / `<FocusOut>` swap a highlighted style
- → `focus_get()` returns the focused widget object

## FOCUS METHODS

| METHOD | DOES |
|---|---|
| `focus_set()` | Request focus |
| `focus_get()` | Focused widget |
| `focus_force()` | Force focus |
| `tk_focusNext()` | Next in order |
| `tk_focusPrev()` | Previous in order |

## FOCUS OPTIONS

| OPTION | EFFECT |
|---|---|
| `takefocus=True` | Always a tab stop |
| `takefocus=False` | Never a tab stop |
| `highlightthickness` | Focus ring (classic) |
| `highlightcolor` | Ring color |

## ACCELERATOR PAIRS

| MENU LABEL | BINDING |
|---|---|
| `Ctrl+S` | <Control-s> |
| `Ctrl+Shift+S` | <Control-S> |
| `F5` | <F5> |
| `Cmd+S (mac)` | <Command-s> |
| `Alt+X` | <Alt-x> |

## ⚠ COMMON MISTAKES

- Expecting `accelerator=` in menus to bind keys
- Calling `focus_set()` before the window is mapped
- Creating widgets out of visual order, scrambling Tab order

## ✓ TIPS & BEST PRACTICES

- Set initial focus to the first field users need
- Reorder a Tab stop with `w.lift(other)` instead of re-creating it
- Keep keyboard-only use working for every form

## QUICK SUMMARY

- One focused widget receives keys
- Tab order = creation order
- `takefocus` controls tab stops
- Bind accelerator keys yourself

# Menu Bars



## BUILDING A MENU BAR

A menu bar is a `tk.Menu` attached to a window with `config(menu=menubar)`. Each top-level entry (File, Edit, Help) is another `Menu` added with `add_cascade()`, and its items are added with `add_command()`. Set `tearoff=0` on every menu, or Tk adds a dashed line that detaches the menu into its own window (an old X11 feature). `accelerator=` only **displays** a shortcut; bind the key separately. `underline=` marks the Alt-key mnemonic.

## NOTES

- There is no ttk menu; `tk.Menu` is native on Windows and macOS
- macOS puts the menu bar at the top of the screen
- A menu named `"apple"` becomes the macOS application menu
- `root.option_add("*tearOff", False)` disables tear-offs globally
- Menus are owned by the window; they can't be packed

## FILE / EDIT / HELP MENUS

```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.option_add("*tearOff", False)
def new(e=None): print("new")
def save(e=None): print("save")

menubar = tk.Menu(root)
root.config(menu=menubar)

file_menu = tk.Menu(menubar)
menubar.add_cascade(label="File", menu=file_menu, underline=0)
file_menu.add_command(label="New", accelerator="Ctrl+N", command=new)
file_menu.add_command(label="Save", accelerator="Ctrl+S", command=save)
file_menu.add_separator()
file_menu.add_command(label="Exit", command=root.destroy)

help_menu = tk.Menu(menubar, name="help")   # special Help menu name
menubar.add_cascade(label="Help", menu=help_menu, underline=0)
help_menu.add_command(label="About",
    command=lambda: messagebox.showinfo("About", "Demo 1.0"))

root.bind("<Control-n>", new); root.bind("<Control-s>", save)
root.mainloop()
```

## CODE EXPLAINED

- → `option_add("*tearOff", False)` must come before menus are made
- → `config(menu=menubar)` installs the bar on the window
- → Each cascade's `menu` is a child Menu of the bar
- → `name="help"` gives the Help menu its platform placement
- → `accelerator` text plus a real `bind()` for each shortcut

## MENU METHODS

| METHOD | ADDS |
|---|---|
| `add_cascade()` | Submenu |
| `add_command()` | Clickable item |
| `add_separator()` | Divider line |
| `add_checkbutton()` | Toggle item |
| `add_radiobutton()` | Choice item |

## ITEM OPTIONS

| OPTION | PURPOSE |
|---|---|
| `label` | Item text |
| `command` | Callback |
| `accelerator` | Shortcut label |
| `underline` | Mnemonic index |
| `state` | normal / disabled |

## SPECIAL NAMES

| NAME= | EFFECT |
|---|---|
| `"apple"` | macOS app menu |
| `"help"` | Help menu placement |
| `"window"` | macOS Window menu |
| `"system"` | Windows system menu |

## ⚠ COMMON MISTAKES

- Leaving tear-off lines on every menu
- Expecting `accelerator=` to bind the shortcut
- Calling `pack()` on a menu bar

## ✓ TIPS & BEST PRACTICES

- Use the same function for the menu item and the key binding
- Group items with separators: New/Open, Save, Exit
- Test on macOS: the bar moves to the screen top

## QUICK SUMMARY

- `Menu` + `config(menu=…)`
- `add_cascade` for submenus, `add_command` for items
- `tearoff=0` everywhere
- Accelerators need separate bindings

# Menu Items & Context Menus



## MENU ITEMS AND POPUP MENUS

Menus can hold more than commands: `add_checkbutton()` toggles a variable, `add_radiobutton()` picks one value from a group, and nested `add_cascade()` builds submenus. Items are addressed by **index** or **label**; `entryconfigure()` changes them later, such as enabling Paste only when the clipboard has text. `postcommand=` runs just before a menu opens, which is the right time to refresh item states. A **context menu** is a Menu shown at the pointer with `tk_popup(x, y)`.

## NOTES

- Right-click is `<Button-3>` on Windows/Linux, `<Button-2>` on macOS
- Also bind `<Control-Button-1>` on macOS for one-button mice
- `index("end")` gives the last item's index
- Check and radio items share Tk variables like widgets
- `delete(0, "end")` clears a menu for rebuilding

## VIEW OPTIONS AND RIGHT-CLICK MENU

```
import tkinter as tk

root = tk.Tk()
menubar = tk.Menu(root); root.config(menu=menubar)
text = tk.Text(root, width=40, height=8); text.pack()
wrap = tk.BooleanVar(value=True); theme = tk.StringVar(value="light")

view = tk.Menu(menubar, tearoff=0, postcommand=lambda: print("opening"))
menubar.add_cascade(label="View", menu=view)
view.add_checkbutton(label="Word wrap", variable=wrap,
    command=lambda: text.config(wrap="word" if wrap.get() else "none"))
themes = tk.Menu(view, tearoff=0)
view.add_cascade(label="Theme", menu=themes)
for t in ("light", "dark"):
    themes.add_radiobutton(label=t.title(), value=t, variable=theme)
popup = tk.Menu(root, tearoff=0)
for ev in ("Cut", "Copy", "Paste"):
    popup.add_command(label=ev, command=lambda ev=ev: text.event_generate(f"<<{ev}>>"))
def show_popup(e):
    state = "normal" if text.tag_ranges("sel") else "disabled"
    for label in ("Cut", "Copy"): popup.entryconfigure(label, state=state)
    popup.tk_popup(e.x_root, e.y_root)
aqua = root.tk.call("tk", "windowingsystem") == "aqua"
text.bind("<Button-2>" if aqua else "<Button-3>", show_popup)
```

## CODE EXPLAINED

- → `add_checkbutton` toggles `wrap` and applies it
- → A nested Menu becomes the Theme submenu
- → Radio items share `theme`, so only one is ticked
- → `entryconfigure(label, …)` disables Cut/Copy without a selection
- → `tk_popup` opens at screen coordinates from the event

## ITEM TYPES

| METHOD | BEHAVES LIKE |
|---|---|
| `add_command` | Button |
| `add_checkbutton` | Checkbutton |
| `add_radiobutton` | Radiobutton |
| `add_cascade` | Submenu |
| `add_separator` | Divider |

## EDITING ITEMS

| METHOD | DOES |
|---|---|
| `entryconfigure(i, …)` | Change item |
| `entrycget(i, opt)` | Read option |
| `index(label)` | Find index |
| `delete(a, b)` | Remove items |
| `invoke(i)` | Run item |

## POPUP CALLS

| CALL | PURPOSE |
|---|---|
| `tk_popup(x, y)` | Show at screen pos |
| `post(x, y)` | Show (low level) |
| `unpost()` | Hide |
| `grab_release()` | Free pointer (X11) |

## ⚠ COMMON MISTAKES

- Binding only `<Button-3>`, so macOS right-click does nothing
- Using widget `x` / `y` instead of `x_root` / `y_root` for popups
- Changing items by numeric index after inserting new ones

## ✓ TIPS & BEST PRACTICES

- Refresh item states in `postcommand` or just before popping up
- Address items by label so inserts don't break code
- Reuse Cut/Copy/Paste through virtual events

## QUICK SUMMARY

- Check, radio and cascade items
- `entryconfigure` updates items
- `postcommand` refreshes before opening
- `tk_popup(x_root, y_root)` for context menus

# messagebox



## STANDARD MESSAGE BOXES

`tkinter.messagebox` shows native modal dialogs with one call. The **show** functions (`showinfo`, `showwarning`, `showerror`) only inform and return `"ok"`. The **ask** functions return the user's answer: `askyesno()` and `askokcancel()` return `True`/`False`, `askyesnocancel()` adds `None` for Cancel, `askretrycancel()` returns a bool, and `askquestion()` returns the string `"yes"` or `"no"`. Always pass `parent=` so the dialog is centered on, and owned by, the right window.

## NOTES

- The call blocks until the user answers
- `icon=` overrides the default (info, warning, error, question)
- `default=` picks the button activated by Enter
- `detail=` adds smaller secondary text (Tk 8.5+)
- Appearance is native, so it differs by OS

## INFORM, CONFIRM, DECIDE

```
import tkinter as tk
from tkinter import messagebox as mb

root = tk.Tk()
dirty = True

def on_close():
    if dirty:
        answer = mb.askyesnocancel(
            "Unsaved changes", "Save changes before closing?",
            detail="Your edits will be lost if you don't save.",
            icon="warning", parent=root)
        if answer is None:          # Cancel: keep the window open
            return
        if answer:                  # Yes
            print("saving…")
    root.destroy()                  # Yes, No, or nothing to save

def delete():
    if mb.askokcancel("Delete", "Delete 3 files?", icon="warning",
                      default="cancel", parent=root):
        mb.showinfo("Done", "Files deleted.", parent=root)
tk.Button(root, text="Delete…", command=delete).pack(padx=30, pady=20)
root.protocol("WM_DELETE_WINDOW", on_close)
```

## CODE EXPLAINED

- → `askyesnocancel` gives three outcomes: True, False, None
- → Test `is None` first; `None` is also falsy
- → `detail=` explains consequences under the main question
- → `default="cancel"` makes Enter the safe choice
- → `parent=root` keeps dialogs on top of the app

## FUNCTIONS

| FUNCTION | RETURNS |
|---|---|
| `showinfo()` | "ok" |
| `showwarning()` | "ok" |
| `showerror()` | "ok" |
| `askyesno()` | True / False |
| `askokcancel()` | True / False |

## MORE

| FUNCTION | RETURNS |
|---|---|
| `askretrycancel()` | True / False |
| `askquestion()` | "yes" / "no" |
| `Message(...).show()` | Custom type |

## OPTIONS

| OPTION | VALUES |
|---|---|
| `parent` | Owner window |
| `icon` | info/warning/error/question |
| `default` | ok/cancel/yes/no/retry |
| `detail` | Secondary text |

## ⚠ COMMON MISTAKES

- Treating `askquestion()` like a bool (`"no"` is truthy)
- Checking `if not answer` and merging No with Cancel
- Omitting `parent=` so dialogs appear behind Toplevels

## ✓ TIPS & BEST PRACTICES

- Make destructive confirmations default to Cancel
- Write titles as short nouns, messages as questions
- Use `showerror` for failures the user must act on

## QUICK SUMMARY

- `show*` inform; `ask*` return answers
- `askyesnocancel` → True/False/None
- `parent=`, `icon=`, `default=`, `detail=`
- Modal: blocks until answered

# filedialog

askopenfilename() · asksaveasfilename() · askdirectory() · askopenfilenames() · filetypes · initialdir · defaultextension

## OPEN AND SAVE FILE PICKERS

`tkinter.filedialog` wraps the native file pickers. `askopenfilename()` returns a **path string** (or `""` if cancelled), `askopenfilenames()` returns a tuple of paths, `asksaveasfilename()` returns the chosen save path, and `askdirectory()` returns a folder. The `askopenfile()`/`asksaveasfile()` variants return an **open file object** (or `None`). `filetypes` is a list of `(label, pattern)` pairs that fills the type drop-down.

## NOTES

- Cancel returns `""` (or `()` / `None` for some variants)
- Paths use forward slashes even on Windows
- `defaultextension` is appended if the user omits one
- `confirmoverwrite` (default True) asks before replacing
- `initialfile` pre-fills the file name box

## OPEN, SAVE AS, CHOOSE FOLDER

```
import tkinter as tk
from tkinter import filedialog as fd
from pathlib import Path

root = tk.Tk()
TYPES = [("Text files", "*.txt"), ("Markdown", "*.md *.markdown"),
         ("All files", "*.*")]
text = tk.Text(root, width=60, height=15); text.pack()

def open_file():
    path = fd.askopenfilename(parent=root, title="Open",
                              filetypes=TYPES, initialdir=Path.home())
    if not path:                          # cancelled
        return
    text.delete("1.0", "end")
    text.insert("1.0", Path(path).read_text(encoding="utf-8"))

def save_as():
    path = fd.asksaveasfilename(parent=root, filetypes=TYPES,
                                defaultextension=".txt", initialfile="notes")
    if path:
        Path(path).write_text(text.get("1.0", "end-1c"), encoding="utf-8")
tk.Button(root, text="Open", command=open_file).pack(side="left")
tk.Button(root, text="Save As", command=save_as).pack(side="left")
tk.Button(root, text="Folder", command=lambda: print(
    fd.askdirectory(parent=root, mustexist=True))).pack(side="left")
```

## CODE EXPLAINED

- → Patterns in one entry are space-separated
- → `if not path` handles Cancel
- → `initialdir` starts browsing in the home folder
- → `defaultextension` adds `.txt` when missing
- → `"end-1c"` avoids saving Text's extra newline

## FUNCTIONS

| FUNCTION | RETURNS |
|---|---|
| `askopenfilename` | Path str |
| `askopenfilenames` | Tuple of paths |
| `asksaveasfilename` | Path str |
| `askdirectory` | Folder str |
| `askopenfile` | File object |

## OPTIONS

| OPTION | PURPOSE |
|---|---|
| `filetypes` | Type filter list |
| `initialdir` | Start folder |
| `initialfile` | Default name |
| `defaultextension` | Auto suffix |
| `title` | Dialog title |

## FILETYPES PATTERNS

| ENTRY | MATCHES |
|---|---|
| `("Images", "*.png *.jpg")` | Several patterns |
| `("Python", "*.py")` | One extension |
| `("All", "*.*")` | Everything |
| `("Data", (".csv", ".tsv"))` | Tuple form |

## ⚠ COMMON MISTAKES

- Opening `""` after the user cancels
- Using `askopenfile()` without closing the file object
- Missing a `("All files", "*.*")` fallback entry

## ✓ TIPS & BEST PRACTICES

- Remember the last folder and pass it as `initialdir`
- Prefer the filename functions plus `pathlib` for I/O
- Always pass an encoding when reading text

## QUICK SUMMARY

- `askopenfilename`, `asksaveasfilename`, `askdirectory`
- Cancel → empty string
- `filetypes` list of (label, pattern)
- `defaultextension` for save dialogs

# simpledialog & colorchooser



## QUICK INPUT AND COLOR DIALOGS

`tkinter.simpledialog` provides tiny modal prompts: `askstring()`, `askinteger()` and `askfloat()` show a field with OK/Cancel and return the typed value, or `None` if cancelled. The numeric versions validate input and support `minvalue`/`maxvalue`. `tkinter.colorchooser.askcolor()` opens the native color picker and returns `((r, g, b), "#rrggbb")`, or `(None, None)` on cancel. For anything richer, subclass `simpledialog.Dialog` and override `body()` and `apply()`.

## NOTES

- Numeric dialogs re-prompt automatically on bad input
- `initialvalue=` pre-fills the field
- `show="*"` works on `askstring` for passwords
- `Dialog` subclasses get OK/Cancel buttons for free
- Tk has no built-in date picker; see `tkcalendar` (Sheet 60)

## PROMPTS AND A CUSTOM DIALOG

```
import tkinter as tk
from tkinter import simpledialog as sd, colorchooser, ttk

root = tk.Tk()
name = sd.askstring("Name", "Your name?", initialvalue="Ada", parent=root)
age = sd.askinteger("Age", "Age in years:", minvalue=0, maxvalue=130,
                    parent=root)
rgb, hexcolor = colorchooser.askcolor("#3366cc", title="Pick a color",
                                      parent=root)
if hexcolor:
    root.config(bg=hexcolor)

class LoginDialog(sd.Dialog):
    def body(self, master):
        ttk.Label(master, text="User").grid(row=0, column=0, sticky="w")
        ttk.Label(master, text="Password").grid(row=1, column=0, sticky="w")
        self.user = ttk.Entry(master); self.user.grid(row=0, column=1)
        self.pw = ttk.Entry(master, show="*"); self.pw.grid(row=1, column=1)
        return self.user                        # widget with initial focus
    def validate(self):
        return bool(self.user.get())            # False keeps dialog open
    def apply(self):
        self.result = (self.user.get(), self.pw.get())
dlg = LoginDialog(root, title="Log in")
print(name, age, hexcolor, dlg.result)          # result is None on Cancel
```

## CODE EXPLAINED

- → Each prompt returns `None` if the user cancels
- → `askinteger` enforces the 0–130 range
- → `askcolor` returns RGB and hex; hex is `None` on cancel
- → `body()` builds fields and returns the focus widget
- → `apply()` stores the answer in `self.result`

## SIMPLEDIALOG

| FUNCTION | RETURNS |
|---|---|
| `askstring()` | str or None |
| `askinteger()` | int or None |
| `askfloat()` | float or None |
| `Dialog subclass` | .result |

## PROMPT OPTIONS

| OPTION | PURPOSE |
|---|---|
| `initialvalue` | Pre-filled text |
| `minvalue` | Lower bound |
| `maxvalue` | Upper bound |
| `show` | Mask characters |
| `parent` | Owner window |

## DIALOG HOOKS

| METHOD | ROLE |
|---|---|
| `body(master)` | Build fields |
| `buttonbox()` | Custom buttons |
| `validate()` | OK allowed? |
| `apply()` | Store result |

## ⚠ COMMON MISTAKES

- Using the result without checking for `None`
- Reading `askcolor()[1]` as always present
- Building fields in `__init__` instead of `body()`

## ✓ TIPS & BEST PRACTICES

- Use `simpledialog` for one value, a `Dialog` subclass for forms
- Return the first Entry from `body()` for focus
- Validate in `validate()` and explain errors with a messagebox

## QUICK SUMMARY

- `askstring` / `askinteger` / `askfloat`
- `askcolor` → (rgb, hex)
- Cancel returns `None`
- Subclass `Dialog` for custom forms

# Custom Modal Dialogs

Toplevel · transient() · grab_set() · wait_window() · returning a result · <Escape> to cancel · centering on parent

## WRITING YOUR OWN MODAL DIALOG

A **modal** dialog blocks the rest of the app until it closes. Build one from a `Toplevel` with four steps: `transient(parent)` to keep it above its parent, `grab_set()` to route all input to it, `wait_window()` to pause the caller until it's destroyed, and a result attribute the caller reads afterward. Bind `<Return>` to OK and `<Escape>` to Cancel, route the close button through `protocol()`, and center the dialog on its parent so it appears where the user is looking.

## NOTES

- `wait_window()` keeps processing events (not frozen)
- `grab_set()` may fail before the window is visible; call `wait_visibility()` first on X11
- Release grabs automatically by destroying the dialog
- Store the result before `destroy()`
- A `simpledialog.Dialog` subclass does most of this for you

## REUSABLE MODAL DIALOG

```
import tkinter as tk
from tkinter import ttk

class RenameDialog(tk.Toplevel):
    def __init__(self, parent, old_name):
        super().__init__(parent)
        self.title("Rename"); self.result = None
        self.transient(parent)
        self.var = tk.StringVar(value=old_name)
        e = ttk.Entry(self, textvariable=self.var, width=30)
        e.pack(padx=12, pady=12); e.select_range(0, "end"); e.focus_set()
        ttk.Button(self, text="OK", command=self.ok).pack(side="left", padx=12)
        ttk.Button(self, text="Cancel", command=self.destroy).pack(side="right", padx=12)
        self.bind("<Return>", lambda e: self.ok())
        self.bind("<Escape>", lambda e: self.destroy())
        self.update_idletasks()                      # center on parent
        x = parent.winfo_rootx() + (parent.winfo_width() - self.winfo_width()) // 2
        y = parent.winfo_rooty() + (parent.winfo_height() - self.winfo_height()) // 3
        self.geometry(f"+{x}+{y}")
        self.wait_visibility(); self.grab_set()
        self.wait_window()                           # block until closed
    def ok(self):
        self.result = self.var.get().strip() or None
        self.destroy()
# usage: name = RenameDialog(root, "report.txt").result
```

## CODE EXPLAINED

- → `transient` + `grab_set` = classic modal behavior
- → Entry text is pre-selected so typing replaces it
- → Enter and Escape map to OK and Cancel
- → Geometry is computed from the parent's position
- → `wait_window()` returns after `destroy()`, then `.result` is read

## MODAL RECIPE

| STEP | CALL |
|---|---|
| Create | Toplevel(parent) |
| Stay on top | transient(parent) |
| Capture input | grab_set() |
| Block caller | wait_window() |
| Return value | self.result |

## GRAB METHODS

| METHOD | DOES |
|---|---|
| grab_set() | Local grab (this app) |
| grab_set_global() | Whole display (avoid) |
| grab_release() | Release |
| grab_current() | Who holds the grab |
| grab_status() | local / global / None |

## WAIT METHODS

| METHOD | WAITS FOR |
|---|---|
| wait_window(w) | w destroyed |
| wait_visibility(w) | w visible |
| wait_variable(v) | v written |

## ⚠ COMMON MISTAKES

- Forgetting `grab_set()`, so users click the main window
- Reading `.result` before `wait_window()` returns
- Using `grab_set_global()`, which locks the whole desktop

## ✓ TIPS & BEST PRACTICES

- Return `None` for Cancel, a value for OK
- Wrap the pattern in a base class for all your dialogs
- Keep dialogs non-resizable unless content needs it

## QUICK SUMMARY

- Toplevel + transient + grab_set + wait_window
- Enter = OK, Escape = Cancel
- Center on the parent window
- Caller reads `dialog.result`

# ttk Introduction & Themes



## THEMED WIDGETS AND THEMES

`tkinter.ttk` provides **themed** versions of most widgets plus several new ones (Combobox, Notebook, Progressbar, Treeview, Separator, Sizegrip). Their look comes from the active **theme** rather than per-widget color options, so the same code looks native on Windows (`vista`), macOS (`aqua`) and gets a clean cross-platform look with `clam`. `ttk.Style().theme_use()` reads or switches the theme for the whole app at once. Appearance changes go through **styles** (Sheet 34).

## NOTES

- Importing ttk doesn't replace tk widgets; use `ttk.` explicitly
- `from tkinter.ttk import *` after `from tkinter import *` shadows tk widgets
- Linux defaults to `default`; `clam` usually looks better
- Theme changes apply instantly to existing widgets
- No ttk version of Text, Canvas, Listbox, Menu or Toplevel

## LISTING AND SWITCHING THEMES

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
style = ttk.Style(root)
print(style.theme_names())    # ('winnative','clam','alt','default','classic','vista
print(style.theme_use())      # current theme, e.g. 'vista'

theme = tk.StringVar(value=style.theme_use())
box = ttk.Combobox(root, values=style.theme_names(),
                   textvariable=theme, state="readonly")
box.pack(padx=10, pady=10)
box.bind("<<ComboboxSelected>>", lambda e: style.theme_use(theme.get()))

ttk.Button(root, text="ttk.Button").pack(pady=4)
ttk.Checkbutton(root, text="ttk.Checkbutton").pack(pady=4)
ttk.Entry(root).pack(pady=4)
ttk.Progressbar(root, value=60, length=160).pack(pady=4)
tk.Button(root, text="classic tk.Button").pack(pady=4)  # unaffected

if root.tk.call("tk", "windowingsystem") == "x11":
    style.theme_use("clam")      # nicer default on Linux
root.mainloop()
```

## CODE EXPLAINED

- → `theme_names()` lists themes available on this platform
- → `theme_use()` with no argument returns the current theme
- → Choosing a theme restyles every ttk widget immediately
- → Classic `tk.Button` ignores ttk themes
- → Switch Linux apps to `clam` for a modern flat look

## BUILT-IN THEMES

| THEME | PLATFORM |
|---|---|
| `vista / xpnative` | Windows native |
| `winnative` | Windows classic |
| `aqua` | macOS native |
| `clam` | All; flat, modern |
| `alt / default` | All; basic |
| `classic` | All; old Motif look |

## TK → TTK

| CLASSIC | THEMED |
|---|---|
| `Button, Label` | ttk.Button, ttk.Label |
| `Entry, Frame` | ttk.Entry, ttk.Frame |
| `Checkbutton` | ttk.Checkbutton |
| `Scale, Spinbox` | ttk.Scale, ttk.Spinbox |
| `OptionMenu` | ttk.Combobox / OptionMenu |
| `PanedWindow` | ttk.Panedwindow |

## TTK-ONLY WIDGETS

| WIDGET | PURPOSE |
|---|---|
| `Combobox` | Entry + drop-down |
| `Notebook` | Tabs |
| `Progressbar` | Progress |
| `Treeview` | Tree / table |
| `Separator` | Divider line |
| `Sizegrip` | Resize corner |

## ⚠ COMMON MISTAKES

- Wildcard-importing tkinter and ttk in the wrong order
- Setting `bg=` / `fg=` on ttk widgets
- Hard-coding `vista` and crashing on macOS/Linux

## ✓ TIPS & BEST PRACTICES

- Use ttk for every widget it offers
- Check `theme_names()` before calling `theme_use()`
- Pick `clam` when you need consistent custom colors

## QUICK SUMMARY

- ttk = themed, native-looking widgets
- `Style().theme_use(name)` switches globally
- Appearance via styles, not options
- Six widgets exist only in ttk

# ttk.Style — configure & map



## STYLING TTK WIDGETS

A **style** is a named bundle of appearance settings. Every ttk widget uses a default style named after its class (`TButton`, `TLabel`, `TEntry`, `Treeview`). `Style.configure(name, **opts)` sets options for a style, and `Style.map(name, **opts)` sets **state-dependent** options as lists of `(state spec, value)` pairs, such as a different background while `active` or `pressed`. Create a custom style by **prefixing** a class style (`"Danger.TButton"`) and passing `style=` to the widget.

## NOTES

- Changing `TButton` restyles every ttk Button
- Prefixed styles inherit everything from the base style
- Some themes (vista, aqua) ignore many color options; use `clam`
- `lookup()` reads an effective option value
- Use `"."` to configure the root style for all widgets

## CUSTOM BUTTON STYLES

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
style = ttk.Style(root)
style.theme_use("clam")                    # honors color options

style.configure(".", font=("Segoe UI", 10))          # all widgets
style.configure("TButton", padding=(10, 4))
style.configure("Danger.TButton", foreground="white",
                background="#c62828", bordercolor="#8e0000")
style.map("Danger.TButton",
          background=[("pressed", "#8e0000"), ("active", "#e53935"),
                      ("disabled", "#bdbdbd")],
          foreground=[("disabled", "#eeeeee")])

style.configure("Title.TLabel", font=("Segoe UI", 16, "bold"))
style.configure("Hint.TLabel", foreground="gray40")

ttk.Label(root, text="Account", style="Title.TLabel").pack(anchor="w", padx=10)
ttk.Label(root, text="This can't be undone.", style="Hint.TLabel").pack(padx=10)
ttk.Button(root, text="Cancel").pack(side="left", padx=10, pady=10)
ttk.Button(root, text="Delete", style="Danger.TButton").pack(side="right", padx=10)
print(style.lookup("Danger.TButton", "background"))   # '#c62828'
root.mainloop()
```

## CODE EXPLAINED

→ `clam` respects background/foreground; native themes may not

→ `"."` sets defaults for every style

→ `Danger.TButton` inherits TButton and overrides colors

→ `map()` entries are checked in order; first matching state wins

→ Widgets opt in with `style="Name.TClass"`

## STYLE METHODS

| METHOD | DOES |
|---|---|
| `configure(s, **o)` | Set options |
| `map(s, **o)` | State-based options |
| `lookup(s, opt)` | Read effective value |
| `layout(s)` | Element tree (Sheet 35) |
| `theme_use(name)` | Switch theme |

## CLASS STYLE NAMES

| WIDGET | STYLE |
|---|---|
| `Button` | TButton |
| `Label / Entry` | TLabel / TEntry |
| `Frame` | TFrame |
| `Notebook` | TNotebook |
| `Treeview` | Treeview |

## STATE SPECS

| SPEC | MATCHES |
|---|---|
| `active` | Mouse over |
| `pressed` | Being clicked |
| `disabled` | state disabled |
| `focus` | Has focus |
| `!disabled` | NOT disabled |

## ⚠ COMMON MISTAKES

- Styling on `vista` / `aqua` and seeing no color change
- Naming a style `Danger` without the `.TButton` suffix
- Using `configure` for hover colors instead of `map`

## ✓ TIPS & BEST PRACTICES

- Define all styles in one function at startup
- Name styles by role: `Primary.TButton`, `Muted.TLabel`
- Put specific states ( `pressed` ) before general ones ( `active` )

## QUICK SUMMARY

- Styles named `Prefix.TClass`
- `configure` = static, `map` = per-state
- Widgets choose via `style=`
- `clam` for full color control

# ttk Layouts & Elements



## LAYOUTS, ELEMENTS AND CUSTOM THEMES

Under the hood, each ttk style is drawn from **elements** (border, padding, label, indicator…) arranged in a **layout** tree. `Style.layout(name)` shows or replaces that tree, and `element_options(element)` lists what each element accepts, which tells you which `configure()` options actually work in the current theme. `element_create()` builds new elements (for example from images), and `theme_create()` with `theme_settings()` defines a whole theme based on an existing parent theme.

## NOTES

- Layouts differ between themes; inspect, don't assume
- Removing an element from a layout hides that part
- Image elements enable fully custom-looking widgets
- `theme_settings()` can change an existing theme too
- Libraries like sv-ttk ship complete image-based themes

## INSPECT AND MODIFY A LAYOUT

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
style = ttk.Style(root); style.theme_use("clam")
print(style.layout("TButton"))
# [('Button.border', {'sticky': 'nswe', 'border': '1', 'children':
#   [('Button.focus', {... 'children': [('Button.padding', {...
#     'children': [('Button.label', {'sticky': 'nswe'})]})]})])]
print(style.element_options("Button.label"))  # ('compound', 'space', 'text', …)
# Notebook without the dotted focus ring around tab labels
style.layout("TNotebook.Tab", [("Notebook.tab", {"sticky": "nswe", "children": [
    ("Notebook.padding", {"side": "top", "sticky": "nswe", "children": [
        ("Notebook.label", {"side": "top", "sticky": ""})]})]})])
# A new theme derived from clam
style.theme_create("brand", parent="clam", settings={
    "TButton": {"configure": {"padding": 8, "background": "#00695c",
                              "foreground": "white"},
                "map": {"background": [("active", "#00897b")]}},
    "TLabel": {"configure": {"foreground": "#004d40"}},
})
style.theme_use("brand")
ttk.Label(root, text="Branded").pack(padx=20, pady=6)
ttk.Button(root, text="Go").pack(pady=6)
root.mainloop()
```

## CODE EXPLAINED

- → `layout()` returns a nested list of (element, options)
- → `element_options` reveals supported options per element
- → Replacing the Tab layout drops `Notebook.focus`
- → `settings` uses `configure` / `map` / `layout` keys per style
- → Derived themes inherit everything they don't override

## ELEMENT API

| METHOD | DOES |
|---|---|
| `element_names()` | Elements in theme |
| `element_options(e)` | Its options |
| `element_create()` | New element |
| `layout(s, spec)` | Set layout |
| `layout(s)` | Read layout |

## THEME API

| METHOD | DOES |
|---|---|
| `theme_create(n, parent)` | New theme |
| `theme_settings(n, d)` | Modify theme |
| `theme_use(n)` | Activate |
| `theme_names()` | List themes |

## LAYOUT KEYS

| KEY | MEANING |
|---|---|
| `side` | Pack side in parent |
| `sticky` | Stretch edges |
| `expand` | Take extra space |
| `border` | Border size |
| `children` | Nested elements |

## ⚠ COMMON MISTAKES

- Configuring options the theme's elements don't support
- Copying a layout from one theme into another blindly
- Creating the same theme twice ( `TclError` )

## ✓ TIPS & BEST PRACTICES

- Print `layout()` before editing it
- Derive custom themes from `clam`
- Reach for sv-ttk or ttkbootstrap before hand-building themes

## QUICK SUMMARY

- Styles = layouts of elements
- `layout()` and `element_options()` to inspect
- `theme_create` / `theme_settings` for themes
- Derive from `clam` for control

# ttk Core Widgets & State



## TTK BASICS AND WIDGET STATE

Core ttk widgets (`Button`, `Label`, `Entry`, `Checkbutton`, `Radiobutton`, `Frame`, `Scale`, `Scrollbar`) work like their classic counterparts but share a common **state** system. Each widget has a set of state flags (`disabled`, `focus`, `pressed`, `selected`, `readonly`, `invalid`, `alternate`…). `state(["disabled"])` sets flags, `state(["!disabled"])` clears them, and `instate(["selected"])` tests them. Styles map their looks to these same flags.

## NOTES

- `state()` with no args returns current flags
- `configure(state="disabled")` still works on most ttk widgets
- `invalid` is set by validation and can be styled
- `padding` is a ttk option on many widgets
- Classic-only options ( `bg` , `relief` ) raise `TclError`

## TOGGLING AND TESTING STATE

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
frm = ttk.Frame(root, padding=12); frm.pack()

agree = tk.BooleanVar()
name = ttk.Entry(frm, width=24)
check = ttk.Checkbutton(frm, text="I agree", variable=agree)
submit = ttk.Button(frm, text="Submit")
name.grid(row=0, column=0, pady=4)
check.grid(row=1, column=0, sticky="w")
submit.grid(row=2, column=0, pady=6)
def refresh(*_):
    ok = agree.get() and name.get().strip()
    submit.state(["!disabled"] if ok else ["disabled"])
    name.state(["!invalid"] if name.get().strip() else ["invalid"])
agree.trace_add("write", refresh)
name.bind("<KeyRelease>", refresh)
refresh()
ttk.Style().map("TEntry", fieldbackground=[("invalid", "#ffe0e0")])
print(submit.state())                 # ('disabled',)
print(check.instate(["selected"]))    # False
check.state(["alternate"])            # tri-state "mixed" look
root.mainloop()
```

## CODE EXPLAINED

→ `state(["disabled"])` / `["!disabled"]` toggle a flag

→ Submit is enabled only when both conditions hold

→ `invalid` flag plus a style map tints the Entry

→ `instate()` returns a bool for a flag test

→ `alternate` shows a Checkbutton as partially checked

## STATE FLAGS

| FLAG | MEANING |
|---|---|
| `disabled` | Not interactive |
| `readonly` | No editing |
| `focus` | Has keyboard focus |
| `selected` | Checked / on |
| `invalid` | Failed validation |

## STATE METHODS

| CALL | DOES |
|---|---|
| `state()` | Current flags |
| `state(["f"])` | Set flag |
| `state(["!f"])` | Clear flag |
| `instate(["f"])` | Test flag |
| `instate(["f"], cb)` | Call cb if set |

## COMMON TTK OPTIONS

| OPTION | WIDGETS |
|---|---|
| `style` | All |
| `padding` | Button, Label, Frame |
| `width` | Button, Label, Entry |
| `cursor` | All |
| `takefocus` | Interactive widgets |

## ⚠ COMMON MISTAKES

- Passing a string instead of a list: `state("disabled")`
- Mixing `configure(state=…)` and `state()` inconsistently
- Expecting `instate` to return the flag list

## ✓ TIPS & BEST PRACTICES

- Drive enabled/disabled from one `refresh()` function
- Style the `invalid` state for validation feedback
- Use `alternate` for "some selected" parent checkboxes

## QUICK SUMMARY

- Shared state flags across ttk
- `state([...])` sets/clears with `!`
- `instate([...])` tests
- Styles map looks to states

# Combobox & ttk.Spinbox



## DROP-DOWNS AND SPIN INPUTS

`ttk.Combobox` combines an Entry with a drop-down list from `values`. By default users can type anything; `state="readonly"` restricts them to the list, which suits fixed choices. Read the value with `get()` or a `textvariable`, the position with `current()`, and react with `<<ComboboxSelected>>`. `ttk.Spinbox` steps through a range or list with arrows, and `ttk.Scale` is a themed slider whose value is a float you round yourself.

## NOTES

- `values` can be updated at any time with `configure()`
- `current(i)` selects by index; `current()` returns it (-1 if none)
- `<<ComboboxSelected>>` fires only on list picks, not typing
- `postcommand` refreshes the list just before it opens
- ttk.Spinbox fires `<<Increment>>` and `<<Decrement>>`

## COUNTRY / STATE PICKER

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
data = {"USA": ["California", "Texas", "Wisconsin"],
        "Canada": ["Ontario", "Quebec", "Alberta"]}

country = ttk.Combobox(root, values=list(data), state="readonly", width=18)
region = ttk.Combobox(root, state="readonly", width=18)
country.grid(row=0, column=0, padx=8, pady=6)
region.grid(row=0, column=1, padx=8)

def on_country(event):
    region.configure(values=data[country.get()])
    region.current(0)
country.bind("<<ComboboxSelected>>", on_country)
country.current(0); on_country(None)
qty = ttk.Spinbox(root, from_=1, to=20, width=5, wrap=True)
qty.set(1); qty.grid(row=1, column=0, sticky="w", padx=8)
zoom = tk.DoubleVar(value=100)
ttk.Scale(root, from_=25, to=400, variable=zoom,
          command=lambda v: zoom.set(round(float(v) / 25) * 25)
          ).grid(row=1, column=1, sticky="ew", padx=8)
ttk.Label(root, textvariable=zoom).grid(row=2, column=1)
root.mainloop()
```

## CODE EXPLAINED

- → `readonly` limits choices to `values`
- → Selecting a country reloads the region list
- → `current(0)` selects the first entry in code
- → `Spinbox.set()` sets the shown value
- → The Scale callback snaps the float to steps of 25

## COMBOBOX

| MEMBER | PURPOSE |
|---|---|
| `values` | Choice list |
| `state` | normal / readonly |
| `current(i)` | Select / get index |
| `get() / set(v)` | Read / write text |
| `postcommand` | Refresh before open |

## TTK.SPINBOX

| OPTION | PURPOSE |
|---|---|
| `from_ / to` | Range |
| `increment` | Step |
| `values` | Fixed list |
| `wrap` | Loop at ends |
| `command` | On arrow click |

## TTK.SCALE

| OPTION | PURPOSE |
|---|---|
| `from_ / to` | Range |
| `variable` | Linked DoubleVar |
| `orient` | horizontal / vertical |
| `length` | Pixel length |
| `command` | On change (str) |

## ⚠ COMMON MISTAKES

- Binding `<<ComboboxSelected>>` and expecting typed input to fire it
- Using `current()` result -1 as a valid index
- Expecting ttk.Scale to support `resolution`

## ✓ TIPS & BEST PRACTICES

- Use readonly Comboboxes for enums and categories
- Chain dependent Comboboxes with the selected event
- Show a Scale's value in a Label for clarity

## QUICK SUMMARY

- Combobox: `values`, `current()`, `readonly`
- `<<ComboboxSelected>>` on list picks
- ttk.Spinbox: arrows over range/list
- ttk.Scale: float value, round yourself

# Notebook & ttk.Panedwindow



## TABS AND SPLIT PANES

`ttk.Notebook` shows one child at a time behind a row of **tabs**. Each page is usually a Frame added with `add(child, text=…)`. `select()` switches tabs, `tab()` reads or changes tab options, and `<<NotebookTabChanged>>` fires when the tab changes. `enable_traversal()` adds Ctrl+Tab navigation. `ttk.Panedwindow` is the themed splitter: add panes with `add(child, weight=n)` and users drag the sash between them.

## NOTES

- Tabs can be identified by index, child widget or `"current"`
- `hide(tab)` hides a tab; `add()` the same child to show it again
- `forget(tab)` removes it entirely
- `state="disabled"` greys out a tab
- ttk.Panedwindow uses `weight`, not `minsize` / `stretch`

## SETTINGS WINDOW WITH TABS

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk(); root.geometry("420x260")
nb = ttk.Notebook(root)
nb.pack(fill="both", expand=True, padx=6, pady=6)

general, advanced, about = (ttk.Frame(nb, padding=10) for _ in range(3))
nb.add(general, text="General", underline=0)
nb.add(advanced, text="Advanced", underline=0)
nb.add(about, text="About", state="disabled")
nb.enable_traversal()            # Ctrl+Tab, Alt+G, Alt+A

ttk.Checkbutton(general, text="Start at login").pack(anchor="w")
panes = ttk.Panedwindow(advanced, orient="horizontal")
panes.pack(fill="both", expand=True)
panes.add(ttk.Treeview(panes, show="tree"), weight=1)
panes.add(tk.Text(panes, width=20), weight=3)
def changed(event):
    i = nb.index("current")
    root.title(f"Settings — {nb.tab(i, 'text')}")
nb.bind("<<NotebookTabChanged>>", changed)
nb.select(advanced)              # open on Advanced
nb.tab(about, state="normal")    # enable later
root.mainloop()
```

## CODE EXPLAINED

- → Each tab's page is a padded Frame
- → `underline` plus `enable_traversal()` adds Alt shortcuts
- → A disabled tab is visible but not selectable
- → Pane weights 1:3 split extra width
- → `index("current")` + `tab(i, "text")` read the active tab

## NOTEBOOK METHODS

| METHOD | DOES |
|---|---|
| `add(w, **o)` | Add tab |
| `select(tab)` | Switch / get current |
| `tab(tab, **o)` | Tab options |
| `hide(tab)` | Hide tab |
| `forget(tab)` | Remove tab |

## TAB OPTIONS

| OPTION | PURPOSE |
|---|---|
| `text` | Label |
| `image / compound` | Icon |
| `state` | normal/disabled/hidden |
| `underline` | Mnemonic |
| `sticky` | Page stretch |

## TTK.PANEDWINDOW

| METHOD | DOES |
|---|---|
| `add(w, weight=n)` | Add pane |
| `insert(pos, w)` | Insert pane |
| `forget(w)` | Remove pane |
| `sashpos(i, x)` | Get/set sash |
| `panes()` | List panes |

## ⚠ COMMON MISTAKES

- Packing pages into the Notebook instead of `add()`
- Using `select()` return value as an index (it's a widget name)
- Passing `minsize=` to ttk.Panedwindow panes

## ✓ TIPS & BEST PRACTICES

- Build each tab as its own Frame subclass
- Load expensive tabs lazily on first `<<NotebookTabChanged>>`
- Save `sashpos` in settings to restore layouts

## QUICK SUMMARY

- Notebook: `add`, `select`, `tab`
- `<<NotebookTabChanged>>` event
- `enable_traversal()` for keyboard
- Panedwindow: `add(w, weight=n)`

# Progressbar, Separator & Sizegrip



## PROGRESS, DIVIDERS AND RESIZE GRIPS

`ttk.Progressbar` shows progress in two modes. **Determinate** fills toward `maximum` as you set `value` (or call `step()`). **Indeterminate** bounces back and forth with `start(ms)` / `stop()` when the total is unknown. Update the bar from the main thread, usually inside `after()` callbacks; a loop that blocks the event loop freezes it. `ttk.Separator` draws a thin horizontal or vertical divider, and `ttk.Sizegrip` adds the resize handle in a window's bottom-right corner.

## NOTES

- `maximum` defaults to 100
- `step(n)` adds n and wraps at `maximum`
- Bind the bar to a `DoubleVar` with `variable=`
- Separators need `fill="x"` / `sticky="ew"` to stretch
- macOS shows its own resize corner; Sizegrip is optional there

## DOWNLOAD SIMULATION

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk(); root.geometry("360x180")
progress = tk.DoubleVar()

ttk.Label(root, text="Downloading…").pack(anchor="w", padx=10, pady=(10, 0))
bar = ttk.Progressbar(root, mode="determinate", maximum=50,
                      variable=progress, length=320)
bar.pack(padx=10, pady=6)
spinner = ttk.Progressbar(root, mode="indeterminate", length=320)
spinner.pack(padx=10)
ttk.Separator(root, orient="horizontal").pack(fill="x", pady=10)
status = ttk.Label(root, text="0%"); status.pack(side="left", padx=10)
ttk.Sizegrip(root).pack(side="right", anchor="se")
def tick():
    if progress.get() < bar["maximum"]:
        progress.set(progress.get() + 1)
        status.config(text=f"{progress.get() / bar['maximum']:.0%}")
        root.after(80, tick)          # schedule next update
    else:
        spinner.stop(); status.config(text="Done")
spinner.start(15)                     # ms per animation step
root.after(200, tick)
root.mainloop()
```

## CODE EXPLAINED

- → `maximum=50` means 50 units fill the bar
- → `variable=progress` drives the bar from a DoubleVar
- → The indeterminate bar animates on its own after `start()`
- → `after(80, tick)` updates without blocking the loop
- → Sizegrip packs into the bottom-right corner

## PROGRESSBAR

| MEMBER | PURPOSE |
|---|---|
| `mode` | determinate / indeterminate |
| `maximum` | Full value |
| `value` | Current value |
| `variable` | Linked var |
| `start(ms) / stop()` | Auto animate |

## SEPARATOR

| OPTION | USE |
|---|---|
| `orient=horizontal` | Row divider |
| `orient=vertical` | Column divider |
| `pack(fill="x")` | Stretch with pack |
| `grid(sticky="ns")` | Stretch with grid |

## MODE CHOICE

| SITUATION | MODE |
|---|---|
| `Known total` | determinate |
| `Unknown total` | indeterminate |
| `Many files` | determinate by count |
| `Network wait` | indeterminate |

## ⚠ COMMON MISTAKES

- Updating the bar in a `for` loop with `time.sleep` (frozen UI)
- Forgetting `stop()` on an indeterminate bar
- Setting `value` above `maximum`

## ✓ TIPS & BEST PRACTICES

- Pair the bar with a text percentage for accessibility
- Feed progress from threads through a queue (Sheet 51)
- Use `length=` for a consistent bar width

## QUICK SUMMARY

- Determinate: `value` / `maximum`
- Indeterminate: `start()` / `stop()`
- Update via `after()`, not loops
- Separator divides, Sizegrip resizes

# Treeview — Basics



## TABLES AND TREES WITH TREEVIEW

`ttk.Treeview` displays **hierarchical** items, **tabular** rows with columns, or both. Declare data columns with `columns=(…)`, then set each header with `heading()` and width with `column()`. `show="headings"` hides the tree column for a plain table, and `show="tree"` shows only the tree. `insert(parent, index, values=…)` adds a row and returns its **item id**; use `""` as the parent for top-level items. `item()` reads or updates a row, and `delete()` removes rows.

## NOTES

- Column `#0` is the tree column (labels via `text=`)
- Item ids are strings; pass `iid=` to choose your own
- `get_children()` lists items under a parent
- Values are stored as strings; convert when reading
- Add a Scrollbar with `yscrollcommand` like a Listbox

## EMPLOYEE TABLE

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
cols = ("name", "dept", "salary")
tree = ttk.Treeview(root, columns=cols, show="headings", height=8)
sb = ttk.Scrollbar(root, orient="vertical", command=tree.yview)
tree.configure(yscrollcommand=sb.set)
tree.grid(row=0, column=0, sticky="nsew"); sb.grid(row=0, column=1, sticky="ns")
root.columnconfigure(0, weight=1); root.rowconfigure(0, weight=1)

for c, title, w, anchor in [("name", "Name", 160, "w"),
                            ("dept", "Department", 120, "w"),
                            ("salary", "Salary", 90, "e")]:
    tree.heading(c, text=title, anchor=anchor)
    tree.column(c, width=w, anchor=anchor, stretch=(c == "name"))
rows = [("Ada Lovelace", "R&D", 98000), ("Alan Turing", "R&D", 102000),
        ("Grace Hopper", "Ops", 95000)]
for r in rows:
    tree.insert("", "end", values=(r[0], r[1], f"{r[2]:,}"))
iid = tree.insert("", 0, iid="new", values=("New Hire", "Sales", "60,000"))
tree.item(iid, values=("Jo Doe", "Sales", "61,000"))   # update row
print(tree.item(iid, "values"))           # ('Jo Doe', 'Sales', '61,000')
print(len(tree.get_children()))           # 4
tree.delete(*tree.get_children()[2:])     # remove rows 3+
root.mainloop()
```

## CODE EXPLAINED

- → `show="headings"` hides the tree column for a flat table
- → `heading()` sets titles; `column()` sets width and alignment
- → `stretch` lets only the Name column absorb extra width
- → `insert("", "end", values=…)` appends a top-level row
- → `item(iid, values=…)` edits a row in place

## SETUP

| CALL | PURPOSE |
|---|---|
| `columns=(…)` | Column ids |
| `show=` | tree / headings / both |
| `heading(c, text=)` | Header label |
| `column(c, width=)` | Width |
| `column(c, anchor=)` | Alignment |

## ITEMS

| METHOD | DOES |
|---|---|
| `insert(p, i, …)` | Add; returns iid |
| `item(iid, **o)` | Read / update |
| `set(iid, col, v)` | One cell |
| `delete(*iids)` | Remove |
| `get_children(p)` | Child ids |

## INSERT() OPTIONS

| OPTION | PURPOSE |
|---|---|
| `text` | Tree column label |
| `values` | Column values |
| `iid` | Custom id |
| `open` | Expanded? |
| `tags` | Style/event tags |

## ⚠ COMMON MISTAKES

- Forgetting `show="headings"` and getting an empty first column
- Reusing an `iid` (raises `TclError`)
- Sorting numeric columns as strings

## ✓ TIPS & BEST PRACTICES

- Use database primary keys as `iid` for easy lookup
- Right-align numeric columns with `anchor="e"`
- Clear with `delete(*tree.get_children())`

## QUICK SUMMARY

- `columns`, `heading`, `column` define the table
- `insert("", "end", values=…)` adds rows
- Item ids identify rows
- `show="headings"` for flat tables

# Treeview — Selection, Tags & Sorting



## WORKING WITH TREEVIEW DATA

Treeview selections are **item ids**: `selection()` returns the selected ids, and `<<TreeviewSelect>>` fires when it changes. `focus()` gives the item with keyboard focus. For hierarchies, pass a parent id to `insert()`, open or close nodes with `item(iid, open=True)`, and react to `<<TreeviewOpen>>` to **lazy-load** children. **Tags** style rows (striping, highlighting) and can receive events via `tag_bind()`. Sorting isn't built in; reorder rows with `move()` from a heading's `command`.

## NOTES

- `selectmode`: `extended` (default), `browse`, `none`
- `see(iid)` scrolls an item into view
- `identify_row(y)` finds the row under the mouse
- `parent(iid)` and `next(iid)` navigate the tree
- Tag colors need the `clam` -style themes on some platforms

## SORTABLE, STRIPED, HIERARCHICAL

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
tree = ttk.Treeview(root, columns=("size",)); tree.pack(fill="both", expand=True)
tree.heading("#0", text="Name")
tree.tag_configure("odd", background="#f3f3f8")
tree.tag_configure("big", foreground="#c62828")

docs = tree.insert("", "end", text="Documents", open=True)
for i, (name, kb) in enumerate([("cv.pdf", 120), ("video.mp4", 9800),
                                ("notes.txt", 4)]):
    tags = (("odd",) if i % 2 else ()) + (("big",) if kb > 1000 else ())
    tree.insert(docs, "end", text=name, values=(kb,), tags=tags)
def sort_by(col, reverse=False):
    for parent in ("",) + tree.get_children(""):
        kids = list(tree.get_children(parent))
        kids.sort(key=lambda k: float(tree.set(k, col) or 0), reverse=reverse)
        for idx, k in enumerate(kids):
            tree.move(k, parent, idx)
    tree.heading(col, command=lambda: sort_by(col, not reverse))
tree.heading("size", text="KB", command=lambda: sort_by("size"))
tree.bind("<<TreeviewSelect>>",
          lambda e: print([tree.item(i, "text") for i in tree.selection()]))
tree.bind("<Double-1>", lambda e: print("open", tree.identify_row(e.y)))
root.mainloop()
```

## CODE EXPLAINED

- → `#0` is the tree column; its header is set like any other
- → Children are inserted under the `docs` parent id
- → Tags combine: striping plus a red highlight for big files
- → `move()` reorders rows; the heading toggles sort direction
- → `identify_row(e.y)` maps a click to an item id

## SELECTION

| MEMBER | PURPOSE |
|---|---|
| `selection()` | Selected ids |
| `selection_set()` | Replace selection |
| `selection_add()` | Add to selection |
| `focus()` | Focused item |
| `selectmode` | extended/browse/none |

## TREE NAVIGATION

| METHOD | RETURNS |
|---|---|
| `parent(iid)` | Parent id |
| `next() / prev()` | Sibling id |
| `index(iid)` | Position |
| `move(iid, p, i)` | Reparent / reorder |
| `see(iid)` | Scroll into view |

## EVENTS

| EVENT | FIRES ON |
|---|---|
| `<<TreeviewSelect>>` | Selection change |
| `<<TreeviewOpen>>` | Node expanded |
| `<<TreeviewClose>>` | Node collapsed |
| `<Double-1>` | Double-click |
| `tag_bind()` | Events on tagged rows |

## ⚠ COMMON MISTAKES

- Treating `selection()` results as row values
- Sorting with string comparison ("100" < "9")
- Loading huge trees up front instead of lazily

## ✓ TIPS & BEST PRACTICES

- Lazy-load children on `<<TreeviewOpen>>` with a dummy child
- Re-apply stripe tags after sorting
- Keep a dict mapping `iid` → your data object

## QUICK SUMMARY

- `selection()` returns item ids
- Parents make hierarchies; `open=` expands
- Tags style and bind rows
- Sort via `move()` in heading commands

# Canvas Basics & Shapes



## DRAWING ON A CANVAS

A `Canvas` is a drawing surface for lines, shapes, text, images and even embedded widgets. Each `create_*()` call adds an **item** and returns its integer **id**. Coordinates are pixels from the canvas's top-left, with **y growing downward**. Rectangles and ovals are defined by a **bounding box** `(x1, y1, x2, y2)`, lines and polygons by a list of points. Unlike drawing in many libraries, items are **retained objects**: you can move, recolor or delete them later without redrawing everything.

## NOTES

- Ovals are drawn inside their bounding box
- `fill=""` means no fill (transparent interior)
- `width` is the outline thickness
- Later items draw on top of earlier ones
- Use `highlightthickness=0` to remove the focus border

## BASIC SHAPES

```
import tkinter as tk

root = tk.Tk()
c = tk.Canvas(root, width=360, height=240, bg="white", highlightthickness=0)
c.pack()

c.create_line(10, 10, 350, 10, fill="gray", dash=(4, 2))
c.create_line(20, 200, 80, 140, 140, 180, 200, 120,
              fill="navy", width=3, smooth=True, arrow="last")
c.create_rectangle(20, 30, 120, 100, fill="#90caf9", outline="#1565c0", width=2)
c.create_oval(140, 30, 240, 100, fill="#ffcc80", outline="")
c.create_oval(260, 30, 330, 100, outline="green", width=4)       # circle
c.create_polygon(260, 220, 300, 140, 340, 220, fill="#a5d6a7",
                 outline="black")
c.create_arc(140, 120, 240, 220, start=30, extent=300,
             style="pieslice", fill="gold")                      # Pac-Man

c.create_text(180, 230, text="Canvas demo", font=("Helvetica", 10, "bold"))
c.create_rectangle(0, 0, 360, 240, outline="red", width=1)
root.mainloop()
```

## CODE EXPLAINED

- `dash=(4, 2)` draws 4 px on, 2 px off
- Multi-point lines can be smoothed into curves
- Rectangles and ovals take two opposite corners
- `create_arc` uses `start` / `extent` in degrees
- `create_text` centers on the point by default

## CREATE_* METHODS

| METHOD | COORDS |
|---|---|
| `create_line` | x1,y1, x2,y2, … |
| `create_rectangle` | x1,y1, x2,y2 |
| `create_oval` | Bounding box |
| `create_polygon` | x1,y1, … (closed) |
| `create_arc` | Bounding box |

## SHAPE OPTIONS

| OPTION | EFFECT |
|---|---|
| `fill` | Interior / line color |
| `outline` | Border color |
| `width` | Line thickness |
| `dash` | Dash pattern |
| `stipple` | Bitmap pattern |

## LINE & ARC OPTIONS

| OPTION | VALUES |
|---|---|
| `arrow` | first / last / both |
| `smooth` | True for curves |
| `capstyle` | butt / round / projecting |
| `style (arc)` | pieslice / chord / arc |
| `start, extent` | Degrees |

## ⚠ COMMON MISTAKES

- Expecting y to increase upward like math plots
- Passing width/height to `create_oval` instead of corners
- Using `fill` on an arc with `style="arc"` (no interior)

## ✓ TIPS & BEST PRACTICES

- Wrap repeated shapes in helper functions
- Store returned ids for items you'll change later
- Use `outline=""` for flat, borderless shapes

## QUICK SUMMARY

- `create_*` returns an item id
- Pixel coords, y downward
- Boxes for rect/oval/arc, points for lines
- Items persist and can be changed

# Canvas Items & Tags

item IDs · tags= · itemconfigure() · coords() · move() · delete() · find_withtag() · tag_raise()

## CHANGING ITEMS WITH IDS AND TAGS

Every canvas item has a unique integer **id** and any number of string **tags**. Methods that take an item accept either form: an id affects one item, a tag affects **every item carrying it**, and the special tag `"all"` means everything. `itemconfigure()` changes options, `coords()` reads or replaces coordinates, `move()` shifts by an offset, and `delete()` removes items. `find_withtag()` and friends return matching ids, and `tag_raise()`/`tag_lower()` change stacking order.

## NOTES

- Tags are added with `tags=` or `addtag_withtag()`
- `"current"` is the item under the mouse pointer
- `gettags(id)` lists an item's tags
- `type(id)` returns `"oval"`, `"line"`, etc.
- `moveto()` (Python 3.8+) moves to absolute coordinates

## GROUPING WITH TAGS

```
import tkinter as tk

root = tk.Tk()
c = tk.Canvas(root, width=360, height=200, bg="white"); c.pack()

for i in range(5):                      # five "pieces" in two groups
    x = 20 + i * 68
    team = "red" if i % 2 else "blue"
    c.create_oval(x, 80, x + 50, 130, fill=team, tags=("piece", team))

label = c.create_text(180, 20, text="Click a key: r, b, d, s")
def recolor(team, color):
    c.itemconfigure(team, fill=color)          # every item tagged team
root.bind("r", lambda e: recolor("red", "salmon"))
root.bind("b", lambda e: c.move("blue", 0, 20))   # shift blue pieces
root.bind("d", lambda e: c.delete("red"))
root.bind("s", lambda e: c.itemconfigure(label,
          text=f"{len(c.find_withtag('piece'))} pieces"))
first = c.find_withtag("piece")[0]
print(c.coords(first))                  # [20.0, 80.0, 70.0, 130.0]
c.coords(first, 20, 150, 70, 190)       # replace coordinates
c.addtag_withtag("selected", first)
print(c.gettags(first))                 # ('piece', 'blue', 'selected')
c.tag_raise("selected")
root.mainloop()
```

## CODE EXPLAINED

→ Each oval carries two tags: `piece` and its team

→ `itemconfigure("red", …)` changes all red items at once

→ `move(tag, dx, dy)` shifts by an offset

→ `find_withtag` returns a tuple of ids

→ `coords(id, …)` replaces an item's coordinates

## MODIFY ITEMS

| METHOD | DOES |
|---|---|
| `itemconfigure(t, **o)` | Change options |
| `itemcget(t, opt)` | Read option |
| `coords(t, *xy)` | Get / set coords |
| `move(t, dx, dy)` | Shift |
| `delete(t)` | Remove |

## FIND ITEMS

| METHOD | RETURNS |
|---|---|
| `find_withtag(t)` | Ids with tag |
| `find_all()` | Every id |
| `find_closest(x, y)` | Nearest item |
| `find_overlapping()` | Items in box |
| `find_enclosed()` | Items fully inside |

## TAG METHODS

| METHOD | DOES |
|---|---|
| `addtag_withtag(n, t)` | Add tag n |
| `dtag(t, n)` | Remove tag n |
| `gettags(id)` | List tags |
| `tag_raise(t)` | Bring to front |
| `tag_lower(t)` | Send to back |

## ⚠ COMMON MISTAKES

- Using numeric strings as tags ( `"1"` looks like an id)
- Calling `coords()` with too few numbers for the item type
- Forgetting `"all"` matches the background too

## ✓ TIPS & BEST PRACTICES

- Tag items by role and by group
- Use `delete("all")` to clear before redrawing
- Prefer tags over storing long lists of ids

## QUICK SUMMARY

- Ids for one item, tags for groups
- `itemconfigure`, `coords`, `move`, `delete`
- `find_*` methods return ids
- `tag_raise` / `tag_lower` set stacking

# Canvas Text, Images & Windows



## TEXT, IMAGES AND WIDGETS ON A CANVAS

Besides shapes, a canvas can hold **text items** (`create_text`), **images** (`create_image` with a `PhotoImage`) and real **widgets** (`create_window`). For all three, `anchor` says which point of the item sits on the given coordinate: `"center"` by default, `"nw"` for top-left. `bbox(item)` returns the bounding box `(x1, y1, x2, y2)` of any item or tag, which is how you size a background behind text or check whether things overlap.

## NOTES

- Text items support `width=` for wrapping and `justify=`
- Keep a Python reference to images or they disappear
- Embedded widgets always draw above other canvas items
- `itemcget(id, "text")` reads a text item's string
- Canvas text can be edited with `dchars()` / `insert()`

## LABELLED CARD WITH IMAGE AND BUTTON

```
import tkinter as tk

root = tk.Tk()
c = tk.Canvas(root, width=320, height=200, bg="#263238",
              highlightthickness=0)
c.pack()

img = tk.PhotoImage(file="avatar.png").subsample(2)
c.image = img                                  # keep a reference
c.create_image(20, 20, image=img, anchor="nw", tags="card")

title = c.create_text(110, 30, text="Ada Lovelace", anchor="nw",
                      fill="white", font=("Helvetica", 14, "bold"))
c.create_text(110, 60, anchor="nw", width=190, fill="#b0bec5",
              text="Wrote the first published algorithm for a machine.")

x1, y1, x2, y2 = c.bbox(title)                 # box around the name
bg = c.create_rectangle(x1 - 4, y1 - 2, x2 + 4, y2 + 2, fill="#37474f",
                        outline="")
c.tag_lower(bg, title)                         # put it behind the text

btn = tk.Button(c, text="Follow", command=lambda: print("followed"))
c.create_window(300, 180, window=btn, anchor="se")

print(c.itemcget(title, "text"))               # 'Ada Lovelace'
root.mainloop()
```

## CODE EXPLAINED

- → `anchor="nw"` places the image's top-left at (20, 20)
- → Storing `img` on the canvas prevents garbage collection
- → `width=190` wraps the description text
- → `bbox` + `tag_lower` creates a highlight behind text
- → `create_window` embeds a real Button at the corner

## ITEM KINDS

| METHOD | KEY OPTIONS |
|---|---|
| `create_text` | text, font, fill, width |
| `create_image` | image, anchor |
| `create_window` | window, width, height |
| `create_bitmap` | bitmap (built-ins) |

## ANCHOR POINTS

| VALUE | POINT ON ITEM |
|---|---|
| `center` | Middle (default) |
| `nw / ne` | Top corners |
| `sw / se` | Bottom corners |
| `n / s` | Top / bottom middle |
| `w / e` | Left / right middle |

## MEASURING

| CALL | RETURNS |
|---|---|
| `bbox(t)` | (x1, y1, x2, y2) |
| `bbox("all")` | Box of everything |
| `coords(t)` | Item coordinates |
| `itemcget(t, o)` | Option value |
| `type(t)` | Item type |

## ⚠ COMMON MISTAKES

- Creating a PhotoImage in a function and losing it
- Expecting canvas items to draw over embedded widgets
- Using `x` / `y` of text as its top-left without `anchor="nw"`

## ✓ TIPS & BEST PRACTICES

- Use `anchor="nw"` for layout-style positioning
- Measure text with `bbox` before placing neighbors
- Embed forms with `create_window` inside scrollable canvases

## QUICK SUMMARY

- `create_text`, `create_image`, `create_window`
- `anchor` picks the reference point
- `bbox()` measures items
- Keep image references alive

# Canvas Events & Dragging

tag_bind() · find_closest() · find_overlapping() · canvasx() · canvasy() · drag-and-drop pattern

## INTERACTIVE CANVAS ITEMS

`tag_bind(tag_or_id, sequence, handler)` attaches events to canvas **items** rather than the whole widget, so each shape can react to clicks, hovers or drags. Inside a handler, the special tag `"current"` names the item under the pointer. Dragging is a three-event pattern: on **press**, remember the item and pointer position; on **motion**, `move()` the item by the difference; on **release**, finish. `find_closest()` and `find_overlapping()` help with hit-testing and snapping.

## NOTES

- Item bindings fire only over painted areas, not empty interiors with `fill=""`
- Widget-level bindings still run after item bindings
- Use `canvasx()` / `canvasy()` when the canvas is scrolled
- `<Enter>` / `<Leave>` on tags give hover effects
- `find_overlapping` takes a rectangle, not a point

## DRAG-AND-DROP SHAPES

```
import tkinter as tk
c = tk.Canvas(tk.Tk(), width=400, height=260, bg="white"); c.pack()
for x, color in ((40, "tomato"), (160, "gold"), (280, "skyblue")):
    c.create_rectangle(x, 100, x + 80, 160, fill=color, tags="draggable")
c.create_rectangle(150, 200, 250, 250, dash=(3, 3), tags="target")
drag = {"item": None, "x": 0, "y": 0}
def press(e):
    drag["item"] = c.find_withtag("current")[0]
    drag["x"], drag["y"] = c.canvasx(e.x), c.canvasy(e.y)
    c.tag_raise(drag["item"])
def motion(e):
    x, y = c.canvasx(e.x), c.canvasy(e.y)
    c.move(drag["item"], x - drag["x"], y - drag["y"])
    drag["x"], drag["y"] = x, y
def release(e):
    hits = c.find_overlapping(*c.bbox("target"))
    if drag["item"] in hits:
        c.itemconfigure(drag["item"], outline="green", width=3)
    drag["item"] = None
c.tag_bind("draggable", "<ButtonPress-1>", press)
c.tag_bind("draggable", "<B1-Motion>", motion)
c.tag_bind("draggable", "<ButtonRelease-1>", release)
c.tag_bind("draggable", "<Enter>", lambda e: c.config(cursor="fleur"))
c.tag_bind("draggable", "<Leave>", lambda e: c.config(cursor=""))
c.mainloop()
```

## CODE EXPLAINED

- → Only items tagged `draggable` respond to the mouse
- → `"current"` gives the item that was clicked
- → Motion moves by the delta since the last event
- → `find_overlapping(*bbox)` checks for a drop on the target
- → Cursor changes on hover signal draggability

## ITEM EVENTS

| CALL | EFFECT |
|---|---|
| `tag_bind(t, ev, fn)` | Bind item events |
| `tag_unbind(t, ev)` | Remove binding |
| `"current"` tag | Item under pointer |
| `find_closest(x, y)` | Nearest item |
| `find_overlapping()` | Items in rectangle |

## DRAG PATTERN

| EVENT | ACTION |
|---|---|
| `<ButtonPress-1>` | Record item + pos |
| `<B1-Motion>` | move() by delta |
| `<ButtonRelease-1>` | Drop / snap |
| `<Enter>/<Leave>` | Cursor feedback |

## COORDINATES

| VALUE | SPACE |
|---|---|
| `e.x, e.y` | Widget pixels |
| `canvasx(e.x)` | Canvas coords |
| `canvasy(e.y)` | Canvas coords |
| `e.x_root` | Screen pixels |

## ⚠ COMMON MISTAKES

- Using `find_closest` for clicks (it always returns something)
- Forgetting `canvasx` / `canvasy` on scrolled canvases
- Binding drag events on the canvas, so the background drags too

## ✓ TIPS & BEST PRACTICES

- Store drag state in a dict or object, not globals
- Raise the dragged item so it stays on top
- Snap to a grid by rounding coordinates on release

## QUICK SUMMARY

- `tag_bind` = events per item or tag
- `"current"` = item under the mouse
- Press / motion / release for dragging
- `find_*` for hit-testing

# Canvas Scrolling & Animation



## LARGE DRAWINGS AND MOVING PICTURES

A canvas can be bigger than its window. Set `scrollregion` to the drawing's extent (often `bbox("all")`) and connect scrollbars with `xview`/`yview`, exactly as with a Listbox. `scale(tag, x, y, sx, sy)` zooms items around a point (it scales coordinates, not line widths or fonts). **Animation** means updating item positions on a timer: a function moves things, then reschedules itself with `after(ms, func)`. Never animate with `while True` + `time.sleep`, which freezes the window.

## NOTES

- About 16 ms per frame ≈ 60 fps; 33 ms ≈ 30 fps
- `after_cancel(id)` stops a scheduled frame
- `xview_moveto(0)` jumps to the left edge
- Mouse wheel scrolling needs your own bindings
- For heavy graphics, consider Pygame or OpenGL instead

## BOUNCING BALL ON A SCROLLABLE CANVAS

```
import tkinter as tk
root = tk.Tk()
W, H = 800, 600                                  # world size
c = tk.Canvas(root, width=400, height=300, bg="black",
              scrollregion=(0, 0, W, H))
hs = tk.Scrollbar(root, orient="horizontal", command=c.xview)
vs = tk.Scrollbar(root, orient="vertical", command=c.yview)
c.configure(xscrollcommand=hs.set, yscrollcommand=vs.set)
c.grid(row=0, column=0); vs.grid(row=0, column=1, sticky="ns")
hs.grid(row=1, column=0, sticky="ew")
ball = c.create_oval(10, 10, 40, 40, fill="orange")
vel = [4, 3]
def frame():
    x1, y1, x2, y2 = c.coords(ball)
    if x1 + vel[0] < 0 or x2 + vel[0] > W: vel[0] = -vel[0]
    if y1 + vel[1] < 0 or y2 + vel[1] > H: vel[1] = -vel[1]
    c.move(ball, *vel)
    root.anim = root.after(16, frame)            # ~60 fps
def zoom(factor):
    c.scale("all", 0, 0, factor, factor)
    c.configure(scrollregion=c.bbox("all"))
root.bind("+", lambda e: zoom(1.25)); root.bind("-", lambda e: zoom(0.8))
root.bind("<space>", lambda e: root.after_cancel(root.anim))
frame()
root.mainloop()
```

## CODE EXPLAINED

- → `scrollregion` defines the full 800×600 world
- → Scrollbars wire to `xview` / `yview` both ways
- → Each frame bounces off world edges, then moves
- → `after(16, frame)` schedules the next frame
- → `scale()` zooms; `scrollregion` is updated to match

## SCROLLING

| MEMBER | PURPOSE |
|---|---|
| `scrollregion` | (x1, y1, x2, y2) |
| `xview / yview` | Scroll commands |
| `xview_moveto(f)` | Jump to fraction |
| `yview_scroll(n, u)` | Scroll units/pages |
| `canvasx(px)` | Window → canvas x |

## ZOOM & TRANSFORM

| CALL | EFFECT |
|---|---|
| `scale(t, x, y, sx, sy)` | Scale coords |
| `move(t, dx, dy)` | Translate |
| `moveto(t, x, y)` | Absolute move |
| `bbox("all")` | Current extent |

## ANIMATION LOOP

| STEP | CODE |
|---|---|
| `Update state` | positions, velocity |
| `Redraw` | move() / coords() |
| `Schedule` | after(ms, frame) |
| `Stop` | after_cancel(id) |
| `Pace` | 16–33 ms |

## ⚠ COMMON MISTAKES

- Animating with `time.sleep` in a loop
- Forgetting to update `scrollregion` after zooming
- Starting several `after` loops by calling `frame()` twice

## ✓ TIPS & BEST PRACTICES

- Keep the `after` id so you can pause and resume
- Move items instead of deleting and recreating them
- Base movement on elapsed time for steady speed

## QUICK SUMMARY

- `scrollregion` + scrollbars for big drawings
- `scale()` zooms coordinates
- Animate with `after()` loops
- Never block with `sleep`

# Colors, Reliefs & Borders



## COLORS AND BORDERS ON CLASSIC WIDGETS

Classic `tk` widgets accept colors as **names** (`"red"`, `"SteelBlue"`, around 750 X11 names) or **hex** strings `"#rgb"`, `"#rrggbb"` (or 12-digit `"#rrrrggggbbbb"`). Platform-specific **system colors** such as `"SystemButtonFace"` (Windows) or `"systemWindowBackgroundColor"` (macOS) follow the OS theme. Borders come from `relief` and `borderwidth`, and the **focus ring** from `highlightthickness`/`highlightcolor`. ttk widgets ignore these options; style them instead (Sheet 34).

## NOTES

- `winfo_rgb(color)` returns 16-bit (r, g, b) components
- `activebackground` applies on hover for buttons
- `disabledforeground` colors text in disabled widgets
- `selectbackground` colors selected text in Entry/Text
- Avoid relying on color alone to convey meaning

## RELIEFS, COLORS AND HIGHLIGHTS

```
import tkinter as tk

root = tk.Tk()
root.configure(bg="#eceff1")

reliefs = ("flat", "raised", "sunken", "groove", "ridge", "solid")
for i, r in enumerate(reliefs):
    tk.Label(root, text=r, relief=r, borderwidth=3, width=8,
             bg="white").grid(row=0, column=i, padx=4, pady=8)

btn = tk.Button(root, text="Hover me", bg="#1976d2", fg="white",
                activebackground="#0d47a1", activeforeground="white",
                relief="flat", padx=12, pady=4, cursor="hand2")
btn.grid(row=1, column=0, columnspan=3, pady=6)
entry = tk.Entry(root, highlightthickness=2, highlightcolor="#ff9800",
                 highlightbackground="#cfd8dc", selectbackground="#ffe0b2",
                 selectforeground="black", insertbackground="#ff9800")
entry.grid(row=1, column=3, columnspan=3)
def lighten(color, f=0.3):
    r, g, b = (v // 257 for v in root.winfo_rgb(color))
    r, g, b = (int(v + (255 - v) * f) for v in (r, g, b))
    return f"#{r:02x}{g:02x}{b:02x}"
print(lighten("SteelBlue"))          # '#7da7ca'
root.mainloop()
```

## CODE EXPLAINED

- → Six reliefs show the classic border styles
- → `active*` colors apply while the mouse is over the button
- → `highlight*` options style the Entry's focus ring
- → `insertbackground` sets the text cursor color
- → `winfo_rgb` converts any color spec to numbers

## COLOR FORMATS

| FORM | EXAMPLE |
|---|---|
| `Name` | "SteelBlue" |
| `#rgb` | "#f80" |
| `#rrggbb` | "#ff8800" |
| `System (Win)` | "SystemHighlight" |
| `System (mac)` | "systemTextColor" |

## COLOR OPTIONS

| OPTION | APPLIES TO |
|---|---|
| `bg / fg` | Normal state |
| `activebackground` | Hover / press |
| `disabledforeground` | Disabled text |
| `selectbackground` | Selected text |
| `insertbackground` | Text cursor |

## BORDER OPTIONS

| OPTION | EFFECT |
|---|---|
| `relief` | Border style |
| `borderwidth / bd` | Border size |
| `highlightthickness` | Focus ring size |
| `highlightcolor` | Ring when focused |
| `highlightbackground` | Ring unfocused |

## ⚠ COMMON MISTAKES

- Setting `bg` on a macOS `tk.Button` (native buttons ignore it)
- Using colors on ttk widgets instead of styles
- Choosing text/background pairs with poor contrast

## ✓ TIPS & BEST PRACTICES

- Keep a palette dict and reference colors by role
- Set `highlightthickness=0` for flat, borderless designs
- Use `cursor="hand2"` on clickable labels

## QUICK SUMMARY

- Names, `#rgb`, `#rrggbb`, system colors
- `relief` + `borderwidth` for borders
- `highlight*` for focus rings
- Classic widgets only; ttk uses styles

# Fonts



## WORKING WITH FONTS

Fonts can be given as a **tuple** `("Family", size, "bold italic")`, a **string** `"Helvetica 12 bold"`, or a `tkinter.font.Font` **object**. Font objects are live: change one with `configure()` and every widget using it updates. Positive sizes are **points**, negative sizes are **pixels**. Tk also defines **named fonts** like `TkDefaultFont`, `TkTextFont` and `TkFixedFont`; adjusting them with `nametofont()` restyles the whole app consistently, including ttk widgets.

## NOTES

- `font.families()` lists installed font families
- `measure(text)` gives pixel width; `metrics()` gives heights
- Unknown families silently fall back to a default
- Keep a reference to `Font` objects you create
- Import as `tkfont` to avoid clashing with variables named `font`

## NAMED FONTS AND FONT OBJECTS

```
import tkinter as tk
import tkinter.font as tkfont
from tkinter import ttk

root = tk.Tk()

default = tkfont.nametofont("TkDefaultFont")      # app-wide base font
default.configure(family="Segoe UI", size=10)
tkfont.nametofont("TkFixedFont").configure(size=11)

heading = tkfont.Font(family="Georgia", size=16, weight="bold")
ttk.Label(root, text="Report", font=heading).pack(anchor="w", padx=10)
ttk.Label(root, text="Uses TkDefaultFont").pack(anchor="w", padx=10)
tk.Text(root, height=3, font="TkFixedFont").pack(padx=10, pady=4)

def bigger(): heading.configure(size=int(heading.cget("size")) + 2)
ttk.Button(root, text="A+", command=bigger).pack(pady=4)

print(heading.measure("Report"))        # pixel width of the text
print(heading.metrics("linespace"))     # line height in px
print(heading.actual())                 # resolved family, size, weight…
print("Consolas" in tkfont.families())
mono = ("Courier New", 10)              # tuple form
small_italic = "Helvetica 9 italic"     # string form
root.mainloop()
```

## CODE EXPLAINED

→ `nametofont("TkDefaultFont")` changes the base font everywhere

→ A `Font` object updates all widgets using it on `configure()`

→ Text widgets can use named fonts by name string

→ `measure` and `metrics` help size layouts precisely

→ `actual()` shows what the OS really chose

## FONT SPECS

| FORM | EXAMPLE |
|---|---|
| `Tuple` | ("Arial", 12, "bold") |
| `String` | "Arial 12 bold" |
| `Spaces in name` | "{Segoe UI} 10" |
| `Pixels` | ("Arial", -14) |
| `Font object` | tkfont.Font(…) |

## NAMED FONTS

| NAME | USED BY |
|---|---|
| `TkDefaultFont` | Most widgets |
| `TkTextFont` | Entry, Text |
| `TkFixedFont` | Monospace |
| `TkHeadingFont` | Treeview headings |
| `TkMenuFont` | Menus |

## FONT METHODS

| METHOD | RETURNS |
|---|---|
| `configure(**o)` | Change font |
| `actual()` | Resolved attributes |
| `measure(text)` | Width in px |
| `metrics(opt)` | ascent/descent/linespace |
| `copy()` | Independent copy |

## ⚠ COMMON MISTAKES

- Writing `("Segoe UI 10")` (a string in parentheses, not a tuple)
- Creating `Font()` before `Tk()` exists
- Hard-coding a font family that isn't installed elsewhere

## ✓ TIPS & BEST PRACTICES

- Change named fonts once at startup for a consistent look
- Use `TkFixedFont` for code and tabular text
- Scale fonts, not widgets, for accessibility settings

## QUICK SUMMARY

- Tuple, string or `Font` object
- Positive = points, negative = pixels
- `nametofont` to restyle the whole app
- `measure` / `metrics` for sizing

# Images — PhotoImage & Pillow



## SHOWING IMAGES

`tk.PhotoImage` loads **PNG** and **GIF** (and PPM/PGM) natively; Tk 9.0 adds SVG support. It can scale by whole factors with `zoom(n)` and `subsample(n)`, and set pixels with `put()`. For **JPEG**, smooth resizing or other formats, use **Pillow**: open with `PIL.Image`, process, then wrap with `PIL.ImageTk.PhotoImage`. Whatever you use, **keep a reference** to the image object. If it's only a local variable, Python frees it and the widget goes blank.

## NOTES

- `pip install pillow` provides `PIL.Image` and `PIL.ImageTk`
- `compound="left"` shows image and text together
- `PhotoImage(width=, height=)` creates a blank canvas image
- Animated GIF frames load with `format="gif -index n"`
- Use `Image.LANCZOS` for high-quality downscaling

## PHOTOIMAGE AND PILLOW

```
import tkinter as tk
from tkinter import ttk
from PIL import Image, ImageTk, ImageOps

root = tk.Tk()

icon = tk.PhotoImage(file="save.png")                  # native PNG
small = icon.subsample(2)                              # half size
ttk.Button(root, text="Save", image=small, compound="left").pack(pady=4)

pil = Image.open("photo.jpg")                          # JPEG via Pillow
pil = ImageOps.exif_transpose(pil)                     # respect rotation
pil.thumbnail((300, 200), Image.LANCZOS)               # fit, keep ratio
photo = ImageTk.PhotoImage(pil)
label = ttk.Label(root, image=photo); label.pack(padx=10, pady=10)
label.image = photo                                    # keep references!
root.images = [icon, small]
gray = ImageTk.PhotoImage(ImageOps.grayscale(pil))
label.bind("<Enter>", lambda e: label.configure(image=gray))
label.bind("<Leave>", lambda e: label.configure(image=photo)); label.gray = gray
dot = tk.PhotoImage(width=8, height=8)
dot.put("red", to=(0, 0, 8, 8))                        # solid red square
ttk.Label(root, image=dot, text=" Online", compound="left").pack()
root.dot = dot
root.mainloop()
```

## CODE EXPLAINED

- → `subsample(2)` keeps every 2nd pixel: half size
- → `exif_transpose` fixes sideways phone photos
- → `thumbnail` resizes in place, preserving aspect ratio
- → Attaching images to widgets keeps them alive
- → `put(color, to=box)` fills pixels on a blank image

## PHOTOIMAGE

| MEMBER | PURPOSE |
|---|---|
| `file= / data=` | Load from file / bytes |
| `zoom(n)` | Enlarge ×n |
| `subsample(n)` | Shrink ÷n |
| `put(c, to=)` | Set pixels |
| `width(), height()` | Size |

## FORMAT SUPPORT

| FORMAT | HOW |
|---|---|
| `PNG, GIF` | PhotoImage |
| `PPM / PGM` | PhotoImage |
| `SVG` | Tk 9.0 PhotoImage |
| `JPEG, WebP` | Pillow ImageTk |
| `ICO (window)` | iconbitmap() |

## PILLOW OPS

| CALL | DOES |
|---|---|
| `Image.open(p)` | Load |
| `thumbnail(size)` | Fit within box |
| `resize(size)` | Exact size |
| `rotate(deg)` | Rotate |
| `ImageTk.PhotoImage` | Tk-ready image |

## ⚠ COMMON MISTAKES

- Images vanishing because only a local variable held them
- Loading JPEG with `tk.PhotoImage` (unsupported in Tk 8.6)
- Creating images before `Tk()` exists

## ✓ TIPS & BEST PRACTICES

- Cache resized images instead of resizing on every redraw
- Store icons in a dict on the app object
- Use `compound` for toolbar buttons with labels

## QUICK SUMMARY

- PhotoImage: PNG/GIF natively
- Pillow for JPEG and resizing
- `zoom` / `subsample` scale by integers
- Always keep a reference

# after() & Timers



## SCHEDULING WORK ON THE EVENT LOOP

`after(ms, func, *args)` asks Tk to call `func` once, at least `ms` milliseconds later, from the event loop. It returns an **id** you can pass to `after_cancel()`. Repeating timers are made by having the function **reschedule itself**. `after_idle(func)` runs once the loop has no pending events. `update_idletasks()` processes pending redraws and geometry work immediately, which is safe; `update()` processes **all** events, including clicks, and can re-enter your code unexpectedly.

## NOTES

- Timing is a minimum, not exact; long callbacks delay timers
- `after(0, f)` runs `f` as soon as the loop is free
- Cancel pending timers before destroying widgets they touch
- `after` without a function sleeps but blocks; avoid it
- Each `after` callback must finish quickly

## CLOCK, COUNTDOWN AND DEBOUNCE

```
import tkinter as tk, time

root = tk.Tk()
clock = tk.Label(root, font=("Consolas", 24)); clock.pack(padx=20)
def tick():
    clock.config(text=time.strftime("%H:%M:%S"))
    root.after(1000 - int(time.time() * 1000) % 1000, tick)  # align to second
tick()
count = tk.IntVar(value=10)
tk.Label(root, textvariable=count).pack()
def countdown():
    if count.get() > 0:
        count.set(count.get() - 1)
        root.countdown_id = root.after(1000, countdown)
countdown()
tk.Button(root, text="Stop",
          command=lambda: root.after_cancel(root.countdown_id)).pack()
search = tk.Entry(root); search.pack(pady=6)
pending = None
def on_key(event):                      # debounce: run 300 ms after typing stops
    global pending
    if pending: root.after_cancel(pending)
    pending = root.after(300, lambda: print("search", search.get()))
search.bind("<KeyRelease>", on_key)
root.after_idle(lambda: print("UI ready"))
```

## CODE EXPLAINED

- → The clock reschedules itself, aligned to whole seconds
- → The countdown stores its id so Stop can cancel it
- → Debounce cancels the previous timer on every keystroke
- → Only the last pending search runs after typing pauses
- → `after_idle` runs once startup events are processed

## TIMER METHODS

| METHOD | DOES |
|---|---|
| `after(ms, f, *a)` | Run later; returns id |
| `after_idle(f)` | Run when idle |
| `after_cancel(id)` | Cancel pending |
| `after_info()` | Pending ids (3.13+) |

## UPDATE METHODS

| METHOD | PROCESSES |
|---|---|
| `update_idletasks()` | Redraws, geometry |
| `update()` | All events (risky) |
| `mainloop()` | Everything, forever |
| `wait_variable(v)` | Until v changes |

## PATTERNS

| PATTERN | SHAPE |
|---|---|
| Repeat | f reschedules f |
| Countdown | Stop at 0 |
| Debounce | Cancel + reschedule |
| Poll | Check queue, reschedule |
| Chunked work | Do a slice, reschedule |

## ⚠ COMMON MISTAKES

- `time.sleep()` in callbacks, freezing the window
- Calling `update()` inside callbacks and re-entering handlers
- Leaving timers running after their widget is destroyed

## ✓ TIPS & BEST PRACTICES

- Split long loops into chunks scheduled with `after(0, …)`
- Debounce expensive work triggered by typing or resizing
- Keep timer ids on `self` for clean cancellation

## QUICK SUMMARY

- `after(ms, f)` schedules; returns an id
- Self-rescheduling = repeating timer
- `after_cancel` stops it
- Prefer `update_idletasks` over `update`

# Threading & Responsive UIs



## KEEPING THE UI RESPONSIVE

Tk runs everything on **one thread**: while a callback works, the window can't redraw or respond. Move slow work (network, files, heavy math) to a **worker thread**, but remember that Tkinter is **not thread-safe**: workers must not touch widgets or Tk variables. Instead they put results on a `queue.Queue`, and the main thread **polls** the queue with `after()` and updates the UI. `concurrent.futures` works the same way with a future you check from an `after()` loop.

## NOTES

- Mark workers `daemon=True` so they don't block exit
- Pass data, not widgets, into worker functions
- Disable the start button while work runs
- Catch exceptions in the worker and queue them
- CPU-bound work may need `multiprocessing` instead

## WORKER THREAD + QUEUE + AFTER()

```
import tkinter as tk, threading, queue, time
from tkinter import ttk

root = tk.Tk(); q = queue.Queue()
bar = ttk.Progressbar(root, length=260); bar.pack(padx=10, pady=8)
status = ttk.Label(root, text="Idle"); status.pack()
def worker(n):                          # runs off the main thread
    try:
        for i in range(1, n + 1):
            time.sleep(0.05)            # slow work stand-in
            q.put(("progress", i * 100 / n))
        q.put(("done", f"Processed {n} items"))
    except Exception as exc:
        q.put(("error", str(exc)))
def poll():                             # runs on the main thread
    while not q.empty():
        kind, value = q.get_nowait()
        if kind == "progress":
            bar["value"] = value
        else:
            status.config(text=value); start.state(["!disabled"]); return
    root.after(50, poll)
def begin():
    start.state(["disabled"]); status.config(text="Working…")
    threading.Thread(target=worker, args=(60,), daemon=True).start(); poll()
start = ttk.Button(root, text="Start", command=begin); start.pack(pady=8)
```

## CODE EXPLAINED

→ The worker only writes to the queue, never to widgets

→ Errors are sent through the queue too

→ `poll()` drains all queued messages each tick

→ Polling stops after the final message

→ The button is disabled while work runs

## THREAD RULES

| DO | DON'T |
|---|---|
| `Queue results` | Touch widgets in worker |
| `Poll with after()` | Block on q.get() |
| `daemon=True` | Join in a callback |
| `Copy data in` | Share mutable state |

## TOOLS

| MODULE | USE |
|---|---|
| `threading` | I/O-bound work |
| `queue.Queue` | Thread-safe channel |
| `concurrent.futures` | Pools, futures |
| `multiprocessing` | CPU-bound work |
| `asyncio` | Separate loop thread |

## FUTURES PATTERN

| STEP | CODE |
|---|---|
| `Submit` | fut = pool.submit(f) |
| `Check` | fut.done() |
| `Result` | fut.result() |
| `Reschedule` | after(100, check) |

## ⚠ COMMON MISTAKES

- Calling `label.config()` from a worker thread
- Using `q.get()` (blocking) on the main thread
- Starting a new poll loop on every click

## ✓ TIPS & BEST PRACTICES

- Wrap the pattern in a small helper class
- Send tagged tuples like `("progress", 40)`
- Show progress or a spinner for anything over ~0.2 s

## QUICK SUMMARY

- One UI thread; never block it
- Workers → queue → `after()` poll
- Never touch Tk from workers
- Disable controls while busy

# Input Validation

validate="key" · validatecommand · register() · %P · %S · invalidcommand · focusout validation

## VALIDATING ENTRY INPUT

Entry and Spinbox can check input **before** it's accepted. Set `validate=` to when validation runs (`"key"` for every keystroke, `"focusout"`, `"all"`…) and `validatecommand=` to a callback registered with `register()` plus **percent substitutions** that choose what Tk passes in, such as `%P` (the proposed new text). The callback returns `True` to allow the change or `False` to reject it. `invalidcommand=` runs when validation fails.

## NOTES

- The callback must return a bool, or validation turns off
- Changing the Entry's text inside the callback disables validation
- `%P` includes text not yet inserted
- Allow the empty string so users can clear the field
- Validate the whole form again before saving

## NUMERIC AND LENGTH VALIDATION

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()

def is_number(proposed):
    if proposed in ("", "-", ".", "-."):
        return True                          # allow partial input
    try:
        float(proposed); return True
    except ValueError:
        return False

def max_len(proposed, limit):
    return len(proposed) <= int(limit)

def on_invalid(widget_name):
    root.nametowidget(widget_name).bell()    # beep on rejection
num_cmd = (root.register(is_number), "%P")
len_cmd = (root.register(max_len), "%P", 5)
bad_cmd = (root.register(on_invalid), "%W")
ttk.Entry(root, validate="key", validatecommand=num_cmd,
          invalidcommand=bad_cmd).pack(padx=8, pady=4)      # price
ttk.Entry(root, validate="key", validatecommand=len_cmd,
          width=8).pack(padx=8, pady=4)                     # ZIP code
```

## CODE EXPLAINED

- → `register()` wraps a Python function as a Tcl command
- → The tuple adds substitutions ( `%P` ) and fixed arguments (5)
- → Partial values like `"-"` must be allowed while typing
- → `%W` gives the widget's name for the invalid handler
- → `nametowidget` turns that name back into the widget

## VALIDATE=

| VALUE | RUNS ON |
|---|---|
| `none` | Never (default) |
| `key` | Each edit |
| `focus` | Focus in or out |
| `focusout` | Leaving the field |
| `all` | All of the above |

## SUBSTITUTIONS

| CODE | PASSES |
|---|---|
| `%P` | Text if allowed |
| `%s` | Text before edit |
| `%S` | Inserted/deleted text |
| `%d` | 1 insert, 0 delete |
| `%W` | Widget name |

## MORE CODES

| CODE | PASSES |
|---|---|
| `%i` | Index of edit |
| `%v` | validate setting |
| `%V` | Trigger type |

## ⚠ COMMON MISTAKES

- Passing the plain function instead of the `register()` ed name
- Rejecting `""` , so users can't delete the last character
- Returning `None` (no explicit `return True` )

## ✓ TIPS & BEST PRACTICES

- Use `key` for format, `focusout` for completeness checks
- Show errors next to fields, not just with beeps
- Use ttk's `invalid` state to tint bad fields

## QUICK SUMMARY

- `validate=` when, `validatecommand=` how
- `register()` + `%P` for proposed text
- Return `True` to accept
- Allow partial input while typing

# Scrollable Frames

Canvas + Frame · create_window() · <Configure> · scrollregion · bbox("all") · <MouseWheel> · Button-4/5 on X11

## SCROLLING A PANEL OF WIDGETS

Frames can't scroll, but a Canvas can, and a Canvas can hold a widget with `create_window()`. The standard recipe: put an **inner Frame** inside a Canvas, add widgets to the inner Frame, and whenever its size changes (`<Configure>`), set the canvas `scrollregion` to `bbox("all")`. Also stretch the inner Frame to the canvas width so rows fill horizontally. Mouse-wheel scrolling needs explicit bindings, which differ by platform.

## NOTES

- Bind the wheel only while the pointer is over the panel
- Windows: `delta` ±120; macOS: small deltas; X11: Button-4/5
- Tk 9.0 unifies X11 wheel events as `<MouseWheel>`
- Thousands of rows? Use a Treeview or a virtualized list
- Wrap the recipe in a reusable class

## REUSABLE SCROLLABLEFRAME

```
import tkinter as tk
from tkinter import ttk

class ScrollableFrame(ttk.Frame):
    def __init__(self, parent, **kw):
        super().__init__(parent, **kw)
        self.canvas = tk.Canvas(self, highlightthickness=0)
        sb = ttk.Scrollbar(self, orient="vertical", command=self.canvas.yview)
        self.canvas.configure(yscrollcommand=sb.set)
        self.canvas.pack(side="left", fill="both", expand=True)
        sb.pack(side="right", fill="y")
        self.inner = ttk.Frame(self.canvas)
        win = self.canvas.create_window((0, 0), window=self.inner, anchor="nw")
        self.inner.bind("<Configure>", lambda e: self.canvas.configure(
            scrollregion=self.canvas.bbox("all")))
        self.canvas.bind("<Configure>",
            lambda e: self.canvas.itemconfigure(win, width=e.width))
        self.bind("<Enter>", lambda e: self.bind_all("<MouseWheel>", self._wheel))
        self.bind("<Leave>", lambda e: self.unbind_all("<MouseWheel>"))
    def _wheel(self, e):                          # Windows/macOS
        self.canvas.yview_scroll(-1 if e.delta > 0 else 1, "units")
root = tk.Tk(); root.geometry("300x250")
panel = ScrollableFrame(root); panel.pack(fill="both", expand=True)
for i in range(40):
    ttk.Checkbutton(panel.inner, text=f"Option {i + 1}").pack(anchor="w")
```

## CODE EXPLAINED

→ The inner Frame lives in the canvas via `create_window`

→ Inner `<Configure>` keeps `scrollregion` in sync

→ Canvas `<Configure>` stretches the inner Frame's width

→ Wheel is bound globally only while hovering

→ Callers add widgets to `panel.inner`

## RECIPE STEPS

| STEP | CODE |
|---|---|
| Canvas + bar | yscrollcommand wiring |
| Inner frame | create_window(…, nw) |
| Track size | inner <Configure> |
| Match width | itemconfigure(width=) |
| Wheel | bind_all on <Enter> |

## WHEEL BY PLATFORM

| PLATFORM | EVENT |
|---|---|
| Windows | <MouseWheel>, ±120 |
| macOS | <MouseWheel>, ±1–n |
| X11 (Tk 8.6) | <Button-4/5> |
| Tk 9.0 | <MouseWheel> everywhere |
| Horizontal | <Shift-MouseWheel> |

## SCROLL CALLS

| CALL | DOES |
|---|---|
| `yview_scroll(n, "units")` | By lines |
| `yview_scroll(n, "pages")` | By pages |
| `yview_moveto(0)` | Jump to top |
| `bbox("all")` | Content extent |

## ⚠ COMMON MISTAKES

- Forgetting to update `scrollregion` as content grows
- Binding the wheel with `bind_all` permanently
- Using `pack(fill="x")` in the inner frame without matching width

## ✓ TIPS & BEST PRACTICES

- Expose `inner` as the parent for child widgets
- Call `yview_moveto(0)` after rebuilding content
- Add X11 Button-4/5 bindings for Linux users on Tk 8.6

## QUICK SUMMARY

- Canvas + inner Frame + `create_window`
- `scrollregion = bbox("all")` on resize
- Stretch inner width to canvas width
- Wheel bindings per platform

# Application Architecture

App class · page frames · tkraise() · controller · MVC separation · shared state

## STRUCTURING LARGER APPS

As apps grow, split them into **pages** (Frame subclasses), a **controller** (usually the `tk.Tk` subclass) that owns shared state and navigation, and a **model** layer with no Tk imports. A common navigation trick stacks all pages in the same grid cell and brings one forward with `tkraise()`. Pages talk to the controller, not to each other, and the model can be unit-tested without opening a window. This mirrors MVC without needing a framework.

## NOTES

- Pass the controller into each page's constructor
- Keep file, network and math code in the model
- Use virtual events ( `<<Saved>>` ) to notify pages
- Build pages lazily if they're expensive
- One module per page keeps diffs small

## CONTROLLER + PAGES + TKRAISE()

```
import tkinter as tk
from tkinter import ttk
class Model:                                  # no Tk imports needed
    user = None
    def login(self, name): self.user = name.strip() or None; return self.user
class LoginPage(ttk.Frame):
    def __init__(self, parent, app):
        super().__init__(parent, padding=20); name = tk.StringVar()
        ttk.Entry(self, textvariable=name).pack(pady=6)
        ttk.Button(self, text="Log in", command=lambda: app.login(name.get())).pack()
class HomePage(ttk.Frame):
    def __init__(self, parent, app):
        super().__init__(parent, padding=20)
        self.msg = ttk.Label(self); self.msg.pack(pady=6)
        ttk.Button(self, text="Log out", command=lambda: app.show("LoginPage")).pack()
    def on_show(self, app): self.msg.config(text=f"Welcome, {app.model.user}")
class App(tk.Tk):
    def __init__(self):
        super().__init__(); self.model = Model()
        self.pages = {P.__name__: P(self, self) for P in (LoginPage, HomePage)}
        for p in self.pages.values(): p.grid(row=0, column=0, sticky="nsew")
        self.show("LoginPage")
    def show(self, name):
        page = self.pages[name]
        if hasattr(page, "on_show"): page.on_show(self)
        page.tkraise()
    def login(self, name):
        if self.model.login(name): self.show("HomePage")
```

## CODE EXPLAINED

- → `Model` holds state and rules, testable without a GUI
- → Pages receive the controller ( `app` ) to request actions
- → All pages share one grid cell; `tkraise()` picks the visible one
- → `on_show` hooks refresh a page when it appears
- → Navigation lives only in the controller

## ROLES

| PART | OWNS |
|---|---|
| `Model` | Data, rules, I/O |
| `Controller (App)` | Navigation, shared state |
| `Pages (Frames)` | Widgets for one screen |
| `Dialogs` | Short-lived input |

## NAVIGATION OPTIONS

| APPROACH | FITS |
|---|---|
| `tkraise() stack` | Wizard / few pages |
| `Notebook` | User picks freely |
| `Destroy + rebuild` | Rare, heavy pages |
| `Toplevels` | Independent windows |

## PROJECT LAYOUT

| FILE | CONTAINS |
|---|---|
| `app.py` | App + main() |
| `model.py` | Data classes, logic |
| `pages/` | One page per module |
| `tests/` | Model unit tests |

## ⚠ COMMON MISTAKES

- Pages reaching into each other's widgets
- Database or HTTP calls inside button callbacks
- Building every page with global variables

## ✓ TIPS & BEST PRACTICES

- Unit-test the model with pytest; no window needed
- Give pages an `on_show` hook for refreshes
- Prefer composition: small Frame components in pages

## QUICK SUMMARY

- Model / controller / pages
- Stacked frames + `tkraise()`
- Pages talk only to the controller
- Logic stays testable without Tk

# DPI & Cross-Platform Differences

tk scaling · SetProcessDpiAwareness · windowingsystem · aqua vs win32 vs x11 · Button-2/3 swap · native look

## DPI SCALING AND PLATFORM DIFFERENCES

Tkinter apps run on Windows, macOS and Linux, but each platform differs. On **Windows**, apps are blurry on high-DPI screens unless the process declares DPI awareness through `ctypes` **before** creating `Tk()`. Tk's `tk scaling` sets pixels per point, so point-sized fonts scale while pixel sizes don't. `tk windowingsystem` (`win32`, `aqua`, `x11`) lets you branch on mouse buttons, shortcuts, menus and theme choices.

## NOTES

- Specify fonts in points (positive sizes) so they scale
- macOS swaps mouse buttons 2 and 3 compared to others
- Use `<Command-…>` for shortcuts on macOS
- Wayland desktops run Tk through XWayland
- Test on each target OS before release

## CROSS-PLATFORM SETUP

```
import sys, tkinter as tk

if sys.platform == "win32":                      # before Tk() is created
    import ctypes
    try:
        ctypes.windll.shcore.SetProcessDpiAwareness(1)   # system DPI aware
    except (AttributeError, OSError):
        ctypes.windll.user32.SetProcessDPIAware()        # Vista/7 fallback

root = tk.Tk()
ws = root.tk.call("tk", "windowingsystem")       # 'win32', 'aqua', 'x11'
print(ws, root.tk.call("tk", "scaling"))         # pixels per point

dpi = root.winfo_fpixels("1i")                   # pixels per inch
print(f"{dpi:.0f} DPI")
if ws == "x11":
    root.tk.call("tk", "scaling", dpi / 72)      # match real DPI

MOD = "Command" if ws == "aqua" else "Control"
RIGHT = "<Button-2>" if ws == "aqua" else "<Button-3>"
root.bind(f"<{MOD}-s>", lambda e: print("save"))
root.bind(RIGHT, lambda e: print("context menu"))

def px(n):                                       # scale hard-coded pixels
    return int(n * dpi / 96)
tk.Frame(root, width=px(200), height=px(100)).pack()
root.mainloop()
```

## CODE EXPLAINED

- → DPI awareness must be set before the first window
- → `windowingsystem` drives platform branches
- → `winfo_fpixels("1i")` measures the real DPI
- → Shortcuts and right-click use platform constants
- → `px()` scales fixed pixel sizes from a 96-DPI baseline

## WINDOWINGSYSTEM

| VALUE | PLATFORM |
|---|---|
| `win32` | Windows |
| `aqua` | macOS |
| `x11` | Linux / BSD (X or XWayland) |

## PLATFORM DIFFERENCES

| TOPIC | NOTES |
|---|---|
| `Right-click` | Button-3; Button-2 on mac |
| `Shortcut key` | Control; Command on mac |
| `Menu bar` | In window; top of screen on mac |
| `Native theme` | vista / aqua / none |
| `Wheel` | <MouseWheel>; Button-4/5 on X11 |

## SCALING TOOLS

| CALL | PURPOSE |
|---|---|
| `tk scaling` | Pixels per point |
| `winfo_fpixels("1i")` | Actual DPI |
| `SetProcessDpiAwareness` | Sharp on Windows |
| `Point font sizes` | Scale automatically |

## ⚠ COMMON MISTAKES

- Setting DPI awareness after `Tk()` (no effect)
- Hard-coding `<Button-3>` for context menus
- Mixing pixel and point sizes so layouts break at 150%

## ✓ TIPS & BEST PRACTICES

- Centralize platform constants in one module
- Prefer layout managers over fixed pixel sizes
- Use `clam` on Linux for a consistent look

## QUICK SUMMARY

- Declare DPI awareness first on Windows
- Branch on `tk windowingsystem`
- Points scale, pixels don't
- Mind mouse buttons and modifier keys

# Testing & Debugging



## TESTING AND DEBUGGING TKINTER APPS

GUI code is easiest to test when logic lives outside it (Sheet 54). For the UI itself, tests can create a real `Tk()` root, call `update_idletasks()` or `update()` to process events, and drive widgets with `invoke()` or `event_generate()`. Exceptions inside callbacks don't crash the app; Tk passes them to `report_callback_exception`, which prints a traceback. Override it to log errors or show a dialog. On headless CI, run tests under **Xvfb**.

## NOTES

- Create one root per test and `destroy()` it in teardown
- `event_generate` needs the widget mapped and focused for keys
- `winfo_children()` helps find widgets in tests
- `root.tk.call(...)` lets you inspect raw Tcl state
- `pytest-xvfb` starts a virtual display automatically

## UNITTEST WITH A REAL ROOT

```
import unittest, tkinter as tk
from tkinter import ttk

class Counter(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent)
        self.value = tk.IntVar(value=0)
        self.button = ttk.Button(self, text="+1", command=lambda:
                                 self.value.set(self.value.get() + 1))
        self.button.pack()

class CounterTest(unittest.TestCase):
    def setUp(self):
        self.root = tk.Tk(); self.root.withdraw()       # no visible window
        self.w = Counter(self.root); self.w.pack()
    def tearDown(self):
        self.root.destroy()
    def test_click_increments(self):
        self.w.button.invoke()
        self.assertEqual(self.w.value.get(), 1)
    def test_errors_are_reported(self):
        seen = []
        self.root.report_callback_exception = lambda *exc: seen.append(exc[1])
        self.root.after(0, lambda: 1 / 0); self.root.update()
        self.assertIsInstance(seen[0], ZeroDivisionError)
```

## CODE EXPLAINED

- → `withdraw()` keeps test windows off-screen
- → `invoke()` simulates a button click
- → `update()` runs the scheduled `after(0, …)` callback
- → `report_callback_exception` receives (type, value, traceback)
- → Each test gets a fresh root, destroyed in `tearDown`

## DRIVING WIDGETS

| CALL | SIMULATES |
|---|---|
| `invoke()` | Button click |
| `event_generate()` | Key / mouse event |
| `insert("end", s)` | Typing text |
| `focus_force()` | Focus for key events |
| `update()` | Process events |

## DEBUGGING TOOLS

| TOOL | SHOWS |
|---|---|
| `winfo_children()` | Widget tree |
| `w.configure()` | All option values |
| `print(event)` | Event details |
| `report_callback_exception` | Callback errors |
| `tk.call()` | Tcl internals |

## CI SETUP

| NEED | SOLUTION |
|---|---|
| `Linux display` | xvfb-run pytest |
| `pytest plugin` | pytest-xvfb |
| `GitHub Actions` | xvfb on ubuntu |
| `Skip headless` | TclError guard |

## ⚠ COMMON MISTAKES

- Running tests without a display on Linux CI
- Sharing one root across tests and leaking state
- Asserting before `update()` processed pending events

## ✓ TIPS & BEST PRACTICES

- Test models directly; test the GUI lightly
- Override `report_callback_exception` to log in production
- Name widgets ( `name="save"` ) to find them with `nametowidget`

## QUICK SUMMARY

- Real root + `update()` in tests
- `invoke()` and `event_generate()` drive UI
- `report_callback_exception` for errors
- Xvfb for headless CI

# Packaging & Distribution



## SHIPPING A TKINTER APP

End users usually don't have Python, so bundle the interpreter, Tcl/Tk and your code into an executable. **PyInstaller** is the most common tool: `--windowed` hides the console window, `--onefile` produces a single file (slower to start), and `--add-data` includes images and other resources. Inside the bundle, locate resources relative to `sys._MEIPASS` or the script file. **Nuitka** compiles to C for speed, and **cx_Freeze** or **Briefcase** suit installers and app stores.

## NOTES

- Build on each target OS; there's no cross-compiling
- `--add-data` uses `src:dest` ; Windows needed `;` before PyInstaller 6
- Antivirus tools sometimes flag `--onefile` builds
- Rename a script to `.pyw` to run without a console on Windows
- Sign and notarize macOS apps for distribution

## BUNDLING WITH PYINSTALLER

```
# Install and build
pip install pyinstaller
pyinstaller --windowed --onefile --name Inventory \
    --icon assets/app.ico --add-data "assets:assets" app.py
# Output: dist/Inventory(.exe)  |  build/ = scratch  |  Inventory.spec

# One-folder build (faster startup, easier debugging)
pyinstaller --windowed --name Inventory --add-data "assets:assets" app.py

# Rebuild from the generated spec after editing it
pyinstaller Inventory.spec

# Resource helper inside app.py (works frozen or not):
#   import sys
#   from pathlib import Path
#   BASE = Path(getattr(sys, "_MEIPASS", Path(__file__).parent))
#   icon = tk.PhotoImage(file=BASE / "assets" / "save.png")

# Alternatives
python -m nuitka --standalone --enable-plugin=tk-inter \
    --windows-console-mode=disable app.py
cxfreeze build                        # cx_Freeze with pyproject config
briefcase create && briefcase build   # BeeWare installers
```

## CODE EXPLAINED

- → `--windowed` = no console window behind the GUI
- → `--add-data "src:dest"` copies resources into the bundle
- → The spec file records options for repeatable builds
- → `sys._MEIPASS` is the unpack folder in frozen apps
- → Nuitka needs the `tk-inter` plugin for Tkinter

## PYINSTALLER FLAGS

| FLAG | EFFECT |
|---|---|
| `--windowed` | No console |
| `--onefile` | Single executable |
| `--name` | Output name |
| `--icon` | App icon |
| `--add-data` | Bundle resources |

## TOOLS COMPARED

| TOOL | STRENGTH |
|---|---|
| PyInstaller | Popular, simple |
| Nuitka | Compiled, faster |
| cx_Freeze | MSI / installers |
| Briefcase | Native installers |
| py2app | macOS .app |

## CHECKLIST

| ITEM | WHY |
|---|---|
| Relative resources | Paths work frozen |
| Test clean machine | Missing files show |
| Version info | Windows properties |
| Code signing | Fewer warnings |
| One-folder first | Easier debugging |

## ⚠ COMMON MISTAKES

- Loading files by relative path from the current directory
- Shipping a console window with a GUI app
- Only testing the build on the developer's machine

## ✓ TIPS & BEST PRACTICES

- Keep all assets in one folder and add it once
- Commit the `.spec` file for reproducible builds
- Debug with a console build before switching to `--windowed`

## QUICK SUMMARY

- PyInstaller `--windowed --onefile`
- `--add-data` for assets
- Resolve paths via `sys._MEIPASS`
- Build on every target OS

# Modern Themes — sv-ttk & ttkbootstrap



## MODERN LOOKS WITH THEME PACKAGES

Two popular packages give ttk apps a modern look with almost no code changes. **sv-ttk** (Sun Valley) applies a Windows 11-style light or dark theme with one call, plus extra styles like `Accent.TButton` and `Switch.TCheckbutton`. **ttkbootstrap** brings more than 20 Bootstrap-inspired themes, a `bootstyle=` keyword for color roles (`success`, `danger-outline`…) and extra widgets such as `Meter`, `DateEntry`, `Floodgauge` and `Tableview`. Both are on PyPI: `pip install sv-ttk ttkbootstrap`.

## NOTES

- Both work with regular ttk widgets and layouts
- Pick one theme package per app; don't mix them
- ttkbootstrap's `Window` replaces `tk.Tk()`
- ttkbootstrap can also be used as `import ttkbootstrap as ttk`
- Classic `tk` widgets aren't themed by either

## SV-TTK AND TTKBOOTSTRAP

```
# Two separate examples: pick ONE theme package per app
# --- sv-ttk: one call on a normal Tk app ---------------------------
import tkinter as tk
from tkinter import ttk
import sv_ttk

root = tk.Tk()
ttk.Button(root, text="Save", style="Accent.TButton").pack(pady=6)
ttk.Checkbutton(root, text="Dark mode", style="Switch.TCheckbutton",
                command=sv_ttk.toggle_theme).pack(pady=6)
sv_ttk.set_theme("dark")                # or "light"; get_theme()

# --- ttkbootstrap: its own Window and bootstyle keyword ------------
import ttkbootstrap as tb
from ttkbootstrap.constants import SUCCESS, DANGER
app = tb.Window(themename="darkly", title="Dashboard")
tb.Button(app, text="Deploy", bootstyle=SUCCESS).pack(side="left", padx=8)
tb.Button(app, text="Delete", bootstyle=(DANGER, "outline")).pack(side="left")
tb.Meter(app, amountused=72, subtext="CPU", bootstyle="info",
         interactive=True).pack(pady=10)
tb.DateEntry(app, dateformat="%Y-%m-%d").pack()
tb.Progressbar(app, value=40, bootstyle="striped").pack(fill="x", padx=8)
app.style.theme_use("flatly")           # switch themes at runtime
print(app.style.theme_names())          # all ttkbootstrap themes
app.mainloop()
```

## CODE EXPLAINED

- → sv-ttk restyles every ttk widget on the root
- → `Accent.TButton` and `Switch.TCheckbutton` are sv-ttk styles
- → `tb.Window` creates the root with a theme
- → `bootstyle` sets the color role and variant
- → `Meter` and `DateEntry` are ttkbootstrap-only widgets

## SV-TTK API

| CALL | DOES |
|---|---|
| `set_theme("dark")` | Apply theme |
| `toggle_theme()` | Light ↔ dark |
| `get_theme()` | Current theme |
| `Accent.TButton` | Highlighted button |
| `Switch.TCheckbutton` | Toggle switch |

## BOOTSTYLE VALUES

| VALUE | LOOK |
|---|---|
| `primary / secondary` | Main colors |
| `success / info` | Green / blue |
| `warning / danger` | Orange / red |
| `"danger-outline"` | Outlined variant |
| `"success-striped"` | Striped bar |

## TTKBOOTSTRAP EXTRAS

| WIDGET | PURPOSE |
|---|---|
| `Meter` | Radial gauge |
| `DateEntry` | Date picker field |
| `Floodgauge` | Filled progress |
| `Tableview` | Sortable table |
| `ToastNotification` | Popup toast |

## ⚠ COMMON MISTAKES

- Calling `sv_ttk.set_theme()` before creating `Tk()`
- Creating both `tk.Tk()` and `tb.Window()`
- Expecting classic `tk.Button` to pick up the theme

## ✓ TIPS & BEST PRACTICES

- Use sv-ttk for a native Windows 11 look
- Use ttkbootstrap for dashboards and colorful UIs
- Respect the OS dark mode setting with a toggle

## QUICK SUMMARY

- sv-ttk: `set_theme("dark")`
- ttkbootstrap: `Window(themename=…)`
- `bootstyle=` for color roles
- Extra widgets: Meter, DateEntry, Tableview

# CustomTkinter



## CUSTOMTKINTER WIDGETS

**CustomTkinter** (`pip install customtkinter`) is a widget library built on Tkinter that draws its own rounded, modern widgets with automatic **light/dark mode** and **HiDPI scaling**. Its widgets are prefixed `CTk` (`CTkButton`, `CTkEntry`, `CTkSwitch`…) and use their own options such as `corner_radius`, `fg_color` and `hover_color`. Layout still uses `pack`, `grid` and `place`, and standard Tk variables and `command=` callbacks work as usual.

## NOTES

- Use `ctk.CTk()` as the root instead of `tk.Tk()`
- Colors can be a pair: `("light color", "dark color")`
- `CTkImage` supports separate light/dark images and scaling
- Mixing ttk widgets in looks inconsistent
- Use `CTkFont` for fonts that scale with the UI

## SETTINGS PANEL IN CUSTOMTKINTER

```
import customtkinter as ctk

ctk.set_appearance_mode("system")          # "light", "dark", "system"
ctk.set_default_color_theme("blue")        # "green", "dark-blue", or a .json

app = ctk.CTk()
app.title("Settings"); app.geometry("360x300")
app.grid_columnconfigure(0, weight=1)

ctk.CTkLabel(app, text="Profile", font=ctk.CTkFont(size=18, weight="bold")
             ).grid(row=0, column=0, padx=20, pady=(20, 6), sticky="w")
name = ctk.CTkEntry(app, placeholder_text="Display name")
name.grid(row=1, column=0, padx=20, pady=6, sticky="ew")
dark = ctk.StringVar(value="off")
ctk.CTkSwitch(app, text="Dark mode", variable=dark, onvalue="on",
              offvalue="off", command=lambda: ctk.set_appearance_mode(
                  "dark" if dark.get() == "on" else "light")
              ).grid(row=2, column=0, padx=20, pady=6, sticky="w")
size = ctk.CTkSegmentedButton(app, values=["S", "M", "L"])
size.set("M"); size.grid(row=3, column=0, padx=20, pady=6)
ctk.CTkButton(app, text="Save", corner_radius=8,
              fg_color=("#2e7d32", "#43a047"), hover_color="#1b5e20",
              command=lambda: print(name.get(), size.get())
              ).grid(row=4, column=0, padx=20, pady=16, sticky="ew")
app.mainloop()
```

## CODE EXPLAINED

- → Appearance and color theme are set globally first
- → `CTkEntry` supports placeholder text
- → `CTkSwitch` toggles the appearance mode live
- → `CTkSegmentedButton` is a modern radio group
- → `fg_color` pair = (light-mode, dark-mode) colors

## CORE WIDGETS

| CTK CLASS | LIKE |
|---|---|
| `CTkButton` | Button |
| `CTkEntry` | Entry |
| `CTkTextbox` | Text |
| `CTkOptionMenu` | OptionMenu |
| `CTkScrollableFrame` | Scrolling Frame |

## STYLE OPTIONS

| OPTION | EFFECT |
|---|---|
| `corner_radius` | Rounded corners |
| `fg_color` | Main color |
| `hover_color` | Hover color |
| `border_width` | Outline width |
| `text_color` | Text color |

## GLOBAL SETTINGS

| CALL | EFFECT |
|---|---|
| `set_appearance_mode()` | Light / dark / system |
| `set_default_color_theme()` | Accent theme |
| `set_widget_scaling(f)` | Widget size |
| `set_window_scaling(f)` | Window size |

## ⚠ COMMON MISTAKES

- Using `bg=` / `fg=` instead of `fg_color` / `text_color`
- Creating `tk.Tk()` and placing CTk widgets in it
- Setting the color theme after widgets exist

## ✓ TIPS & BEST PRACTICES

- Default to `"system"` appearance to follow the OS
- Use `CTkTabview` and `CTkScrollableFrame` for layout
- Keep logic separate so you can swap toolkits later

## QUICK SUMMARY

- Modern drawn widgets prefixed `CTk`
- Automatic light/dark mode
- `fg_color`, `corner_radius`, `hover_color`
- Same layout managers and callbacks

# Embedding Matplotlib & Add-ons



## CHARTS AND POPULAR ADD-ONS

**Matplotlib** plots embed in Tkinter through `FigureCanvasTkAgg`, which turns a `Figure` into a Tk widget, plus an optional `NavigationToolbar2Tk` for zoom and pan. Create the figure with `matplotlib.figure.Figure` rather than `pyplot`, so no extra window or global state is created. Other widely used add-ons fill gaps in Tk: **tkcalendar** (date pickers), **tkinterdnd2** (drag files in from the OS) and **Pillow** (image formats, Sheet 49).

## NOTES

- Call `canvas.draw()` (or `draw_idle()`) after changing plots
- `pack_toolbar=False` lets you grid the toolbar yourself
- `mpl_connect` handles clicks and hovers on the plot
- tkinterdnd2 needs its own root: `TkinterDnD.Tk()`
- `get_date()` from tkcalendar returns a `datetime.date`

## LIVE PLOT, DATE PICKER, FILE DROP

```
import tkinter as tk
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from tkcalendar import DateEntry
from tkinterdnd2 import TkinterDnD, DND_FILES

root = TkinterDnD.Tk()                    # Tk root with drag-and-drop
fig = Figure(figsize=(5, 3), dpi=100)
ax = fig.add_subplot()
line, = ax.plot([1, 2, 3, 4], [3, 1, 4, 2], marker="o")
canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack(fill="both", expand=True)
NavigationToolbar2Tk(canvas, root).update()

def add_point():
    xs, ys = list(line.get_xdata()), list(line.get_ydata())
    line.set_data(xs + [xs[-1] + 1], ys + [ys[-1] + 1])
    ax.relim(); ax.autoscale_view(); canvas.draw_idle()
tk.Button(root, text="Add point", command=add_point).pack()
when = DateEntry(root, date_pattern="yyyy-mm-dd"); when.pack(pady=4)
when.bind("<<DateEntrySelected>>", lambda e: print(when.get_date()))
drop = tk.Label(root, text="Drop files here", relief="groove", height=3)
drop.pack(fill="x", padx=8, pady=8)
drop.drop_target_register(DND_FILES)
drop.dnd_bind("<<Drop>>", lambda e: print(root.tk.splitlist(e.data)))
root.mainloop()
```

## CODE EXPLAINED

- → `Figure` + `FigureCanvasTkAgg` = plot as a Tk widget
- → The toolbar adds zoom, pan and save buttons
- → `relim` / `autoscale_view` refit axes after new data
- → `DateEntry` is a drop-down calendar field
- → `splitlist` splits dropped paths safely (spaces in braces)

## MATPLOTLIB EMBED

| CALL | DOES |
|---|---|
| `Figure(figsize, dpi)` | Plot container |
| `FigureCanvasTkAgg` | Figure → widget |
| `get_tk_widget()` | Widget to pack |
| `draw_idle()` | Redraw soon |
| `mpl_connect(ev, f)` | Plot events |

## ADD-ON PACKAGES

| PACKAGE | PROVIDES |
|---|---|
| `tkcalendar` | Calendar, DateEntry |
| `tkinterdnd2` | OS drag-and-drop |
| `Pillow` | Image formats |
| `pystray` | System tray icon |
| `tktooltip` | Hover tooltips |

## DRAG & DROP

| CALL | PURPOSE |
|---|---|
| `TkinterDnD.Tk()` | DnD-capable root |
| `drop_target_register` | Accept drops |
| `dnd_bind("<<Drop>>")` | Handle drop |
| `e.data` | Dropped paths |

## ⚠ COMMON MISTAKES

- Using `plt.figure()` and getting a second window
- Changing data without calling `draw()` / `draw_idle()`
- Splitting dropped paths on spaces

## ✓ TIPS & BEST PRACTICES

- Reuse one Figure and update data instead of recreating
- Use `draw_idle` for frequent updates
- Check each add-on's license before shipping

## QUICK SUMMARY

- `FigureCanvasTkAgg` embeds Matplotlib
- Use `Figure`, not `pyplot`
- tkcalendar for dates, tkinterdnd2 for drops
- Pillow for images

# Quick Reference — Widgets & Options



## WIDGETS AT A GLANCE

This sheet collects the widget set on one page: every common widget, whether it has a **ttk** version, the options you'll set most often, and the three geometry managers. Prefer the **ttk** widget whenever one exists; it looks native and is styled through `ttk.Style`. Classic `tk` widgets remain the right choice for **Text**, **Canvas**, **Listbox** and **Menu**, which have no ttk equivalent. Sheet numbers in the tables point to the detailed pages.

## NOTES

- Every widget's first argument is its parent
- Create, then place: `w = W(parent); w.grid(…)`
- `configure()` changes options; `cget()` reads them
- Constants: `"end"`, `"both"`, `"left"` as strings
- Detailed pages: widgets 07–15, ttk 33–41

## WIDGET CREATION CHEAT CODES

```
import tkinter as tk
from tkinter import ttk
root = tk.Tk()

ttk.Label(root, text="Hi", image=img, compound="left")   # img = PhotoImage
ttk.Button(root, text="OK", command=fn, style="Accent.TButton")
ttk.Entry(root, textvariable=sv, show="*", width=20)
ttk.Checkbutton(root, text="On", variable=bv)
ttk.Radiobutton(root, text="A", value="a", variable=sv)
ttk.Combobox(root, values=["a", "b"], state="readonly")
ttk.Spinbox(root, from_=0, to=10, increment=1)
ttk.Scale(root, from_=0, to=100, orient="horizontal", variable=dv)
ttk.Progressbar(root, mode="determinate", maximum=100, value=40)
ttk.Treeview(root, columns=("a", "b"), show="headings")
ttk.Notebook(root); ttk.Separator(root); ttk.Sizegrip(root)
ttk.Frame(root, padding=8); ttk.LabelFrame(root, text="Group")
ttk.Panedwindow(root, orient="horizontal")
ttk.Scrollbar(root, orient="vertical", command=widget.yview)

tk.Text(root, wrap="word", undo=True, height=10)
tk.Canvas(root, width=300, height=200, bg="white")
tk.Listbox(root, selectmode="extended", height=6)
tk.Menu(root, tearoff=0); tk.Toplevel(root)
tk.PhotoImage(file="icon.png")
```

## CODE EXPLAINED

- → One line per widget with its most-used options
- → `sv`, `bv`, `dv` stand for String/Boolean/Double vars
- → `fn` is any zero-argument callback
- → ttk widgets first, then classic-only widgets
- → Placeholders ( `img`, `widget` ) must exist in real code

## CLASSIC-ONLY

| WIDGET | SHEET |
|---|---|
| `Text` | 10–11 |
| `Canvas` | 42–46 |
| `Listbox` | 13 |
| `Menu` | 27–28 |
| `Toplevel` | 05 |

## GEOMETRY MANAGERS

| MANAGER | KEY OPTIONS |
|---|---|
| `pack` | side, fill, expand |
| `grid` | row, column, sticky |
| `grid weights` | columnconfigure |
| `place` | relx, rely, anchor |
| `All` | padx, pady |

## UNIVERSAL OPTIONS

| OPTION | MEANING |
|---|---|
| `text / textvariable` | Displayed text |
| `command` | Click callback |
| `state` | normal / disabled |
| `width` | Chars or px |
| `style (ttk)` | Named style |

## ⚠ COMMON MISTAKES

- Chaining `.grid()` onto the constructor and keeping `None`
- Using classic widgets where a ttk one exists
- Mixing pack and grid in one parent

## ✓ TIPS & BEST PRACTICES

- Keep this sheet next to the detailed pages
- Start every app with `import tkinter as tk` + `ttk`
- Reach for Treeview before building tables by hand

## QUICK SUMMARY

- ttk first; tk for Text, Canvas, Listbox, Menu
- pack / grid / place
- Variables link data to widgets
- Options via `configure()`

# Quick Reference — Events & Methods



## EVENTS, METHODS AND HELPERS

The second quick reference covers what happens **after** widgets exist: binding events, reading the event object, scheduling with `after()`, querying geometry with `winfo_*`, standard dialogs and control variables. These calls make up most of the glue code in a Tkinter app. Keep callbacks short, move slow work off the main thread (Sheet 51), and remember that Tk is single-threaded: only the main thread may touch widgets.

## NOTES

- Handlers from `bind()` take one `event` argument
- `command=` callbacks take none
- `after()` returns an id for `after_cancel()`
- Dialogs return `None` / `""` when cancelled
- Detailed pages: events 21–26, dialogs 29–32, timers 50

## GLUE CODE CHEAT CODES

```
w.bind("<Button-1>", lambda e: print(e.x, e.y))     # widget binding
root.bind_all("<Control-s>", save)                  # app-wide shortcut
w.event_generate("<<Refresh>>")                     # fire virtual event

var = tk.StringVar(value="x"); var.trace_add("write", lambda *a: ...)
job = root.after(500, tick); root.after_cancel(job)

w.winfo_width(); w.winfo_height()                   # size (after layout)
w.winfo_rootx(); w.winfo_rooty()                    # screen position
w.winfo_children(); w.winfo_toplevel(); w.winfo_exists()
root.update_idletasks()                             # finish layout now

from tkinter import messagebox, filedialog, simpledialog
messagebox.askyesno("Quit?", "Really quit?", parent=root)   # True/False
filedialog.askopenfilename(filetypes=[("Text", "*.txt")])   # "" on cancel
simpledialog.askinteger("Age", "Age?", minvalue=0)          # None on cancel

root.clipboard_clear(); root.clipboard_append("copied")
text = root.clipboard_get()
root.protocol("WM_DELETE_WINDOW", on_close)
w.focus_set(); w.lift(); w.destroy()
```

## CODE EXPLAINED

- → Bindings: widget, app-wide and virtual events
- → Variables and timers are the main state tools
- → `winfo_*` queries need layout done first
- → Standard dialogs return values or cancel markers
- → Clipboard, close handling and focus helpers

## EVENT SEQUENCES

| SEQUENCE | WHEN |
|---|---|
| `<Button-1>` | Left click |
| `<Double-Button-1>` | Double click |
| `<B1-Motion>` | Drag |
| `<Key>` / `<Return>` | Keys |
| `<Configure>` | Resize |

## EVENT ATTRIBUTES

| ATTRIBUTE | HOLDS |
|---|---|
| `widget` | Source widget |
| `x, y` | Widget coords |
| `x_root, y_root` | Screen coords |
| `keysym / char` | Key name / text |
| `delta` | Wheel amount |

## CONTROL VARIABLES

| CLASS | TYPICAL USE |
|---|---|
| `StringVar` | Entry, Label, Combobox |
| `IntVar` | Spinbox, Radiobutton |
| `DoubleVar` | Scale, Progressbar |
| `BooleanVar` | Checkbutton |
| `trace_add` | React to changes |

## ⚠ COMMON MISTAKES

- Touching widgets from worker threads
- Blocking the loop with `sleep` or long loops
- Reading `winfo_*` before the window is drawn

## ✓ TIPS & BEST PRACTICES

- Bind shortcuts once on the root
- Use virtual events to decouple parts of the app
- Wrap dialogs in helpers that pass `parent=`

## QUICK SUMMARY

- `bind` + event attributes
- `after` for timing
- `winfo_*` for geometry
- Dialogs, clipboard, protocol